密　　级：公开

加密论文编号：

论文题目：基于大语言模型的测试用例自动生成技术研究及系统实现

学　　号：M202340074

作　　者：曹倩

专 业 名 称：软件工程

2025 年 12 月 25 日

115.25.60.50

基于大语言模型的测试用例自动生成技术研究及系统实现

Automated Test Case Generation With Large Language Models: Research and System Implementation

研究生：曹倩

指导教师：罗熊

北京科技大学数理学院

北京 100083，中国

Master Degree Candidate: Qian Cao

Supervisor: Xiong Luo

School of Mathematics and Physics

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

115.25.60.50

中图分类号：TP311　　学校代码：10008
U D C：　　密　　级：公开

北京科技大学硕士学位论文

论文题目:基于大语言模型的测试用例自动生成技术研究及系统实现

作者：_____曹倩_____

指 导 教 师：罗熊　单位：北京科技大学　职称：教授

指导小组成员：_____　单位：_____　职称：_____

_____　单位：_____　职称：_____

论文提交日期：2025 年 12 月 25 日

学位授予单位：北 京 科 技 大 学

摘 要

随着软件系统规模与业务复杂度持续提升，高效生成高质量的测试用例成为关键挑战。传统人工编写模式存在成本高、周期长、效率低的弊端，现有的自动化与半自动化方法存在领域适应差、可执行性不足、难以直接从自然语言需求生成用例等问题。大语言模型的快速发展，为从自然语言需求直接生成结构化、可执行的测试用例提供了新可能。本文围绕“提示工程-模型微调-系统实现”三个层面，提出并设计了一套基于大语言模型的软件系统测试用例自动生成方法：

(1) 在提示工程层面，通过融合模板化结构、少样本示例、思维链推理与自适应机制，构建多策略协同提示方案，引导大语言模型生成结构规范、覆盖全面的高质量测试用例。实验表明，其综合得分较零样本提示基线提升约 28.7%，在结构规范性、场景覆盖全面性与功能正确性方面均有提升；

(2) 在模型微调层面，构建了测试领域数据集，通过采用量化低秩适配（Quantized Low-Rank Adaptation，QLoRA）技术对 Qwen3-4B 大语言模型进行参数高效微调。微调后模型相较于原始模型，在自动化 BERTScore 评价中的加权聚合得分由 69.65% 提升至 73.58%，整体提升约 5.6%，增强了对领域术语、复杂业务规则及边界条件的理解与生成能力；

(3) 在系统实现层面，设计并实现了一个需求驱动的测试用例自动生成系统，实现了从需求到测试用例的闭环流程。

综上，本文提出的端到端生成方案为大语言模型驱动的测试用例生成提供了“方法-数据-系统”一体化解决路径，对推动软件测试智能化和自动化发展具有一定的理论意义和实践价值。

关键词：测试用例自动生成；大语言模型；提示工程；QLoRA 微调

Automated Test Case Generation With Large Language Models: Research and System Implementation

ABSTRACT

With software systems growing in scale and complexity, efficient generation of high-quality test cases has emerged as a critical challenge. The traditional manual writing mode has drawbacks such as high cost, long cycle, and low efficiency, while existing automated and semi-automated methods suffer from issues including poor domain adaptability, insufficient executability, and difficulty in generating test cases directly from natural language requirements. The rapid development of large language models (LLMs) has provided new possibilities for directly generating structured and executable test cases from natural language requirements. Focusing on three dimensions of prompt engineering, model fine-tuning and system implementation, this paper proposes and designs a large language model-based automatic test case generation method for software systems.

(1) At the prompt engineering level, a multi-strategy collaborative prompting scheme is constructed by integrating templated structures, few-shot examples, thought chain reasoning, and adaptive mechanisms. This guides the large language model to generate high-quality test cases with standardized structures and comprehensive coverage. Experiments show that its overall score is approximately 28.7% higher than the zero-shot prompting baseline, with improvements in structural standardization, comprehensive scenario coverage, and functional correctness.

(2) At the model fine-tuning level, a test domain-specific dataset was built, and the Qwen3-4B large language model (LLM) was efficiently fine-tuned via the Quantized Low-Rank Adaptation (QLoRA) technique. Post-fine-tuning, the model's weighted composite score in automated BERTScore evaluation increased from 69.65% to 73.58%, representing an overall improvement of approximately 5.6%. This enhancement strengthens the model's capabilities in understanding and generating domain-specific terminology, complex business rules, and boundary conditions.

(3) Regarding system implementation, a requirements-driven automated test case generation system was designed and implemented, establishing a closed-loop workflow from natural language requirements to structured test cases. In conclusion, the proposed end-to-end framework provides an integrated "methodology-data-system" solution for LLM-driven test cases generation, offering certain theoretical and practical value for advancing intelligent and automated software testing.

Key Words: Automated Test Case Generation; Large Language Models; Prompt Engineering; QLoRA Fine-tuning

目 录

摘 要 I
ABSTRACT II
1 绪论 1
1.1 研究背景 1
1.2 研究意义 2
1.3 国内外研究现状 3
1.4 研究内容 7
1.5 论文组织结构 8
2 大语言模型相关技术 10
2.1 提示工程 10
2.2 微调 11
2.3 本章小结 16
3 面向测试领域的提示工程设计 17
3.1 提示工程的设计 17
3.2 实验设计 20
3.3 实验评估方法 23
3.4 实验结果与分析 25
3.5 本章小结 29
4 面向测试领域的微调方法 31
4.1 数据集的构建与处理 31
4.2 微调方案设计 38
4.3 实验设计 44
4.4 实验评估方法 45
4.5 实验结果与分析 48
4.6 本章小结 51
5 系统设计与实现 52
5.1 需求分析 52
5.2 架构设计 55
5.3 数据库设计 56
5.4 系统实现 60

5.5 本章小结 84
6 系统测试 85
6.1 功能性测试 85
6.2 非功能性测试 90
6.3 本章小结 92
7 结论与展望 93
7.1 结论 93
7.2 展望 94
参考文献 95
附录 A 提示词 102
附录 B 需求文档 106
附录 C 测试用例及评分 109
致 谢 145
作者简历及在学研究成果 146
独创性说明 147
关于论文使用授权的说明 148
学位论文数据集 149

1 绪论

1.1 研究背景

软件测试伴随着软件的发展而产生。1973 年，Bill Hetzel 博士将软件测试定义为“软件测试就是对程序能够按预期的要求运行建立起一种信心”。1983 年，电气电子工程师协会（Institute of Electrical and Electronics Engineers, IEEE）提出新定义：“使用人工或自动的手段来运行或测量软件系统的过程，目的是检验软件系统是否满足规定的要求，并找出与预期结果之间的差异”。要完成此过程，测试用例（Test Case, TC）就是一个必不可少的手段。测试用例实际上是对软件运行过程中所有可能存在的目标、运动、行动、环境和结果的描述，是对客观世界的一种抽象[1]，是指在软件测试过程中，为验证特定功能或特性而设计的文档化步骤和预期结果的集合。其优点是能明确需求验证，可以结构化、系统化地管理测试过程，其文档化的形式有助于共享，而且还可以支持回归测试。

在软件开发领域，软件测试对保障产品质量至关重要，其核心原则之一便是强调“尽早且持续地进行软件测试”。测试活动不仅应覆盖程序执行阶段，还应贯穿整个软件定义与开发的生命周期。近年来，“测试左移”作为一种前沿的软件测试方法论日益受到关注。该理念倡导将测试活动前置到需求分析、系统设计等项目早期阶段，旨在尽早识别并修复潜在缺陷，进而有效降低后期修复成本，提升软件整体质量。为响应这一趋势，越来越多企业要求测试人员在需求评审完成后立即同步启动测试用例编写工作，以确保在开发生命周期内持续嵌入质量保障机制，进而有效控制项目风险。测试用例在测试流程中的核心地位不可替代：它既是测试执行的行动指南，也是测试结果评估的客观依据，更在复杂系统中承担着验证功能正确性与完整性的关键角色。其重要性，恰如设计文档之于开发人员。缺乏清晰、完备的测试用例，测试活动将失去方向与标准。正因如此，即便面临时间压力，测试人员仍需要投入大量精力，编写详尽、结构化的功能测试用例，以保障质量底线。这种现状容易引发两类问题：一是由于时间紧迫而疏忽了对关键测试点的全面覆盖，从而降低 TC 的质量，致使测试执行流于表面，未被识别的潜在缺陷可能引发线上故障、用户体验受损等连锁问题，最终削弱项目交付效率、产品稳定性及市场竞争力；二是测试人员将大量精力

投入到编写测试用例中，限制了测试人员在其他方面提升自我能力的机会，不利于其专业发展和发挥主观能动性。

在这一背景下，实现自动生成测试用例是非常必要的。尽管当前研究在测试用例自动生成方面取得了一定进展，但对基于需求自动生成测试用例的研究仍显不足。例如，现有方法往往依赖于结构化的输入，如 UML（Unified Modeling Language）模型、形式化规约或源代码，难以直接从非结构化或半结构化的需求文档（如需求文档、产品需求说明书）中准确提取测试意图并生成符合业务逻辑的测试用例；同时，多数自动化方法缺乏对领域术语、业务规则和边界条件的深入理解，导致生成的测试用例语义失真、覆盖不全或可执行性差。针对这些局限性，亟需一种能够直接理解自然语言需求、准确捕捉业务语义并生成结构化测试用例的新范式。近年来，伴随着人工智能技术的进步，大语言模型（Large Language Model，LLM）在文本生成与理解方面展现出强大能力，已广泛应用于代码生成、文档辅助编写等任务。利用 LLM 解析需求文档与设计规范，并自动生成测试用例，已成为一个具有潜力的研究方向。此外，LLM 支持通过持续学习优化输出质量，从而更好地适配实际项目需求。与基于模型的测试（需先构建抽象模型）、符号执行（依赖源代码）或遗传算法（需定义适应度函数）等传统方法相比，LLM 显示出了更低的前期成本与更高的灵活性，其在多项自然语言处理任务中的成功应用，表明它在测试用例自动生成中的潜力巨大。深入研究 LLM 在测试用例自动生成中的应用，可以推动这一前沿技术在软件测试领域的应用和发展。

因此，针对上述现状，本文意在探索和研究大语言模型技术在测试领域的具体应用，为测试用例的自动生成和优化提供新的解决思路，进而帮助测试人员能够迅速响应并高效完成测试任务，最终提高项目的整体质量和交付效率。

1.2 研究意义

在软件开发领域，我们见证了从机器语言、汇编语言至结构化编程语言，再到面向对象编程的飞跃性发展。这一系列的变革不仅极大地提升了开发人员的编程能力和效率，还推动了开发工具的不断进步，使编码速度和调试效果均实现了显著提升。然而，在软件测试领域，当前测试工作仍高度依赖手工操作，测试用例的编写也大多由测试人员手工完成，这在一定程度上限制了测试效率和质量的提升。在此背景下，开展本文研究具有

积极意义：

(1) 经验不同的测试人员写出来的测试用例可能质量不一。如果采用自动生成测试用例的方式，可以大大减少由于水平不一或者人为错误带来的测试质量问题。

(2) 传统的测试用例编写依赖手工操作，费时且重复性高。自动生成测试用例能帮助测试工程师大幅减少编写时间，显著提升效率，尤其在快速迭代开发环境中更具价值。此外，自动生成的测试用例可以快速适应需求变更，减少因需求变更而导致的测试用例重写工作。

(3) 自动化生成测试用例将测试人员从繁琐的手动编写任务中解放出来，使他们能够专注于更高层次的测试策略和覆盖率优化，从而更好地保障项目的可靠性和整体质量。

(4) 本文在已有研究基础上，进一步探索大语言模型在基于自然语言需求生成结构化测试用例中的应用，结合提示工程与领域微调技术，提升生成结果的语义准确性、业务逻辑一致性与格式规范性；并在此基础上开发了一个原型系统，以验证所提方法的可行性与实用性。该工作有助于推动大语言模型在软件测试领域的工程化落地，为测试用例自动生成提供更具实用价值的解决方案。

1.3 国内外研究现状

1.3.1 研究进展

在自动生成测试用例的方法中，可以根据技术的发展和演变将其分为四个主要阶段。

(1) 基于符号执行、模型驱动与遗传算法的早期方法及其局限

在 2011 至 2015 年间，动态符号执行、模型驱动方法以及以遗传算法为代表的元启发式算法成为主流。这些方法通过符号执行分析程序路径、利用模型描述系统行为或通过进化算法模拟自然选择过程，旨在生成有效的测试用例。但是，动态符号执行面临状态空间爆炸的问题，对于复杂程序，生成的路径数量迅速增加，导致计算开销显著上升。不过，在 2014 年董齐兴[2]针对上述问题提出一种基于过程间控制流图的启发式搜索策略，该策略通过计算各分支后继未覆盖分支数的近似期望值来指导搜索过程，进而生成测试用例。不过论文中提出的搜索策略在处理带有不可达代码（死

代码）的程序时，可能会试图生成覆盖这类不可达代码的测试用例，这或许导致无法顺利生成测试用例。刘扬等人[3]提出了一个基于模型驱动架构的测试用例生成框架，该框架包括平台无关模型到平台无关测试模型（Platform-Independent Test, PIT）的水平转换，以及 PIT 到平台相关测试模型和测试用例的竖直转换。该框架能够自动化地从系统模型生成测试用例。但是模型驱动方法的构建成本相对较高，准确、全面的模型需要深入理解系统，这可能耗费大量时间和资源。周瑞杰和江国华[4]在借助遗传算法进行测试数据生成策略基础上，对已有染色体编码方式和遗传变异策略进行改进，引入类方法调用依赖图和方法调用树模型表示方法间的依赖关系，从而实现对类进行单元测试。遗传算法虽然在优化测试用例方面具有优势，但也存在收敛速度慢的问题，可能无法及时找到高质量的测试用例。其性能往往对参数设置（如种群大小和变异率）高度敏感，不当的参数配置可能显著影响算法效果。此外，遗传算法容易陷入局部最优解，而难以找到全局最优解。

(2) 关键点路径与粒子群优化的测试用例生成技术

在 2016 至 2018 年，关键点路径方法及粒子群优化进一步占据主导地位。路径覆盖测试是一种针对白盒测试的常用充分性准则，要求程序的运行覆盖所有完整路径[5]。丁蕊等人[6]在 2016 年提出了一种以关键点路径为基础的快速测试用例自动生成方法，通过控制流图转化得到关键点图，同时定义关键点与关键点路径表达式，来计算程序的理论路径数目，并快速找出已覆盖路径的邻近路径。这种方法虽然可以有效识别程序中的重要路径，但其局限性在于只能针对特定的关键路径进行测试用例生成，这可能导致某些边界情况和次要路径未被覆盖，从而影响整体测试的全面性。粒子群优化[7]是一种受鸟群社会行为启发的群体智能算法，通过个体间协作共享位置与速度信息以协同搜索最优解。在 2017 年，包晓安等人[8]基于粒子群算法，提出了一种自适应调整惯性权重的方法，该算法能够根据粒子的优劣自适应调整惯性权重，提高了算法的适用能力和性能。

(3) 形式化模型与深度学习/强化学习辅助的测试用例生成

2019 至 2020 年，基于形式化模型以及基于深度学习和强化学习的测试用例生成方法开始出现，这些方法通过学习历史测试用例的模式或与环境的交互，自动生成更加高效的测试用例。此处基于模型[9]指的是传统意义上的模型驱动测试（Model-Based Testing, MBT），即通过对被测软件的预期行为建立抽象的形式化描述（如状态转换模型），并从中系统化地派生测

试用例。目前用于 MBT 的典型模型包括 UML 状态图[10]、FSM（Finite State Machine）有限状态机以及马尔可夫链模型[11]。栾家伟等人[12]在 2020 年研究了基于 UML 模型中的状态图实现测试用例的自动生成。首先根据需求规格说明所建立的 UML 模型的状态图，提取对应信息将其转化为有向图，然后采用深度优先算法遍历有向图实现路径覆盖策略，以及利用贪心算法和回溯法实现语句覆盖策略。

(4) 大语言模型驱动的测试用例智能生成技术演进

2021 至 2025 年，模糊测试[13]、符号执行与机器学习结合[14]以及基于 LLM 的方法逐渐兴起，利用随机输入、智能算法以及自然语言处理技术，推动了测试用例生成的智能化和自动化进程。Mathur 等人[15]利用 T5 和 GPT-3 模型，从对话中提取上下文和主题，自动生成测试用例。该方法首先对数据进行预处理，过滤掉不必要的信息，然后利用 T5 模型将自然语言处理问题转换为文本到文本的格式，并使用教师强制方法进行训练。通过 YAKE 提取器从预处理的文本中提取上下文和主题，识别关键词。使用句子片段库对上下文进行标记化，提取相关句子，并将其作为输入提供给 T5 模型进行序列到序列编码。最后使用 GPT-3 模型的自然语言生成能力，根据 T5 模型的输出进一步提炼和生成测试用例。该文提出了一种新的方法来从对话中提取上下文和关键词，实现测试用例的自动生成，减少了对测试人员的依赖。但是由于两个模型要求的性能很高，当需要生成大量测试用例时，模型可能无法产生足够准确或数量的输出。Sungmin Kang 等人[16]研究了 LLM 在通用漏洞复现中的应用。提出 LIBRO（LLM Induced Bug ReprOduction）框架，利用 LLM 生成可能复现错误的测试用例。通过自动化测试用例的生成，减少了开发人员在测试阶段的工作量，提高了软件开发的效率。Tajmilur Rahman 等人[17]研究了如何利用大型语言模型从需求文档中自动生成用户故事（User Story, US）。提出了一种名为“Refine and Thought（RaT）”的提示技术。这是一种改进的思维链（Chain-of-Thought, CoT）方法，分为两个步骤：精炼步骤和思考步骤，通过过滤冗余信息和无意义的符号来精简输入文本，再通过逐步引导模型从复杂的需求文档中提取出有价值的信息，从而提高 LLM 处理复杂文本的能力，但是该篇论文只研究了提示技术对大语言模型的影响并未提及微调方法。杨浠等人[18]提出基于人工智能生成内容（Artificial Intelligence Generated Content, AIGC）的 CodeQwenTest 方法：用 LoRA（Low-Rank Adaptation）微调 CodeQwen1.5-7B-Chat，在 GitHub 焦点数据集上训练，自动生成 Java 单元测试，显著提升行覆盖率与断言质量。只是该研究仅针对 Java 语言验证，数据集规模与多

样性有限，且仍存在少量语法幻觉和分支覆盖不足问题。李博诺等人[19]提出了基于大语言模型的全自动生成方法方法：先由大语言模型生成 JSON 格式的输入输出对，再通过多智能体交互与静态分析相结合的修复流程提取并生成针对 C 函数的断言，实现了测试用例与断言的自动化生成并显著提升了覆盖率。但是该方法在断言生成的准确性上表现有限，所有模型的预言正确率最高仅为 50.05%，距离实际应用仍有较大差距，需进一步优化生成断言的正确率。此外，该方法本质是利用大语言模型辅助生成单元测试用例，而并非基于需求生成测试用例。曹鹏等[20]提出了一种针对单元测试生成的大语言模型高效微调方法，通过解耦功能风格与编码风格并设计增量权重训练策略实现同步迁移，有效提升了生成测试用例的编译率和覆盖率。然而，该方法仍存在计算流程相对复杂、任务分组策略需要精细调优以及对跨语言及项目类型的泛化性验证不足等局限性。Johnsson[21]以 LLM 为基础，结合检索增强生成和提示工程技术，遵循设计思维流程实现测试用例自动生成，并通过对比人工智能（Artificial Intelligence, AI）生成结果与人工测试用例评估系统性能。但是该研究结果表明该实现方案虽能生成格式正确且在一定程度上有效的测试用例，但边缘用例覆盖不足、生成结果存在噪声，同时受限于知识库数据量，对复杂项目适配性待提升。

1.3.2 综述分析

随着时间的推移和技术的发展，自动生成测试用例的方法不断迭代，不断推陈出新。但总的来说，测试用例生成技术分为白盒测试用例生成技术和黑盒测试用例生成技术。白盒方向侧重程序结构与路径信息，代表性方法包括动态符号算法和关键点路径方法等，其优势在于可解释性强、对代码语义把握精细。黑盒方向则以需求文档或规格说明文档为中心，典型方法包含基于模型测试用例生成技术、遗传算法以及粒子群优化等元启发式算法等，其优势在于与实现解耦、对复杂系统可扩展性较好。这一系列的发展反映了自动生成测试用例方法在理论和实践上的不断进步，为软件测试的效率和质量提升提供了有力支持。但是当前研究仍存在一些局限性。例如，动态符号执行和路径探索算法在复杂程序中会产生大量路径，导致计算开销过高。即便引入启发式搜索策略或优化算法（如基于控制流图的启发式方法），也难以有效处理不可达代码。而基于模型的方法需要深入了解系统，构建准确的模型，这具有很高的门槛；同时，其建模成本与维护成本显著，且模型与实现偏差会影响用例有效性。遗传算法和粒子群优

化算法虽然能寻找最优解，但收敛速度慢、容易陷入局部最优的问题，且对参数设置敏感。关键点路径方法只能生成特定路径的测试用例，难以全面覆盖所有情况。近年来，大语言模型为“需求→结构化测试用例”的直接生成提供了新思路，初步研究展示出在语义理解与场景扩展方面的潜力。然而，其对领域术语与业务规则的语义保真性不足，易产生事实性幻觉或关键边界条件遗漏；同时，生成结果在结构上难以严格遵循企业规范（如标题、前置条件、操作步骤与预期结果的一一对应），导致可执行性受限。针对测试领域的系统化提示工程与领域微调结合研究仍相对匮乏。

因此，为了进一步提升测试用例自动生成的效果，有必要深入探索大语言模型在软件测试中的应用路径。结合提示工程与微调技术，优化模型的生成能力，以及将大语言模型技术与 Web 开发技术结合，探索智能化、自动化生成测试用例的解决方案，这将有助于实现更高效、精确的测试过程，进一步推动测试用例生成的智能化水平。

1.4 研究内容

针对人工编写测试用例效率低下、成本高昂的问题，本文致力于探索大语言模型在测试用例自动生成中的应用。尽管以 ChatGPT[22]为代表的大语言模型在通用自然语言处理任务中表现优异，但在软件测试这一特定领域，尤其是在需求驱动的测试用例生成任务中，仍存在生成结果准确性不足、结构不规范、可执行性差等问题。基于上述背景与现存问题，本文核心研究内容如图 1-1 所示，涵盖以下三个层面：

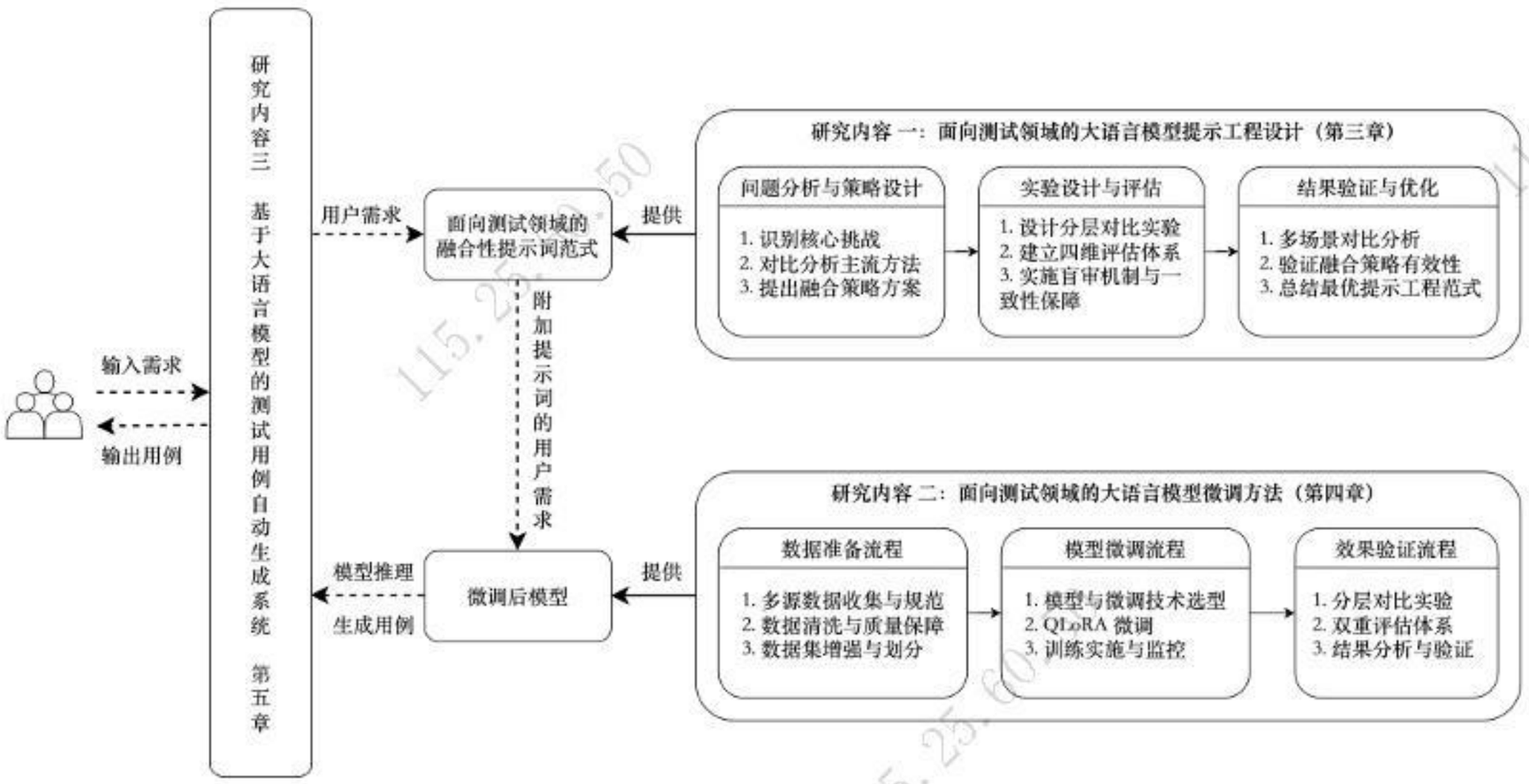


图 1-1 核心研究内容

Fig.1-1 Core Research

(1) 在提示工程层面，核心任务是构建一套融合性的提示新范式，以系统化地解决测试用例生成中的领域适应问题。通过融合思维链、少样本示例、模板化输出与自适应机制四种提示技术，旨在引导模型生成结构规范、覆盖全面、可直接执行的高质量测试用例。

(2) 在模型微调方面，将构建高质量、结构化的测试领域微调数据集，并采用参数高效微调技术对基座模型进行领域适配。实现对模型的领域知识注入与生成风格控制，进而增强其对需求文档中领域术语、复杂业务规则及边界条件的理解能力。在此基础上，本文将通过对照实验，系统验证提示工程与参数高效微调的协同增益效果。

(3) 针对当前缺乏测试用例生成专用成熟系统的现状，本研究将在前述理论研究的基础上，设计并实现一个面向需求的测试用例自动生成系统，同时通过实证评估验证其有效性与可行性，从而切实满足测试工程师的实际需要。

1.5 论文组织结构

本文的组织结构如下，具体如图 1-2 所示。

第 1 章为“引言”，主要阐述研究背景、研究意义，系统梳理国内外在大语言模型提示工程、微调及测试用例生成领域的研究现状，并进行述评。同时对研究内容进行整体说明。

第 2 章为“大语言模型相关技术”，介绍提示工程与微调的基本概念与关键方法，为后续研究奠定理论与技术基础。

第 3 章为“面向测试领域的提示工程设计”，详细说明提示设计策略、实验设计与评估方法，结合实验结果展开分析与总结。

第 4 章为“面向测试领域的微调方法”，介绍数据集构建、微调方案与实验结果分析，探讨其在测试用例生成任务中的适配性。

第 5 章为“系统设计与实现”，本章围绕基于大语言模型的测试用例自动生成系统，介绍从需求分析到系统实现的整体过程。

第 6 章为“系统测试”，通过功能性测试与非功能性测试，从正确性、稳定性、兼容性和可用性等角度对系统进行全面验证。

第 7 章为“结论与展望”，总结全文工作，归纳提示工程与微调结合在需求驱动测试用例生成中的主要成果与贡献，同时指出当前研究的不足。并对未来在需求理解、生成规范化、跨场景适应性及系统化评估等方向的

进一步研究进行展望。

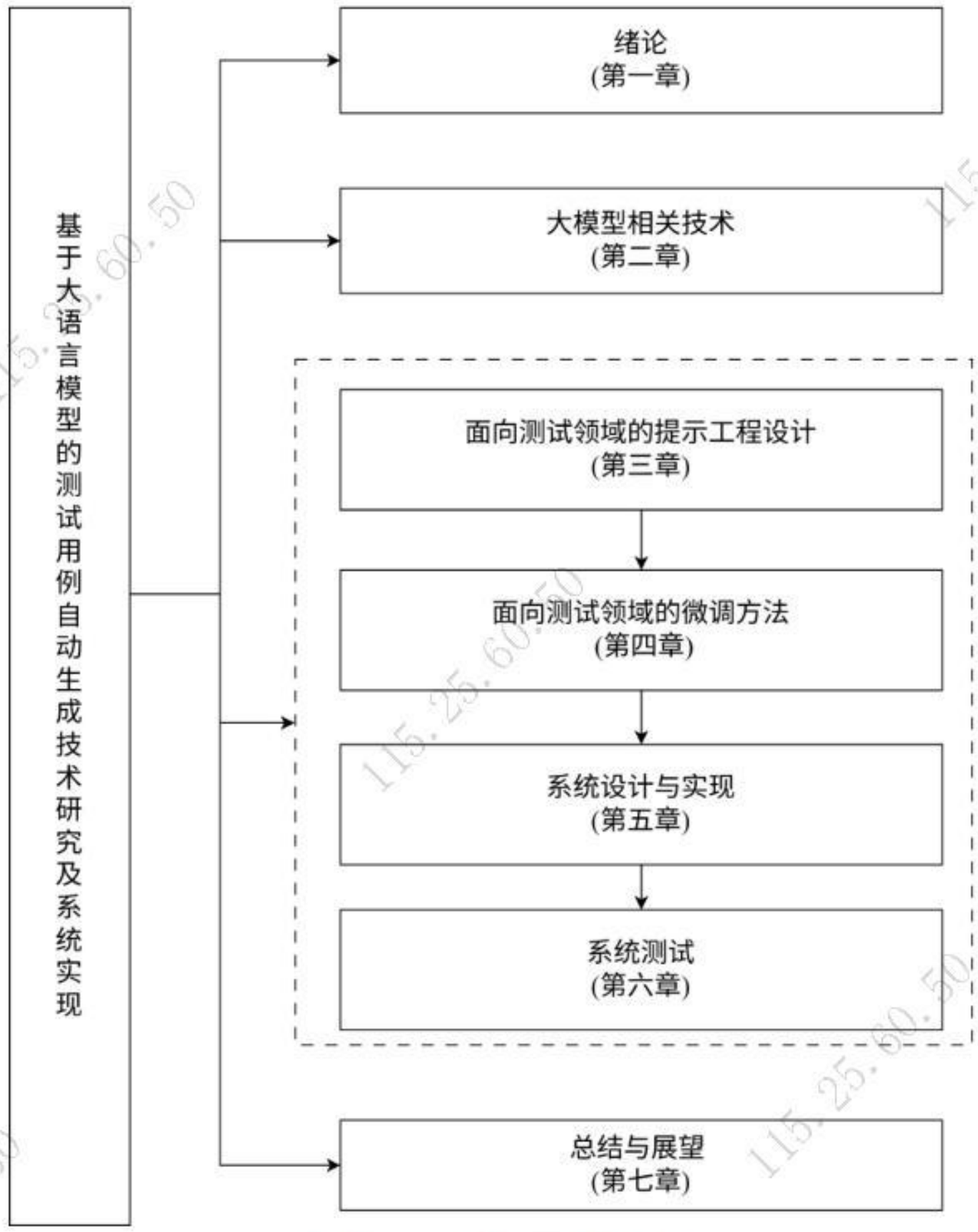


图 1-2 本文组织结构图

Fig.1-2 Organization Structure Diagram

2 大语言模型相关技术

2.1 提示工程

2.1.1 概念与作用

Samir Lemeš[23]提出提示工程是一项完整流程：设计、测试并优化发给 AI 的提示，目标是使模型生成最相关、准确、实用的答案；这需要理解模型的解读方式，并针对不同应用进行调整。这意味着与 LLM 交互通常涉及向其提供“提示”[24]。提示是提供给 LLM 的一组指令，通过这些指令，我们可以自定义 LLM 的行为或增强其功能，从而影响它生成的输出内容。Sahoo P 等人[25]强调：提示工程已成为不可或缺的技巧，通过任务特定的指令，无需调整模型参数，就能驱动预训练模型表现所需行为，广泛适用于各种下游任务。黄峻等人[26]也论述了提示工程在提升大语言模型性能中的重要作用。

2.1.2 提示工程的技术方法

提示工程作为提升大语言模型性能的重要手段，已逐渐形成一套系统的方法论。根据 OpenAI 官方技术指南[27]，提示设计可遵循以下六种常见策略：

(1) 编写清晰明确的说明，避免歧义和模糊描述；

(2) 提供参考文本，以帮助模型减少虚构或“幻觉”内容；

(3) 将复杂任务拆解为更简单的子任务，降低模型推理难度；

(4) 给予模型“思考时间”，即在提示中引导其逐步推理；

(5) 结合外部工具（如检索增强生成），提升答案的可靠性与时效性；

(6) 系统地测试与修改提示，通过迭代优化获得更优输出。

除了上述基本策略外，研究者们还提出了多种增强提示的方法。例如，CoT 技术能够显式引导模型逐步推理，从而提升复杂任务的解答效果[28][29]；范围限定则通过明确约束模型输出的范围与格式，以减少不必要的偏离。此外，White 等人[30]提出了一套提示模式目录，其中包括元语言创建、输出自动化器、问题细化等模式。通过标准化的语法和符号体系，用户可以更精准地指导模型生成所需输出，降低歧义与错误率。

在提示工程的应用中，以下几种基本范式也被广泛使用：零样本提示（Zero-shot Prompting）[31]无需示例，直接通过自然语言指令引导模型完成任务；少样本提示（Few-shot Prompting）[32]是在提示中提供少量任务示例，以提升模型在特定任务上的表现；思维链提示通过显式推理步骤，引导模型逐步完成复杂推理任务。

2.2 微调

2.2.1 概念与作用

随着预训练大语言模型的广泛应用，如何将其有效适配到具体下游任务成为关键研究问题。微调是指在大规模预训练模型的基础上，结合特定任务数据进行参数更新，以提升模型在目标任务上的表现[33]。与从零开始训练模型相比，微调能够显著降低计算成本与数据需求，同时避免模型在有限数据上出现过拟合[34]。

微调的核心思想在于，大语言模型通过大规模语料的预训练已经学习了通用的语言模式与知识结构，而下游任务往往具有特定的输入输出模式或领域知识需求。通过微调，可以在保持模型通用能力的同时，使其具备更强的任务适配性[35]。例如，在自然语言处理中，BERT（Bidirectional Encoder Representations from Transformers）等模型通过在问答、文本分类等任务上进行微调，显著超越了传统方法的性能[36]。

此外，微调不仅提升了模型的任务性能，还具有以下作用：

(1) 领域适应：将通用模型迁移至医疗、法律、金融等专业领域，提高对领域专有词汇与语境的理解[37]。

(2) 效率优化：通过参数高效微调（Parameter-Efficient Fine-tuning，PEFT）方法，如 LoRA，可在减少更新参数量的同时保持性能，从而降低存储与部署成本。

(3) 鲁棒性增强：微调可针对特定场景的输入分布进行校正，提高模型在边界情况或噪声数据下的稳定性[38]。

(4) 可控性与个性化：通过指令微调与对齐技术，模型能够更好地符合人类偏好，提升交互体验与可解释性[39]。

因此，微调已经成为大语言模型落地应用的重要技术途径。它不仅弥补了预训练模型在特定任务上的不足，还通过高效的参数更新方法推动了

大语言模型的可扩展应用。

2.2.2 主流技术和方法

当前主流的微调技术通常可以分为两大类：全参数微调与参数高效微调。其中，全参数微调是指在下游任务数据上对预训练模型的全部参数进行更新，能够获得最优的适配效果，但其计算和存储成本极高，而且在数据量不足的情况下参数微调容易导致过拟合，因为模型可能会过度记忆小数据集的细节。相比之下，参数高效微调方法通过冻结大部分模型参数，仅在少量新增或重参数化的模块上进行训练，从而在显著降低资源开销的同时保持与全参数微调相近的性能。模型层间引入轻量化模块的适配器；通过低秩矩阵分解实现权重更新的低秩适应（LoRA）及其扩展形式QLoRA[40]；以及前缀调优（Prefix Tuning）与提示调优（Prompt Tuning）[41]，通过优化可学习的连续向量前缀或提示来引导模型行为。

此外，在大语言模型的对齐任务中，微调往往还会结合基于强化学习的人类反馈（Reinforcement Learning with Human Feedback, RLHF）等方法进行进一步优化，以使模型更好地符合人类的偏好和价值观，从而提升输出的可控性和安全性。Li 等人[42]提出 Prefix Tuning，这种微调方法是提示工程的延伸，在思路上与提示工程类似，都是通过提示引导模型执行任务。但是 Prefix Tuning 可以通过添加前缀参数或者修改前缀参数的形式，在模型的每一层加入一小段“提示”向量来引导模型生成内容。其原理如图 2-1 所示。

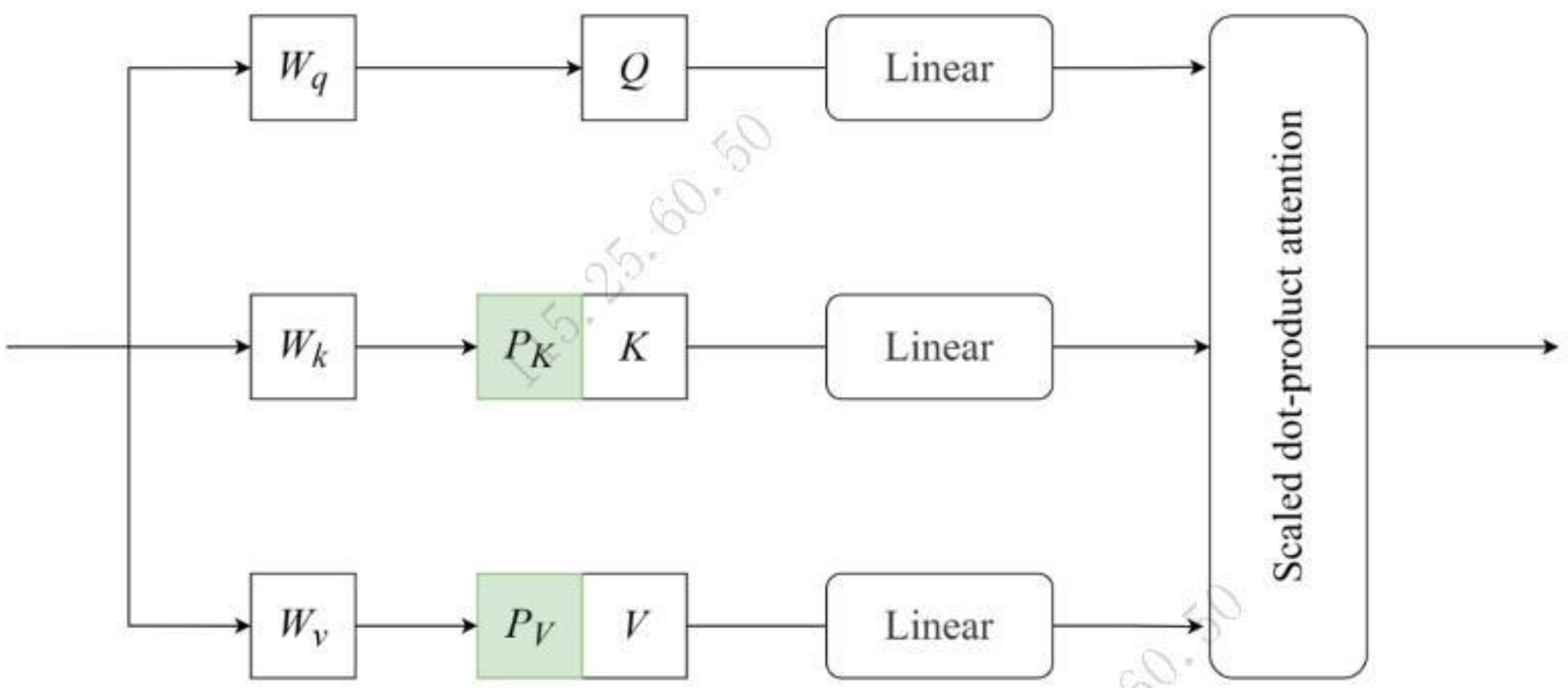


图 2-1 Prefix Tuning 原理图

Fig.2-1 Principle of Prefix Tuning

图中，W_q、W_k、W_v为权重矩阵，Q、K、V分别代表查询、键和值向量，P_k、P_v是两组 Prefix 向量。在 Prefix Tuning 中，输入向量首先通过权重矩阵转换成查询 Q、键 K 和值 V 矩阵。然后引入了前缀矩阵P_k、P_v分别加到 K、V 前，以此调整注意力机制。最后，经过线性变换和前缀调整的 Q、K、V 矩阵进入注意力机制模块，缩放点击注意力通过 Q 和 K 之间的点积计算相关性，并用来加权 V，从而输出注意力结果。

Adapter Tuning 采用增加参数的方法，Houlsby 等人[43]通过在预训练模型中每一层添加了适配器模块，微调时只需要微调这些适配器结构，保持预训练模型的原始参数不变。这种方法的好处是，既能让模型快速学会新任务，又不会因为大量调整原有参数而导致模型性能下降或者训练成本过高。Adapter Tuning 的一个重要优势是它的模块化设计。如果采用此微调方式，可以为每个不同的任务或领域设计不同的适配器层，比如 Web 应用测试、App 应用测试等，并根据任务需求动态加载这些适配器。每个适配器都是小型、可插入的模块，专门为某一任务训练和优化。所以这意味着模型能够在不同任务之间灵活适应，而无需重新训练整个模型，确保模型能够生成更加符合需求的测试用例。

LoRA 的核心思想是通过在模型的权重矩阵上引入一个低秩分解，只调整模型中少量的参数，而不是整个大语言模型的参数。所以它能够降低微调的资源开销。而且 LoRA 可以在提示工程的基础上，通过调整少量参数，实现对模型的内部特性的更为精细的控制，使测试用例更加精准。LoRA 的提出者 Hu 等人[44]认为参数更新量即便投影到较小的子空间中，也不会影响学习的有效性。因此，提出固定预训练模型参数不变，在原本权重矩阵旁路添加低秩矩阵的乘积作为可训练参数，用来模拟参数的变化量。即通过低秩分解来表示约束其更新[45]。LoRA 是一种高效的微调方法，通过分解和微调低秩矩阵，来减少微调过程中需要调整的参数数量。它能在大语言模型上微调少量参数，适应新的任务，如生成特定类型的测试用例。LoRA 的核心思想可通过公式(2-1)和公式(2-2)表现，原理如图 2-2 所示。

$$W=W_0+\Delta W \tag{2-1}$$

$$W_0+\Delta W=W_0+BA \tag{2-2}$$

其中，W 为给定的参数矩阵，W_0 为原始参数矩阵：$W_0 \in R^{d*k}$，ΔW 表示对原始权重矩阵W_0的一个增量更新，$\Delta W=BA$，$B \in R^{d*r}$，$A \in R^{d*r}$，$r<<min$

（d,k），其中 B、A 为低秩分解后的矩阵。训练时 W_0 被固定不再进行梯度更新，只训练 A 和 B。所以对于传输的 x，模型的前向传播过程见公式(2-3)。

$$h=W_0x+\Delta Wx=W_0x+BAx \tag{2-3}$$

通过仅训练 B 和 A 这两个小矩阵，模型能够高效地更新权重，而不需要对整个 W_0 进行大规模调整。

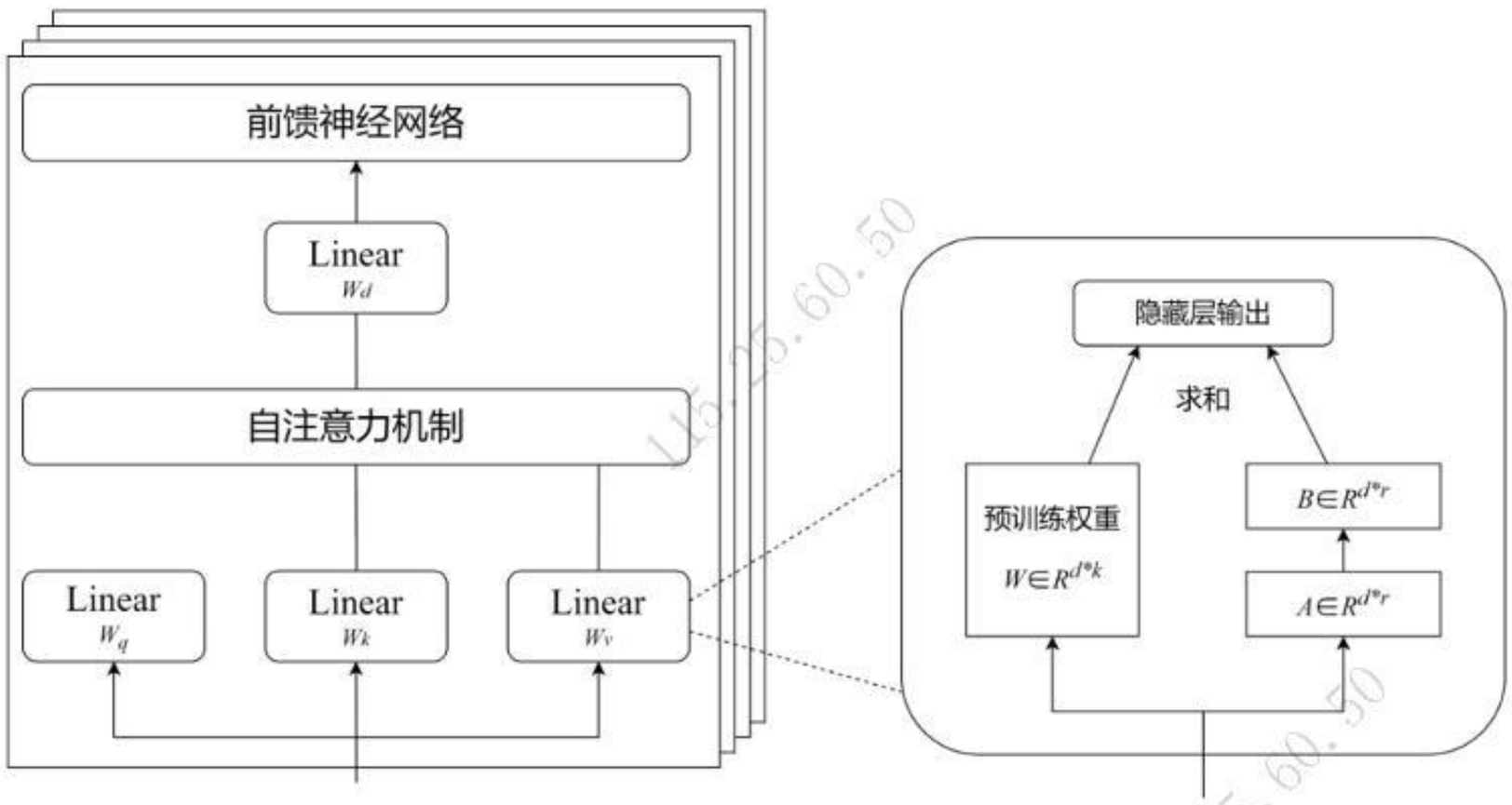


图 2-2 LoRA 微调示意图

Fig.2-2 Schematic Diagram of LoRA fine-tuning

而 QLoRA 是在 LoRA 的基础上将主干权重量化至 4-bit 并引入双重量化、分页优化器，从而在单张 48GB GPU 上微调 65B 级模型且接近 16-bit 全参性能。Full Fine-tuning、LoRA 和 QLoRA 的对比如图 2-3 所示。

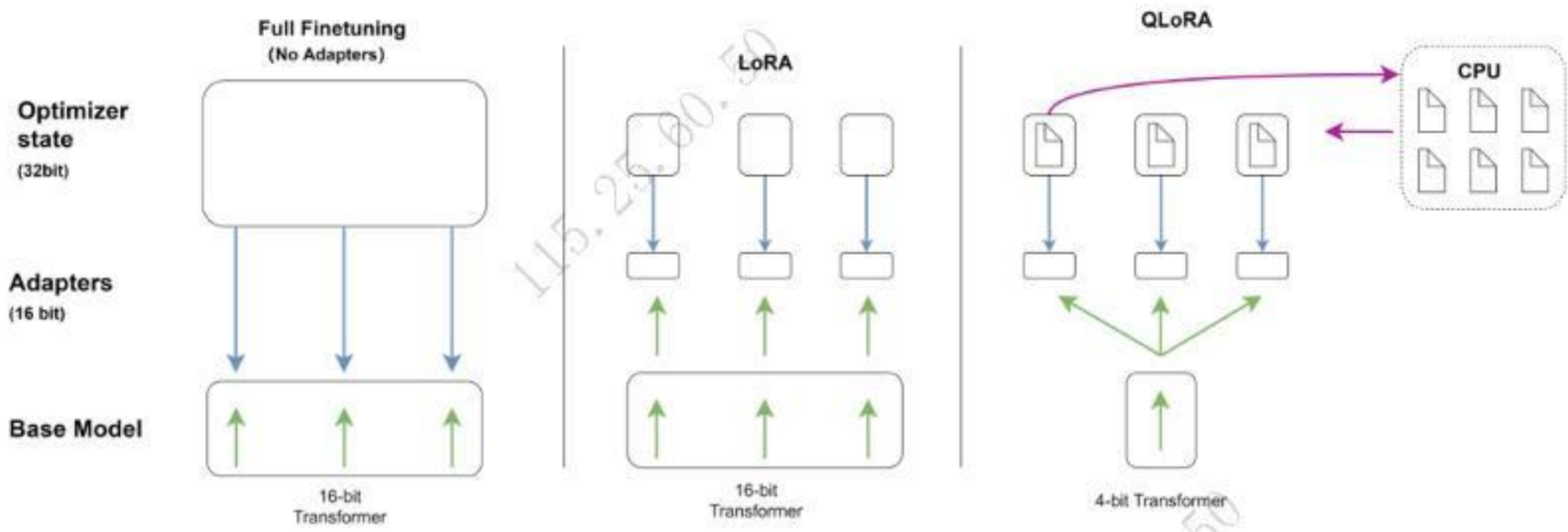


图 2-3 三种微调方法对比

Fig.2-3 Comparison of Three Fine-tuning Methods

上图展示了 Full Fine-tuning、LoRA 和 QLoRA 在训练过程中的关键组

件及其数据流动路径。蓝色箭头表示参数更新，即优化器根据梯度对模型参数进行调整的过程；绿色箭头表示梯度流动，反映反向传播过程中误差信号从输出层向输入层的传递路径；粉色箭头表示分页流，体现当显存不足时，部分模型权重或激活值在 CPU 与 GPU 之间进行交换以实现内存管理。在全参数微调中，所有基础模型参数均参与更新；在 LoRA 中，仅新增的低秩适配器被更新，而原始模型冻结；QLoRA 进一步引入量化和分页机制，显著降低显存占用，使得大语言模型可在消费级硬件上高效微调。

在大语言模型的高效微调研究中，Prefix Tuning、Adapter Tuning、LoRA 与 QLoRA 是目前常用的四种代表性方法。四种方法的原理和优点见表 2-1。

表 2-1 微调方法比较

Tab.2-1 Comparison of Fine-tuning Methods

微调方法	原理	优点
Prefix Tuning	在模型的每一层 Transformer 输入前添加一组可训练的前缀向量，这些前缀向量与模型的参数一同参与前向传播。只需训练这些前缀向量，而冻结预训练模型的其他参数。	1.减少训练开销 2.深度引导模型
Adapter Tuning	在预训练模型的每一层中插入小型的适配器模块，这些模块包含少量可训练参数，预训练模型的原始参数则被冻结。	1.模块化设计 2.减少了训练开销
LoRA	在模型的权重矩阵上添加一个可训练的低秩矩阵分解，只调整模型中少量的参数，而冻结其他权重。	1.降低计算资源 2.细粒度控制
QLoRA	在 LoRA 的基础上，将主干模型参数量化为 4-bit，同时引入双重量化和分页优化器，仅更新 16-bit 的低秩适配器矩阵。	1.显著降低显存需求，可在单卡 GPU 上微调超大语言模型 2.性能接近 16-bit 全参数微调 3.保留 LoRA 的灵活性和低存储开销

四种方法的共同点在于都避免了对全部参数进行更新，而是通过引入额外的可训练结构（如前缀向量、适配器模块、低秩矩阵分解或量化后的适配器矩阵），以降低计算与存储开销。但四者在实现机制与适用场景上存在显著差异：Prefix Tuning 通过可训练前缀向量对模型进行深度引导，训练成本极低；Adapter Tuning 采用模块化插入方式，具有良好的扩展性；LoRA 则在权重矩阵中引入低秩分解，实现细粒度控制与资源节约；QLoRA 进一步结合参数量化与 LoRA 结构，使得超大语言模型能够在单卡环境下完成微调，同时在性能上接近全参数微调。

2.3 本章小结

本章介绍了大语言模型应用中的两项关键技术：提示工程与微调。在提示工程部分，本章阐述了其概念、作用与主要技术方法。提示工程作为无需修改模型参数即可显著提升模型性能的重要手段，核心在于通过合理设计输入指令，引导模型生成更符合期望的输出。在微调技术部分，本章从概念、作用及主要方法三个层面进行了系统阐述。重点介绍了 Prefix Tuning、Adapter Tuning、LoRA 与 QLoRA 等代表性技术。

3 面向测试领域的提示工程设计

本章聚焦于引导 LLM 生成高质量测试用例的提示工程，将系统性阐述其设计、优化与验证方法：先分析测试用例生成任务的特有需求与主要挑战；再通过对比分析多种主流提示策略的优缺点，提出融合思维链、少样本示例、模板化与自适应机制的提示优化方案；最后通过多组对比实验验证提示策略的有效性，为系统实现提供了关键技术支持。

3.1 提示工程的设计

3.1.1 测试用例生成中提示工程的核心挑战

在基于大语言模型的测试用例自动生成研究中，提示词设计是影响生成质量的核心环节。针对从自然语言需求生成高质量测试用例的复杂任务，现有方法仍面临一系列系统性挑战。提示工程本身具有高度的不稳定性。研究表明，提示词的细微变动（如模板、格式或角色设定）会导致模型性能显著波动。这种提示敏感性使得生成结果在可执行性与场景覆盖率上难以保持稳定，在不同模型与提示策略间泛化能力较差[46][47]。生成多样性不足是另一关键瓶颈。模板化方法易导致输出陷入局部最优，产生大量语义重复或功能等价的用例，难以有效探索边界情况与异常路径，更大规模的模型甚至可能加剧该问题[48-50]。在关键场景覆盖上，缺乏覆盖导向机制的提示往往无法引导模型系统性生成针对边界条件、异常流程和多用例依赖关系的测试场景，导致生成内容在复杂需求上覆盖不足[51]。

此外，模型对领域知识及长上下文需求的理解仍存在明显缺陷。LLM 在处理冗长且具有跨条目约束（即需求文档条目间的相互依赖与关联要求，需结合多条目信息完整理解）的需求文档时，对中间信息的捕捉能力下降，易误解业务规则与系统约束，从而影响测试前置条件、数据构造及断言设计的准确性[52][53]。此外，生成结果的结构规范性与格式一致性往往不足，未经约束的解码过程可能导致 JSON/YAML 等结构化格式的语法错误，严重影响生成用例的可执行性与自动化集成[54][55]。

随着任务复杂度的提升，如从 US 到可执行脚本的多阶段推理需求，LLM 表现进一步受限。研究显示，当任务复杂度超过一定阈值，即便提供

思维链或分步指令，模型性能仍可能出现“完全性塌陷”[56][57]。生成过程中还普遍存在冗余与幻觉现象，具体表现为用例重复、断言薄弱或引用不存在的依赖组件，严重影响测试的有效性和真实性[58][59]。

综上所述，现有以直接指令式提示为主的方法在稳定性、多样性、关键场景覆盖、长文本理解和结构化输出等方面均存在不足，这些缺陷在复杂测试任务下会进一步放大，难以支撑高质量、高覆盖率的测试用例生成。由此可见，亟需探索更具针对性的提示工程方法。为此，本文在系统梳理与对比现有提示策略的基础上，提出一种融合式提示语设计，以期在逻辑完整性、结果稳定性和适应性之间取得平衡。

3.1.2 面向软件测试领域的提示工程设计

为了设计适用于软件测试领域的高效提示策略，本文首先对现有主流提示方法进行了全面的调研和比较，分析其原理、特点及优缺点，结果如表 3-1 所示。

表 3-1 提示工程方法对比

Tab.3-1 Comparison of prompt engineering methods

方法	原理	特点	优缺点
直接指令 (Direct Prompting)	直接给模型一个具体指令或问题，让它生成输出	只描述任务本身，无示例或额外结构化信息	优点：简单高效。 缺点：模型输出不稳定，覆盖不足，常出现遗漏或冗余。
零样本提示 (Zero-shot Prompting)	直接输入任务描述，不提供示例，依赖模型的通用知识完成任务	通常会尽量详细描述任务要求，让模型“凭说明就知道做什么”，简单高效，对提示依赖性低；但结果不稳定，覆盖率可能不足	优点：快速、泛化性较强。 缺点：质量依赖模型本身，结果往往不够贴合需求。
少样本提示 (Few-shot Prompting)	在提示中加入若干示例，引导模型模仿生成	稳定性和准确性更高，能保持结构一致性；需要示例集	优点：能显著提升格式一致性和输出稳定性。 缺点：示例过具体会导致示例污染；反之会导致覆盖不足。

表 3-1 续

Tab.3-1 Continued

方法	原理	特点	优缺点
思维链提示（CoT）	在提示中显式要求模型进行逐步推理	强化逻辑推理，适合复杂任务；但响应较长	优点：适合复杂任务，能显著提高逻辑覆盖率。 缺点：容易输出冗余的推理过程，不利于直接落地。
提示链（Prompt Chaining）	将复杂任务拆解为多步提示，分阶段完成	通过多个有序提示实现任务分解，偏重“过程结构化”	优点：过程清晰，适合系统化调用。 缺点：需要多轮交互，容易在链条衔接处丢失信息。
模板化提示（Template-based Prompting）	构建固定输入输出模板，要求模型填充	保证结果结构化，易于解析与落库；灵活性有限	优点：结果高度标准化，便于自动化处理。 缺点：限制了模型自由度，逻辑深度不足。
自适应提示（Adaptive Prompting）	根据反馈或生成结果动态调整提示词	可提升鲁棒性和泛化性；实现成本较高	优点：更接近真实测试人员的思维模式。 缺点：可重复性不足，不同运行可能得出不同复杂度判断。

基于上述比较，各方法在测试用例生成领域均有其适用场景与优势：Zero-shot 通过角色设定和任务说明，引导模型结合常用测试设计方法（如边界值法、场景法），并要求输出固定表格结构，但不提供任何示例或推理链。在测试用例生成中适合从自然语言需求直接生成基础测试用例，尤其是简单功能场景。Few-shot 在 Zero-shot 基础上引入若干示例，帮助模型模仿风格和覆盖方式，引导模型生成更符合预期格式的用例。CoT 进一步要求“先推理再生成”，更适用于覆盖边界值、异常场景等复杂测试逻辑生成。Prompt Chaining 用于分步生成测试用例：先抽取需求点，再生成用例，最后优化补充场景。Adaptive Prompting 可以根据需求复杂度动态选择策略，例如“简单需求生成核心用例，复杂需求分解模块并覆盖集成测试”。

然而，单一提示方法也存在明显局限：Direct Prompt 和 Zero-shot 虽然简洁高效，但在复杂推理任务中表现不稳定且泛化能力有限[60]；Few-shot 能提升格式一致性，却容易受到示例选择与呈现方式的限制[61]；CoT 有助

于深入推理[62]，但输出形式不够规整；Prompt Chaining 增强了过程可控性，但也带来链路搭建与调试的实现复杂度[63]; Template-based Prompt 能保证结果结构化与可解析[64][65]，却难以单独支撑复杂业务逻辑；Adaptive Prompt 具备灵活性，在不同上下文中的可复现性和一致性仍面临挑战。因此，仅依赖单一方法难以在逻辑完整性、格式稳定性与适应性之间取得平衡。

为突破这些局限，本文提出了融合 CoT + Few-shot + Template-based + Adaptive 的提示语设计：通过 CoT 保证推理深度和测试点覆盖；借助 Few-shot 提供参考示例、提升一致性与可控性；利用 Template-based 确保输出结构化；并融合 Adaptive 策略，根据需求复杂度动态调整测试方案。在该融合提示框架中，CoT 模板设计参考了 Tajmilur Rahman 等人[17]提出的 RaT 思想，结合软件测试的认知过程模型与真实的测试实践经验，最终设计出涵盖需求解构、测试策略映射、用例生成三阶段的结构化引导策略。

该融合方案在不同方法间形成互补，既保留推理与覆盖优势，又兼顾稳定性和灵活性，为复杂需求场景下的自动化测试用例生成提供更优解。融合提示策略结构见表 3-2，详细提示语内容见附录 A（表 A-1~表 A-6）。

表 3-2 融合提示策略

Tab.3-2 Integrated Prompt Strategy

方法	设计核心	内容示例（简化）	作用
Adaptive	根据功能点数量自动判断需求复杂度，动态决定生成核心用例或模块化集成场景。	简单需求：5–7 条 TC；复杂需求：模块化+集成+并发/状态转换。	根据复杂度动态调整覆盖范围。
CoT	引导模型执行需求分析、测试策略设计与用例生成的内在推理链。	内部处理流程：需求分析→测试策略→用例生成。	保证推理深度和测试点覆盖。
Few-shot	提供 1～2 个高度精简的参考用例，示范“场景合并”与“断言设计”的格式与粒度。	登录场景示例： 正常登录 vs 输入错误。	提供参考风格，保证一致性与可控性。
Template-based	强制要求以 Markdown 表格输出，并严格规定字段（用例编号、标题、前置条件、步骤、预期结果）。	输出固定 Markdown 表格格式，字段完整，对应关系清晰，预期结果应和测试步骤一一对应。	固化输出结构，确保结果可解析。

3.2 实验设计

(1) 实验环境

为验证提示工程策略的有效性，本实验构建了以下软硬件环境：硬件配置方面，搭载了 13 代 Intel Core i5 处理器、32 GB 内存，并配备 NVIDIA RTX 4060Ti 16GB 显卡。软件层面，操作系统 Ubuntu 22.04 LTS，配合 CUDA 12.3 并行计算平台和 Python 3.11.8 编程环境。所有提示策略均在统一的模型加载与调用框架下执行。

(2) 实验目标

本实验旨在系统评估不同提示优化方案在测试用例生成任务中的表现差异。通过对比实验结果，验证提示优化对生成质量的提升效果，确定最优提示策略，并为后续模型微调提供数据支撑与方法论依据。

(3) 实验流程

本实验流程遵循分层对比设计，具体包括需求选择、按照不同提示词生成测试用例、结果评估与分析四个核心阶段，其整体流程如图 3-1 所示。

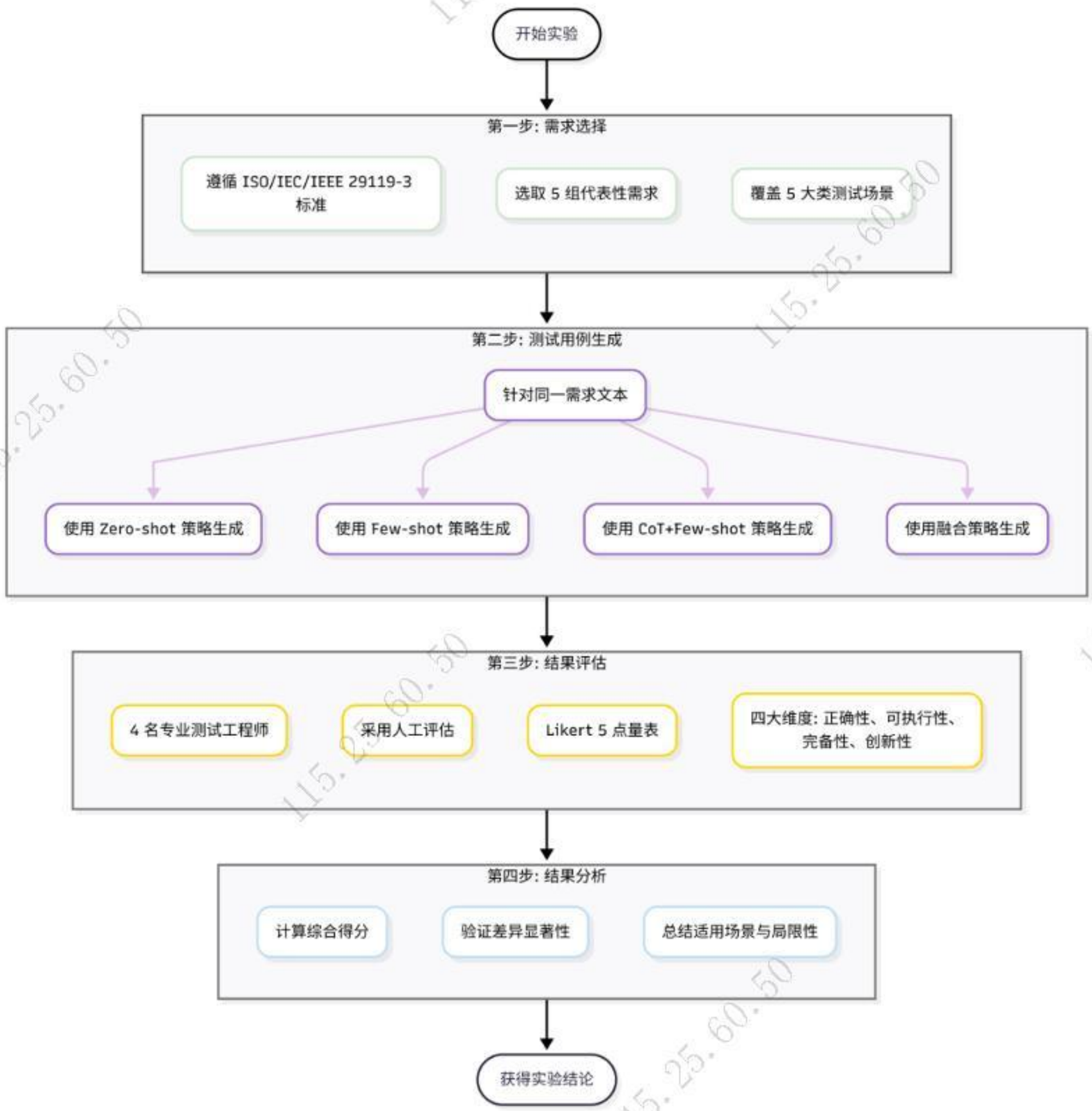


图 3-1 实验流程图

Fig.3-1 Experimental process flowchart

1）需求选择：依据 ISO/IEC/IEEE 29119-3:2013 软件测试文档标准，从功能性需求、非功能性需求及复杂业务流程需求中选取五组代表性案例，确保覆盖不同测试类型与复杂度，以增强结果的全面性与普适性。本次实验的需求涵盖：简单功能场景（用户登录）、非功能性能需求（性能测试）、复杂业务流程（电商下单）、并发竞争场景（抢购库存扣减）、跨模块与兼容性场景（在线考试答题与提交）。需求概述及验证点见表 3-3，具体需求内容见附录 B。

表 3-3 需求类型及验证目标

Tab.3-3 Requirement types and verification objectives

需求号	需求类型	需求功能	验证点
附录 B-1.1	简单功能测试	用户登录功能	验证基础边界、异常、状态
附录 B-2.1	非功能性测试	性能测试	验证对非功能性需求的测试点覆盖
附录 B-3.1	复杂业务测试	电商下单流程	验证模块拆分与集成测试能力
附录 B-3.2	复杂业务测试	抢购活动库存扣减	验证并发与竞争场景生成能力
附录 B-3.3	复杂业务测试	在线考试系统答题与提交流程	验证多模块深度交互+兼容性需求

2）为了全面比较不同提示设计策略生成测试用例的效果，实验设置了四类对照组，详见表 3-4，涵盖从最基础到融合优化的不同阶段。

表 3-4 实验提示方法及选取原因

Tab.3-4 Prompt methods used in the experiment

对比类型	具体方法	选取原因
基础基线	Zero-shot	仅角色设定 + 任务指令，无示例、无结构、无推理引导，作为最基础的生成范式，体现大语言模型在无外部引导下的“原始能力”。
单一增强	Few-shot	示例能稳格式、对齐粒度，但逻辑链不充分、覆盖容易窄化。
部分融合	CoT + Few-shot	代表当前学术界较先进的提示设计，已具备一定智能性。
融合方法	CoT + Few-shot + Template-based+ Adaptive	集合四者优势：推理深度 + 示例稳定 + 结构规范 + 需求自适应，预期在覆盖全面性、复杂场景稳健性、可自动化利用方面最优。

3）结果评估：实验采用人工评价方式，从正确性、可执行性、完备性与创新性四个维度对生成的测试用例进行打分与记录。

4）结果分析：对实验结果进行统计检验，验证不同方法间的差异显著性；并结合不同需求类型，对各提示方法的优劣进行对比，总结其适用场景与局限性，为后续研究提供参考。

3.3 实验评估方法

3.3.1 评审方法

本文采用 Likert 评分量表[66]对生成的测试用例进行人工评价。作为一种由 Likert 提出的经典态度与质量测量方法，该量表能够将主观评价转化为量化数据，已被广泛应用于各类评估场景。在测试用例生成的质量评估中，评价涵盖以下四个维度：

(1) 正确性（Correctness）：测试用例必须准确反映需求目标，避免逻辑偏差或虚构内容。若用例与需求存在偏差，即使形式完整也失去验证价值，因此正确性是最基本且最核心的评价维度。

(2) 可执行性（Executability）：测试用例必须能够转化为可操作的步骤，否则无法在测试过程中落地实施。该维度强调结构清晰、步骤可执行、步骤与预期结果一一对应，是保证用例实用性的关键。

(3) 完备性（Completeness）：高质量测试不仅要覆盖基本功能，还需涵盖边界条件、异常流程和多路径交互。完备性体现了用例对需求空间的覆盖程度，直接影响缺陷发现率。

(4) 创新性（Novelty）：大语言模型具备生成新颖测试场景的潜力，有助于发现传统方法难以覆盖的潜在风险。该维度评估生成结果是否能提出有价值的新场景，体现模型在探索性测试中的优势。

在评分体系中，评分越高表示测试用例在该维度上的质量越高。为保障评价结果的客观性与一致性，每条测试用例均由四名评审人员独立打分，最终取平均值作为该条用例的得分。该方法可有效降低个体主观偏差，提高评估结果的可靠性与可重复性。指标的量化方式及评分标准见表 3-5。

表 3-5 人工评估指标

Tab.3-5 Manual evaluation indicators

指标名称	量化方式	评分范围/标准
正确性	用例是否准确理解需求，无捏造字段，逻辑自洽	1=严重错误/捏造需求 2=部分误解 3=基本正确但有小错 4=准确但略模糊 5=完全正确、精准匹配需求
可执行性	用例是否能直接落地为“标题/前置/步骤/预期”的四段式，步骤动词起句、可操作、可复现，且与预期结果一一对应	1=无法执行或结构混乱 2=结构不全，步骤不可操作 3=大致可执行但需较大修改 4=稍作调整即可执行 5=结构完整、步骤清晰、开箱即用
完备性	用例是否覆盖正常/边界/正向/反向/有效/无效等类型，是否覆盖需求关键分支与异常。	1=仅覆盖单一场景 2=覆盖少量场景 3=覆盖主要正向和部分异常 4=覆盖多数场景，缺少数边界 5=全面覆盖所有关键路径与异常
创新性	用例是否包含新颖且合理的测试场景	1=全部为常规场景 2=略有变化但无新意 3=有个别新场景 4=多个有启发性的新场景 5=提出独特且高价值的新测试点

3.3.1.1 评审流程与一致性

为确保评估过程的客观性与结果的可重复性，本文设计了严格的评审流程。所有评审工作由四位资深测试工程师独立完成，评审人员均来自企业测试部门，平均具备 3.5 年以上软件测试从业经验。评审过程采用“盲评”机制：评审者仅能看到原始需求描述和模型生成的测试用例内容，无法获知该用例是由哪种提示策略生成的，从而避免因对方法的偏好或偏见影响评分结果，确保评估聚焦于用例本身的质量。每名评审者对同一批用例的四个维度（正确性、可执行性、完备性、创新性）分别打分。系统对每个维度计算算术平均作为该维度的最终分；四维度等权平均为综合分。为衡量评分离散度，同时计算标准差（Standard Deviation, SD），计算公式如下：

$$S=\sqrt{\frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n-1}} \tag{3-1}$$

其中，x_i为第 i 个评审者的评分，$\bar{x}$ 为该维度的平均评分，n 为评审者人数。

若任一维度的标准差 ≥1.0，则判定该用例在该维度上存在显著分歧，需标记为“需复核用例”。对于被标记的用例，将组织 4 名评审者进行小组讨论，重新审视原始需求与生成结果，明确评分标准（如：“准确性”是否包含边界条件、“可执行性”是否要求指令型动词、参数具体化等），在达成共识后，评审者进行第二次独立评分。若二次评审的标准差仍 ≥1.0，则取该维度四人评分的中位数作为最终得分，以减弱极端值的影响。最终报告中，所有用例均给出四维度分与综合分；若用例存在二次复核或取中位数的情形，将标注“低一致性样本”以便再现与审计。除特别说明外，后续所有维度的评审结果均取平均值作为总体得分。

以下图 3-2 展示的“简单需求”组为例，4 位评审对 4 种提示策略的用例分别在四个维度上评分，共形成 4（策略）×4（维度）=16 组评分。各组内的 4 人评分计算 SD 后均小于 1.0，因此无需复核。该组每种策略在各维度的最终得分即为对应 4 人评分的算术平均。

evaluate.py

```
for item in raw_data:
    prompt = item["prompt"]
    scores = np.array(item["scores"])
    sds = np.std(scores, axis=0, ddof=1)
    need_review = any(sds >= 1.0)
    avg_scores = np.mean(scores, axis=0)
    overall_score = np.mean(avg_scores)
    results.append({
        "提示工程技术": prompt,
        "正确性(均值)": round(avg_scores[0], 2),
        "可执行性(均值)": round(avg_scores[1], 2),
        "完备性(均值)": round(avg_scores[2], 2),
        "创新性(均值)": round(avg_scores[3], 2),
        "综合得分(均值)": round(overall_score, 2),
        "需复核": "是" if need_review else "否",
        "标准差": ", ".join([f"{sd:.2f}" for sd in sds])
```

运行: evaluate

提示工程技术	正确性(均值)	可执行性(均值)	完备性(均值)	创新性(均值)	综合得分(均值)	需复核	标准差
zero-shot	4	4.75	4.25	3	4	否	0.82, 0.50, 0.96, 0.82
few-shot	3.25	3.75	3.75	2.5	3.31	否	0.50, 0.50, 0.50, 0.58
CoT+few-shot	4	4	4.5	3.5	4	否	0.00, 0.00, 0.58, 0.58
综合策略	5	5	5	4.5	4.88	否	0.00, 0.00, 0.00, 0.58

图 3-2 简单需求评分示例

Fig.3-2 Example of Simple Requirement Scoring

3.4 实验结果与分析

本节基于附录 C 中表 C-1 ~ C-36 的实验数据，系统对比了四类提示方法在不同类型需求下的表现。整体来看，CoT+Few-shot+Template-based+Adaptive 的融合策略在五组实验中表现最优，尤其在正确性和可执行性维

度上接近满分，明显优于其他方法。下面按需求类型分别分析。

(1) 简单功能需求

在简单功能测试（用户登录）场景中，实验结果由三部分构成：第一部分，测试用例结果表（附录 C 表 C-1~ C-4），展示了不同提示策略生成的具体测试用例；第二部分，专家评分表（附录 C 表 C-5 ~ C-8）记录了四位评审专家在正确性、可执行性、完备性和创新性四个维度上的独立评分；第三部分，综合评分表（见表 3-6）对四位评审的评分结果取算术平均，形成最终汇总结果。

实验结果表明，融合策略（CoT+Few-shot+Template-based+Adaptive）表现最佳，其生成的测试用例在专业性、规范性和场景覆盖方面均显著优于其他方法。该策略不仅输出格式规范统一，符合“相似测试条件应合并为单一用例”的测试设计原则，还通过自适应推理机制引入了如“大小写敏感检测“多异常集成测试”等具有较高工程价值的创新场景。相比之下，Zero-shot 方法展现了较强的基础生成能力，但输出格式不够规范且缺乏创新；Few-shot 学会了模仿格式，但过度依赖示例，灵活性不足，甚至引入示例错误。

表 3-6 需求 1.1 简单需求测试用例评分表（总）

Tab.3-6 Test case scoring sheet for Requirement 1.1 Simple Requirement (Total)

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4.75	4.25	3	4
Few-shot	3.25	3.75	3.75	2.5	3.3125
CoT+Few-shot	4	4	4.5	3.5	4
融合策略	5	5	5	4.5	4.875

(2) 非功能性需求

在性能测试场景（测试用例见附录 C 表 C-9~C-12，四位评审打分见附录 C 表 C-13~C-16，综合结果见表 3-7），研究发现了大语言模型生成非功能测试用例时的一个重要局限：部分方法（如 Zero-shot 和 Few-shot）机械迁移了功能测试中的“边界值”概念，生成了诸如“999 并发用户”和“1001 并发用户”等缺乏工程意义的测试点，未能充分理解性能指标（如响应时间、吞吐量）随负载连续变化的本质。相比之下，CoT+Few-shot 及融合策略更注重系统在超负载下的行为表现（如优雅降级、服务稳定性与容错能力），其输出更贴近真实的性能验证需求，体现出更高的测试策略水平。

融合策略在保持高正确性与可执行性的同时，有效避免了无效的离散边界测试。

表 3-7 需求 2.1 非功能需求测试用例评分表(总）

Tab.3-7 Scoring Table: Requirement 2.1 Non-Functional Test Cases (Total)

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3.5	4.75	4.5	3.75	4.125
Few-shot	3.75	4	3.5	2.75	3.5
CoT+Few-shot	4.25	3.75	4.5	4	4.125
融合策略	5	4.5	4	3.75	4.3125

(3) 复杂业务流程需求

在电商下单流程等复杂功能测试中(测试用例和四位评审打分见附录 C 表 C-17~表 C-40，综合结果见表 3-8~表 3-10），融合策略展现出全面优势。能准确覆盖基础流程（正常下单、库存不足、支付失败）；能进一步生成“并发下单库存为 1”、“购物车数据传递一致性验证”等集成用例；充分体现了对模块间协作与并发控制的理解。相比之下，Zero-shot 方法虽在正确性与可执行性上表现优异，但未能覆盖并发与集成维度；Few-shot 覆盖面很强，但容易把示例里的产品假设代入成通用规则；而在高并发场景（抢购活动库存扣减）的测试用例生成任务中，CoT + Few-shot 方法展现出很强的系统级测试思维。该方法不仅覆盖了正常流程、异常路径与边界值，更进一步设计出“100 个用户同时抢购 100 件商品，验证无超卖现象”这一关键测试用例，准确覆盖了抢购活动的核心风险点（超卖、库存错乱），此外，其提出的“网络异常时抢购处理”用例，关注请求中断下的事务幂等性，能够较好满足复杂并发测试的需求。在验证多模块深度交互以及非功能性需求的核心目标下，CoT+Few-shot+Template-based+Adaptive 表现最优，其核心优势在于：能生成“并发交卷”、“跨模块数据一致性校验”、“计时同步校验”等复杂场景，精准覆盖在线考试系统“多模块联动”、“非功能性兼容”的核心测试点，完全满足复杂功能需求的测试能力验证。

四位评审专家从正确性、可执行性、完备性和创新性四个维度对生成结果进行评分，再按不同维度计算算术平均分。综合得分 S 的计算方法如下：

$$S = \frac{Correctness + Executability + Completeness + Novelty}{4} \tag{3-2}$$

各策略在 3 种不同的复杂需求上的综合得分如表 3-8~表 3-10 所示，整体平均结果见表 3-11。

表 3-8 需求 3.1 复杂业务流程测试用例评分表（总）

Tab.3-8 Test case scoring sheet for Requirement 3.1 Complex Business Process(Total)

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4.25	4.75	3.75	3.125	3.96875
Few-shot	3.75	4.25	4	3.25	3.8125
CoT+Few-shot	4.25	4.5	4.25	3.75	4.1875
融合策略	5	5	4.75	5	4.9375

表 3-9 需求 3.2 复杂业务流程测试用例评分表（总）

Tab.3-9 Test case scoring sheet for Requirement 3.2 Complex Business Process(Total)

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3.75	4.25	2.75	2	3.1875
Few-shot	4	4.25	3.75	3.5	3.875
CoT+Few-shot	4.5	4.5	4.75	4.5	4.5625
融合策略	5	5	4.5	4.5	4.75

表 3-10 需求 3.3 复杂业务流程测试用例评分表（总）

Tab.3-10 Test case scoring sheet for Requirement 3.3 Complex Business Process(Total)

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3.75	4	3	2	3.1875
Few-shot	3.5	4	4.25	3	3.6875
CoT+Few-shot	3.75	4	4.25	3.5	3.875
融合策略	5	5	4.75	4.75	4.875

表 3-11 测试用例评分汇总表 - 整体平均

Tab.3-11 Summary table of test case scores - Overall average

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3.85	4.5	3.65	2.775	3.69375
Few-shot	3.65	4.05	3.85	3	3.6375
CoT+Few-shot	4.15	4.15	4.45	3.85	4.15
融合策略	5	4.9	4.6	4.5	4.75

基于五类典型需求场景（简单功能、非功能以及三类复杂业务流程）

的综合评分结果和各策略得分和能力对比图（见图 3-3），可以看出融合策略在所有实验中表现最优，尤其在复杂业务与并发场景中显著领先，正确性与可执行性接近满分。Zero-shot 在简单功能与非功能需求中表现尚可，但随着需求复杂度提升，得分逐渐下降；Few-shot 方法受示例质量限制，覆盖与创新不足；而 CoT+Few-shot 方法在复杂场景下保持了较好的稳定性和泛化能力。

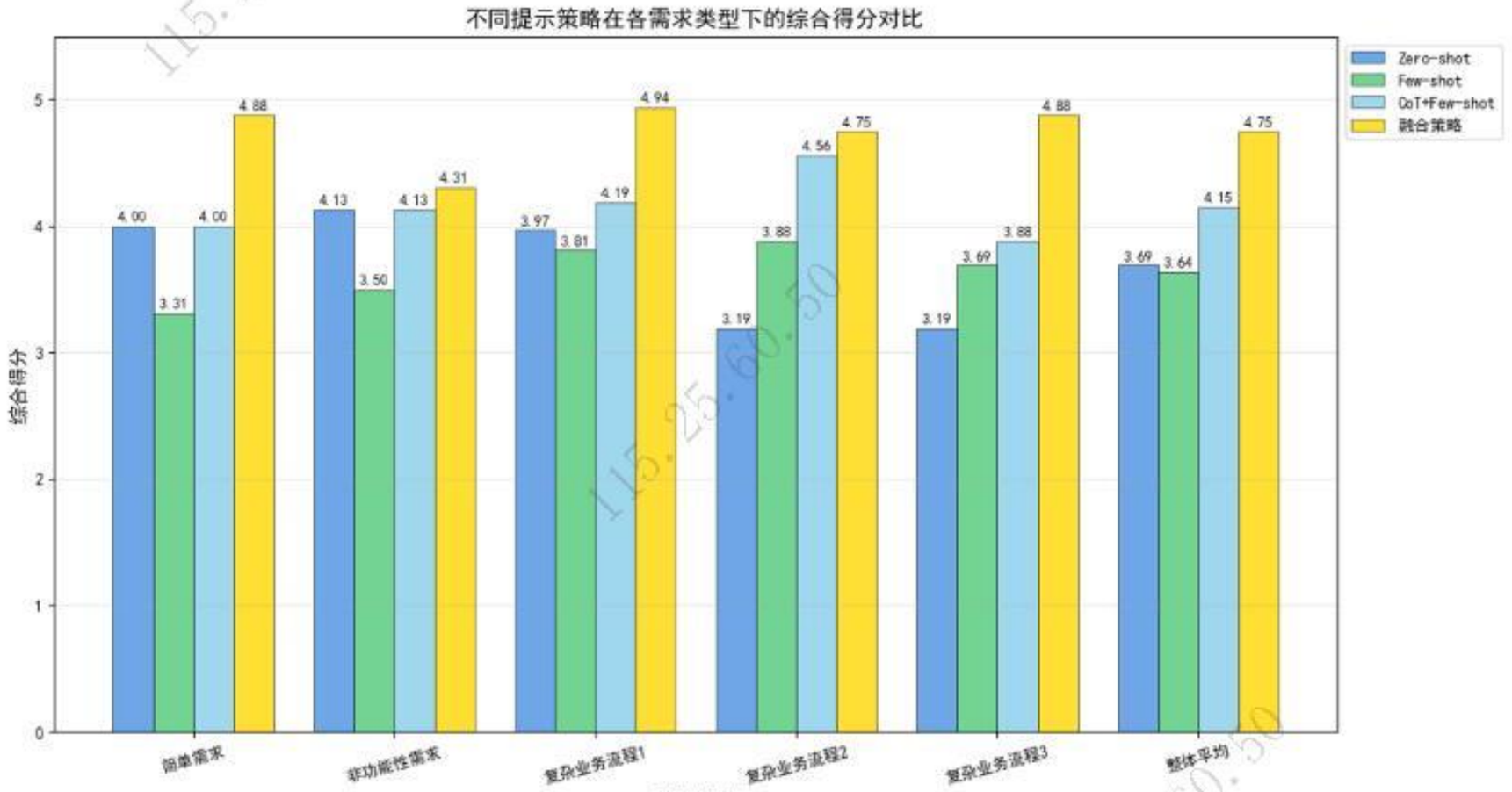


图 3-3 各策略综合得分以及整体平均得分对比图

Fig.3-3 Comprehensive & Average Scores Comparison Among Strategies

综上所述，在复杂、跨模块、含非功能与并发的真实场景下，单一提示方法难以兼顾正确性、可执行性、完备性与创新性。融合提示策略通过优势互补实现了最优表现：CoT 提供推理深度与覆盖，Few-shot 提供格式锚定，模板化确保输出可解析性，自适应机制实现策略动态调整，有效激发了大语言模型的系统级测试思维能力。因此，综合来看，CoT+Few-shot +Template-based+Adaptive 是当前最优的提示工程技术选择，尤其适用于需兼顾规范性与探索性的测试用例生成任务。

3.5 本章小结

本章系统对比了五类提示工程方法在测试用例生成任务中的表现。研究通过分层对比实验发现：零样本提示虽能生成基本用例，但缺乏结构与深度，在工程需求上存在明显局限；单独 Few-shot 提升了格式与风格，但受限于示例范围，存在模仿盲区；CoT+Few-shot 增强了逻辑完整性，但输出不稳定，难以集成到自动化流程；融合方法通过模板化保障格式、Few-shot

锚定风格、CoT 驱动推理、Adaptive 实现适配，实现了从文本生成到结构化、可执行测试资产的跨越。

这表明，单一提示策略难以兼顾生成质量的多个维度。真正的智能测试生成不是单一技巧的胜利，而是多策略协同的系统工程。CoT+Few-shot+Template-based+Adaptive 融合方法在场景覆盖率、可执行性、可验证性与格式一致性方面均表现最优，尤其在高风险场景（如并发、状态瞬时转换）的覆盖上具有显著优势。该融合策略不仅提升了生成质量，也为大语言模型在软件测试领域的工程化落地提供了可复用的提示设计范式。

需要说明的是，受限于相关研究中提示词公开的完整性，本章实验以内部消融验证为主。未来可进一步在公开基准或可复现环境下，与更多前沿提示工程方法进行横向对比，以全面验证本方法的竞争力。

4 面向测试领域的微调方法

在前一章中，我们探讨了提示工程在测试用例生成中的应用。通过精心设计提示语，可以在一定程度上引导大语言模型生成更符合需求的测试用例，提高输出的结构化程度与可读性。然而，提示工程本质上依赖于人工经验调优，对模型底层参数与知识分布没有实质性改变，因此仍存在如下局限：

(1) 适应性不足：对于企业级复杂业务需求，模型容易遗漏业务规则或生成不规范的用例；

(2) 稳定性欠佳：在跨领域应用时，提示语的微小变动常导致生成效果显著波动；

(3) 知识覆盖有限：模型缺乏行业特定知识，难以保证测试用例的完备性与可执行性；

(4) 数据安全隐患：若企业依赖外部大语言模型 API，需求文档和测试数据可能泄露，存在合规与隐私风险。

基于以上不足，单纯依赖提示工程难以满足高质量测试用例生成的需求。微调作为大语言模型领域的重要技术路径，可以在保持模型通用能力的同时，注入特定领域的知识与风格约束，并支持在企业内部完成本地化部署。通过在软件测试相关数据集和企业内部文档上进行有针对性的训练，微调不仅能够显著提升模型对需求文档的理解能力，还能改善生成用例的准确性、规范性与稳定性，同时有效规避数据外泄的风险。因此，本章将聚焦模型微调技术在测试用例生成中的应用，系统开展从数据集构建到实验验证的全流程研究。

4.1 数据集的构建与处理

4.1.1 数据来源

(1) 公司真实项目需求与测试用例：该类数据源自企业的实际研发与测试活动，具有高度的真实性和业务相关性。通过使用内部需求文档与历史测试用例，可以有效提升模型对企业特定领域的适配性，使其生成结果更符合实际项目的规范与风格。

(2) 《Software Engineering》教科书中的案例：软件工程领域的经典教材中包含了大量系统化的案例和规范化的需求描述。这部分数据具有良好的结构化特点和教学示范意义，能够帮助模型学习标准化的需求表达方式和测试用例设计逻辑，从而提升生成结果的完整性与规范性。

(3) 开源的数据集：为了增强模型的通用性与鲁棒性，本文还引入了部分公开可获得的测试用例相关数据集。这些开源数据不仅覆盖不同类型的软件系统与需求场景，还能在一定程度上弥补企业内部数据量有限的问题，有助于提升模型在跨领域和多样化场景下的泛化能力。

表 4-1 对以上三类数据集的数量、用途和数据类型做了详细描述。

表 4-1 数据集相关信息

Tab.4-1 Relevant Information of the Dataset

数据来源	需求数量	测试用例数量	数据类型	用途
教材	52	424	需求-测试用例对	训练基础能力
企业实际项目文档	152	895	需求-测试用例对	领域适应及验证微调效果
开源数据集	167	601	JSON	训练基础能力

4.1.2 数据格式定义

在大语言模型的微调过程中，最常用的数据组织方式是 Prompt-Completion 格式。该格式是监督微调中常用的数据结构，由 Prompt（任务指令或用户输入）和 Completion（目标输出）两部分组成。OpenAI 官方在其微调数据准备指南中明确指出:“每个训练样本应当是一个 JSONL 对象，由 Prompt 和 Completion 字段构成”[67]，这一规范已经成为工业界事实标准。同时，学术界也普遍采用这一形式。Liu 等人[24]在系统性综述中指出，基于提示的学习范式通常将原始输入转化为 Prompt，再由模型生成 Completion，以完成最终预测任务。Taori 等人[68]在 Alpaca 项目中同样采用了该格式，构建了 52K 条指令-响应对，用于指令微调实验。

因此，本文的数据均采用 Prompt-Completion 格式。其设计核心价值在于清晰地建立任务指令与目标输出的配对关系，并配合高效的损失计算机制。在训练过程中，Prompt 作为上下文输入，模型需生成与 Completion 一致的输出。损失函数仅作用于 Completion 部分，从而避免模型在“重复已

知指令”上过拟合或浪费算力，使训练过程更加高效。该数据结构的逻辑关系示意图如图 4-1 所示。

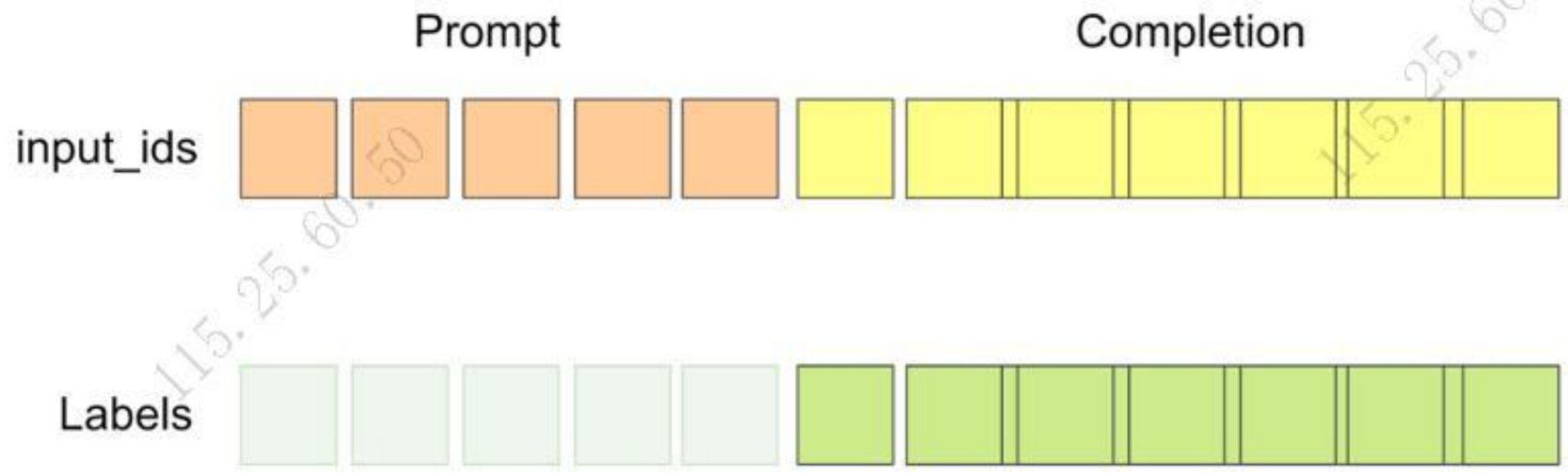


图 4-1 Prompt-Completion 格式示意图

Fig.4-1 Illustration of the Prompt-Completion Format

4.1.3 数据清洗与预处理流程

在完成原始数据的收集之后，需要对数据集进行系统性的清洗与预处理，以确保后续建模与实验的有效性和可靠性。由于需求文档与测试用例数据往往来源多样、格式不一，甚至包含冗余或噪声信息，因此本文设计了一套清洗与预处理流程，见图 4-2。

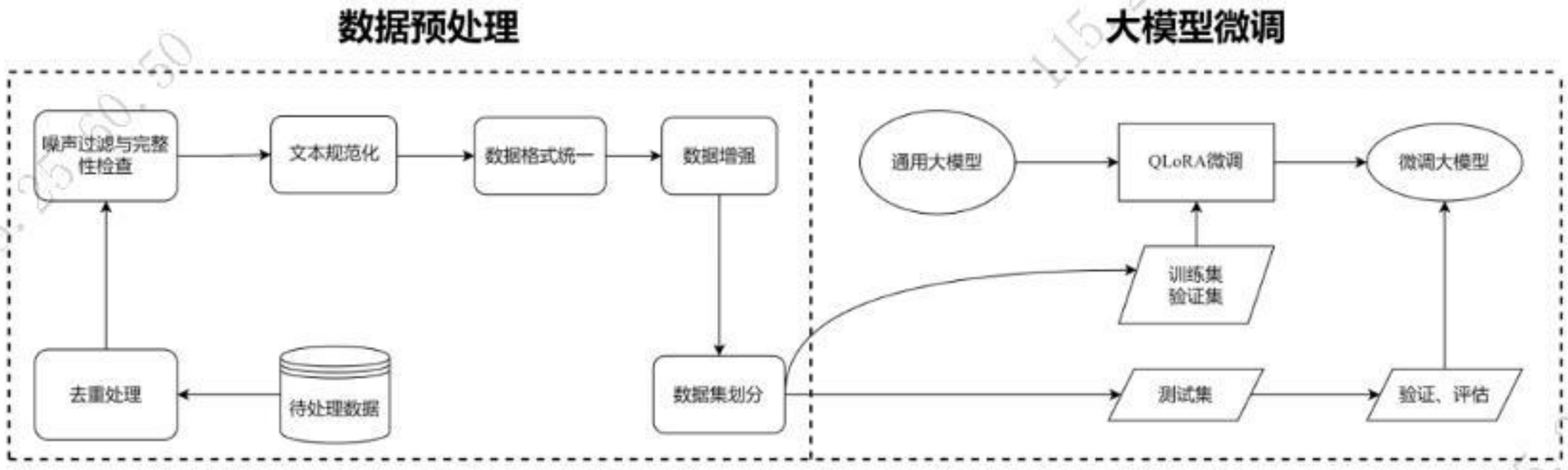


图 4-2 数据预处理流程图

Fig.4-2 Flowchart of Data Preprocessing

(1) 去重处理

在多源数据整合过程中，需求与测试用例往往存在不同程度的重复。若直接将这些重复样本纳入训练，不仅会导致样本分布偏倚，还可能使模型过度依赖高频模式，削弱其在低频场景下的泛化能力。因此，本文在数据预处理阶段首先进行去重操作。

去重策略分为两类：其一是精确去重，如果两条数据在各个字段上完全相同，就直接删除重复的记录。二是近似去重，有些数据虽然文字写法

不同，但意思几乎一样。对于这种情况，本文先把文本转成向量，再计算它们之间的相似度。如果相似度高于设定的阈值（比如 0.9），就认为它们是重复的，只保留其中信息最完整、表达最清楚的一条。通过上述处理，能够显著减少冗余数据，确保样本在语义与结构上的唯一性，从而为后续的噪声过滤与完整性检查奠定基础。

(2) 噪声过滤与完整性检查

在完成去重处理后，本文进一步对数据进行噪声过滤与完整性检查，以确保需求与测试用例的质量与可用性。为建立统一的数据质量评估标准，本文采用行为驱动开发（Behavior Driven Development, BDD）所倡导的前提条件（Given）、触发动作（When）与期望结果（Then）结构作为需求与测试用例的规范化模板。BDD 是一种以自然语言描述软件行为为核心、通过 Given-When-Then 结构统一需求认知的开发方法论[69]，为解决传统需求管理中存在的沟通壁垒、理解偏差等问题提供了有效路径。对于未采用 BDD 结构化表述的需求，本文采用大语言模型辅助抽取 Given-When-Then 片段，并由人工进行校对与修订，从而保证需求数据的结构化一致性。噪声过滤主要针对不符合格式约束或内容缺失的情况，例如步骤与预期结果数量不一致的测试用例。完整性检查则从四个方面展开，具体方法见表 4-2。

1) 句法结构完整性：利用分词与词性标注（Part-of-Speech Tagging，POS Tagging），要求 Given 部分至少包含一个主体名词，When 部分至少包含一个动词，Then 部分至少包含一个动词或结果性形容词。若缺失关键成分，则判定为不完整。

2) 命名实体完整性：结合 jieba 分词与自定义领域词典，对需求和用例中是否出现关键对象进行识别与校验。领域词典覆盖了测试场景中的常见对象，如“页面”“功能”“模块”“数据”“文件”“步骤”“流程”“权限”“角色”“提示”“结果”“链接”“菜单”“选项”“参数”“状态”“操作”“请求”“响应”“记录”“系统”等。判定标准为：每条需求或用例必须至少包含一个关键对象，否则视为不完整样本。

3) 字段一致性：通过正则匹配和结构化解析，检查需求是否包含 Given、When、Then 三个部分且字段非空；同时验证测试用例是否具备标题、前置条件、步骤与预期结果四个必备字段，并要求步骤与预期结果数量一致且编号一一对应。

4) 语言质量：结合词性标注与正则规则，对文本进行质量控制。判定标准包括：When 部分必须包含动词，Then 部分必须包含动词或结果性形

容词；文本不得出现乱码或异常符号。

通过上述多维度的检查与过滤，能够有效剔除结构缺失、语义模糊或内容异常的样本，从而保证需求与测试用例数据的完整性和规范性。

表 4-2 完整性检查维度与方法

Tab.4-2 Dimensions and Methods of Integrity Check

检查维度	技术方法	判定标准
句法结构完整性	POS Tagging	Given 至少包含一个主体名词； When 至少包含一个动词； Then 至少包含一个动词或结果性形容词
命名实体完整性	jieba 分词 + 自定义领域词典	文本至少包含一个关键对象 （如“页面”“功能”“模块”“数据”等）
字段一致性	正则匹配 + 结构化解析	需求必须包含 Given/When/Then 且字段非空； 测试用例必须包含标题、前置条件、步骤与预期结果； 步骤与预期结果数量一致、编号一一对应
语言质量	词性标注 + 正则规则	When 部分必须包含动词； Then 部分必须包含动词或结果性形容词； 文本不得出现非法字符或异常符号

(3) 文本规范化

在保留有效样本后，需要对文本进行统一化与规范化处理，以保证不同来源的需求与测试用例在形式上保持一致。本文采用以下策略：

1) 去除冗余符号与空白：利用正则表达式清理多余的标点符号与空白字符，将输入文本转化为简洁规范的形式。

2) 时间与数值统一：结合日期解析工具与正则匹配，对时间表达（如“2025/01/01”“25 年 1 月 1 日”）和数值（如“1,000”“1000”）进行标准化，统一为 YYYY-MM-DD 与阿拉伯数字格式。

3) 字段一致化：对测试用例表的字段命名与顺序进行标准化，确保样本均符合统一的 schema 结构（如 id、title、precondition、steps、expected），便于后续处理与追溯。

4) 拼写与语法纠错：针对英文描述，采用 language_tool_python 库的 en-US 模式进行拼写与语法纠错，提升文本的准确性与一致性；针对中文描

述，则结合 zh-CN 模式与正则规则，进行常见错别字、符号及格式错误的检测与修正。

(4) 数据格式统一

如图 4-3 所示，通过“信息抽取→格式统一”的两段式流水线，原本松散的自然语言需求被稳健地转化为结构化、可计算、可追溯的领域专业数据。一方面，抽取阶段保证了语义不失真、边界不缺失；另一方面，统一阶段确保了数据模型一致、步骤与期望对齐、格式满足下游系统的接口契约。两者配合，为测试用例的自动生成、质量评审与持续集成提供了坚实的数据基础。

图 4-3 数据格式统一流程图

Fig.4-3 Unified data format flowchart

1) 信息抽取（自然语言 → 结构化要素表）

首先，从自然语言需求文本出发，对零散、口语化的描述进行语义解析与要素识别。本阶段的目标是把“用户想要什么”“在什么条件下触发”“系统该如何响应”等关键信息从原文中剥离出来，形成可读、可操作的字段集合。为降低歧义，抽取时优先保留原句中的量化限定词与边界条件（如长度、范围、是否必填、异常分支等）。

在要素层次上，信息抽取分为“用例意图”与“可执行测试”两条线索并行推进。用例意图包含使用场景、前提条件、触发动作与期望结果，用于概括业务目标及其约束。可执行测试部分聚焦工程落地，提炼测试点/用例名称、前置条件、测试步骤与预期结果等字段，并尽量将复杂表述拆分为多条可验证的原子步骤，便于后续编号、对齐与自动比对。

抽取完成后，产出为一张结构化要素表。这张表既保留了原始需求的语义信息，又对关键字段进行了标准化定义与分栏展示，相当于在“自然语言”与“数据结构”之间建立了一层语义网关。通过这一步，后续的模

板映射与自动生成才能做到有据可依、口径一致。

2) 格式统一（结构化要素表 → JSONL）

在完成字段级抽取后，进入格式统一阶段。该阶段以系统约定的数据模型为准绳，将要素表中的各字段映射到标准化的 JSONL 结构中，确保不同来源、不同撰写风格的需求都能被下游模块一致地消费与处理。在顶层，JSONL 以 usecase 与 testcases 作为核心入口。usecase 节点下承载 Description/Given/When/Then 等要素，完整表达用例意图与业务逻辑主干；testcases 以数组形式存放一组面向执行的测试用例，每条用例包含 title、type、precondition、step[]、expected[]等字段。为保证可验证性，step 与 expected 必须按编号一一对应（如“1.”对应“1.”），复杂期望允许在同一编号下以“（1）（2）...”细化子结果，从而既保持结构严谨，又兼顾表达丰富度。

(5) 数据增强

为了缓解原始数据规模有限所带来的训练不足问题，本文引入了基于大语言模型的自动化数据增强方法。具体而言，利用 GPT 模型对现有需求与测试用例进行多样化改写，生成在语义保持一致的前提下具有不同表述形式的变体样本。增强策略主要包括：对原始需求或测试用例进行重述，保持逻辑含义不变但改变语言风格；替换部分关键词，如将“登录按钮”替换为“登入按键”，或将“数据库记录”替换为“数据存储条目”，以增强模型对词汇多样性的适应能力；在原有需求或测试用例的基础上增加边界条件或异常输入场景等场景扩展以及进行结构化变换。通过上述多维度的增强策略，能够显著提升数据集的多样性与覆盖度，避免模型过度拟合少量高频表达，从而增强在低资源或复杂场景下的泛化能力。

(6) 数据集划分

在完成数据清洗、规范化与增强之后，本文对最终样本集进行了训练、验证与测试的划分，以保证模型开发和评估的科学性。采用 8:1:1 的比例，即 80%用于训练，10%用于验证，10%用于测试。训练集用于模型参数的学习，是模型拟合的主要数据来源；验证集用于超参数调优与模型选择，在训练过程中监控模型的泛化能力，避免过拟合；测试集仅在最终阶段使用，用于对模型性能进行独立评估，以验证实验结果的有效性与可复现性。在实现上，本文借助 scikit-learn 库的 train_test_split()函数对数据进行随机划分，并通过设置固定随机种子（如 random_state=42），确保数据集划分过程的可重复性。对于类别不均衡的情况，采用 stratify 参数保持不同类别样本在训练、验证和测试集中的比例一致，从而提升评估结果的代表性与公平性。

通过上述一系列的数据清洗与预处理流程，保证了数据的高质量与一致性，为后续的模型训练与实验分析奠定了坚实基础。

4.2 微调方案设计

4.2.1 基座模型选择与验证

4.2.1.1 基座模型的多维度评估与初选

本研究以开源可微调、消费级显卡部署兼容、中文场景深度适配为核心约束前提，在此基础上综合考虑了模型性能、开源可用性、硬件适配性及任务相关性四大维度，并分析了 SuperCLUE[70]与 ReLE（原 CLiB）两大中文大语言模型评测榜单，进行了基座模型选型。SuperCLUE 是一个中文通用大语言模型综合性基准测评平台，对多个能力象限（包括语言理解与生成、知识运用、推理逻辑、模型安全性等）进行系统评估，并定期发布报告，是一个具有学术公开性与可复现性的评测基准，其在中文模型测评领域具备较强的学术与产业参考价值。ReLE 覆盖教育、金融、法律等六大核心领域，在实际任务适应性方面具有重要参考价值。截至 2025 年 7 月，SuperCLUE 10B 和 ReLE 5B 以下模型排行榜中 Qwen3-4B 在语言生成能力、指令遵循能力、逻辑推理能力等与测试用例生成相关维度上均表现突出。

结合计算资源约束（NVIDIA RTX 4060Ti 16GB 显存）与工程可行性，本研究最终筛选出三个开源中文大语言模型作为候选：Qwen3-4B（通义千问系列）、ChatGLM3-6B（智谱 AI 系列）和 Baichuan2-7B（百川智能系列）。这一选择基于以下考量：(1) 参数规模覆盖：从 4B 到 7B 的参数范围，代表了消费级 GPU（NVIDIA RTX 4060Ti 16GB 显存）可支持的最大模型规模区间；(2) 工程生态成熟度：三者均具备完善的 Hugging Face Model Hub 支持；稳定的 4-bit 量化实现及社区验证的 LoRA/QLoRA 适配方案，满足本研究的要求。

4.2.1.2 基座模型的选型实验

测试用例生成任务对模型的结构化输出能力具有严格要求。为量化评估候选模型在此关键维度的表现，本研究设计并开展了基座模型选型实验。实验设置与后续主实验保持一致（环境配置详见 4.3.1 节）。评估数据源自

4.1 节构建的全量数据集，随机抽取 20 条样本构成评估子集，并在抽样后人工检查了其需求类型与复杂度的覆盖情况，使该子集在一定程度上具有代表性。

实验要求三个候选模型在统一的提示模板下生成符合预定 JSON Schema 的测试用例，生成内容必须完整包含“标题”“前置条件”“测试步骤”“预期结果”四个必备字段，并确保字段命名、层级结构及类型均满足模式规范。此外，模型输出需保持 JSON 格式，不得包含任何额外的自然语言解释、提示性文字、Markdown 标记或其他非 JSON 内容，以保证输出能够被标准解析器直接解析与验证。核心评估指标为结构化输出达标率，即模型输出能够被标准 JSON 解析器成功解析且完全符合预设字段规范的样本比例。三个候选模型的评估结果对比如下表 4-3 所示。

表 4-3 基座模型结构化输出能力对比

Tab.4-3 Comparison of Structured Output Capabilities among Base Models

候选模型	参数量	达标样本数	达标率
Qwen3-4B	4B	20	100%
Baichuan2-7B	7B	6	30%
ChatGLM3-6B	6B	3	15%

实验结果表明，Qwen3-4B 在结构化输出能力上表现卓越，其生成的 20 条测试用例全部成功解析为合规的 JSON 对象，达标率达到 100%。而 Baichuan2-7B 与 ChatGLM3-6B 的表现则存在显著不足，其输出普遍存在字段缺失、括号或引号不匹配，以及在 JSON 结构外附加冗余解释文本或 Markdown 格式等错误，导致达标率分别仅为 30%和 15%。

鉴于本研究将微调后的模型封装为测试用例生成系统的核心引擎，结构化输出的稳定性直接决定了系统可靠性与用户体验。因此，结合榜单评估与实证分析，本研究最终确定 Qwen3-4B 作为后续微调实验的基座模型。

4.2.2 微调技术选型

第 2.2 节中，已对常见的几种参数高效微调方法（Prefix Tuning、Adapter

Tuning、LoRA、QLoRA）的基本原理与优点进行了总结。为了结合实际应用场景进行选型，本节进一步补充说明这些方法的适用场景与局限性：

(1) Prefix Tuning 适合生成类任务（翻译、对话、文本生成），尤其在低资源、轻量级实验场景中效果突出，但是难以处理复杂任务，模型在需要强逻辑推理或领域适配时效果有限。而且 Prefix Tuning 在某些任务上的效果比 LoRA 等方法弱，并且需要改进前缀模块以提升表现[71]。

(2) Adapter Tuning 在跨任务、多领域情境下具有较强的可扩展性与适应性[72]。然而存储和推理开销并非最小，在任务和模型规模增加时，其资源使用会明显提升[73]。因此，在任务跨度较大或需求泛化性强的场景里，使用 Adapter Tuning 时必须更加关注性能与资源开销之间的折中。结合本文的硬件资源与任务特性，Adapter Tuning 虽可选但在这些条件下可能不如 QLoRA 那样兼顾稳定性与效率。

(3) LoRA 在极大减少可训练参数的前提下，性能与传统的全参数微调相当或接近。适合资源有限的环境，支持在中等规模 GPU 上进行较大语言模型的适配，不过对超大语言模型（>10B 参数）显存压力依然较大，在单卡环境下不够友好。

(4) QLoRA 是在 LoRA 的基础上引入量化与优化策略，显著降低显存需求，使得在单卡 GPU 环境下也能完成数十亿参数规模模型的微调。然而，QLoRA 的实现依赖于特定的量化支持库，其训练稳定性在一定程度上受底层优化实现的影响；此外，在部分对模型表达能力要求极高的任务上，其性能仍略逊于全参数微调。

除了上述微调方法外，近期研究如 DoRA（Weight-Decomposed Low-Rank Adaptation）[74]和 Pissa（Principal Singular Subspace Adaptation）[75]等在 LoRA 基础上进一步解耦权重更新的“方向”与“幅值”，在全精度训练中展现出更优的收敛效率与表示能力；然而，二者目前尚未与 4-bit 量化训练良好兼容，在低显存消费级设备上的稳定性与实用性仍有待系统验证。综上，结合本文的实验条件，在保证训练可行性与结果可靠性的前提下，QLoRA 是最优选项。它既能满足硬件资源限制，又能保证模型在测试用例生成任务中的精度与稳定性。因此，本文采用 QLoRA 作为微调技术方案。

为清晰呈现本文采用的微调流程，以下算法 Answer-Only QLoRA 微调流程给出了基于 Qwen3-4B 模型的 QLoRA 微调伪代码。该实现采用 Answer-Only 监督策略，仅对模型生成部分（assistant 回复）计算损失，避

免对输入提示（system/user）进行无效优化，从而提升训练效率与生成质量。伪代码见表 4-4 所示。

表 4-4 Answer-Only QLoRA 微调算法

Tab.4-4 Answer-Only QLoRA Fine-Tuning Algorithm

Algorithm: Answer-Only QLoRA Fine-Tuning

```
# 输入参数声明：dialogs, template, tokenizer, base, LoRA, hyperparams
require D, t, T, M0, L, h
# 输出结果声明：微调后模型，可选合并权重
ensure  M*, W_merged
# 数据集预处理
for each e in D:
   (p, a) ← SplitRoles(e)          # p: system+user; a: assistant
   s ← t(p, add_generation_prompt=true)
   x ← T(s // a);  k ← |T(s)|;  n ← |x|
   # 仅对模型生成部分（assistant 回复）计算损失
   for i = 1..n:
      if i ⩽ k: y[i] ← -100 else: y[i] ← x[i]
   store(input_ids=x, labels=y, attention_mask)
# 模型量化并增加 LoRA 层
M ← Quantize4bit(M0)
AttachLoRA(M, L) on {q,k,v,o, gate,up,down}
EnableGradCheckpointing(M)
# 优化训练和模型导出
Train(M) with AdamW(8-bit), cosine LR, warm-up
select_best by eval_loss
SaveAdapter(M, L); optionally Merge → W_merged
return M*, W_merged
```

4.2.3 参数设置详述

在本文的微调实验中，参数的设置不仅考虑了硬件资源限制，还兼顾

了模型的收敛效率与泛化性能。包括基础实验配置与微调超参数两大部分。基础实验配置主要涵盖基座模型选择、加载方式、输入序列长度以及数据格式等，它们直接决定了实验的可复现性和数据处理方式，配置见表 4-5。

表 4-5 基础实验配置

Tab.4-5 Basic Experimental Configuration

类别	参数名称	取值/配置	说明
基座模型	模型类型	Qwen3-4B	采用开源大语言模型作为微调基座
	加载方式	Unsloth + 4-bit 量化	使用低比特量化以减少显存占用，提高可训练性
输入设置	最大序列长度	1024	保证单条需求或测试用例输入完整，同时兼顾显存限制
数据设置	数据格式	JSONL	标准化后的结构化数据，包含需求–测试用例对
	数据划分比例	8:1:1	按照训练集、验证集、测试集划分

微调超参数则涉及 LoRA 参数设定、优化器与学习率策略、批次大小与梯度累积方式，以及评估与保存机制。详细配置见表 4-6。

表 4-6 超参数配置

Tab.4-6 Hyperparameter Configuration

类别	参数名称	取值/配置	说明
LoRA 配置	秩(r)	8	控制低秩矩阵维度，兼顾表示能力与显存占用
	LoRAα系数	16	缩放系数，保证训练稳定性
	目标模块	q_proj, v_proj	在注意力投影层注入适配参数
	Dropout	0.05	增强泛化能力，降低过拟合风险
	偏置项	none	不引入额外偏置参数
	任务类型	Causal LM	因果语言建模，自回归生成任务
训练配置	训练批次大小	4	单卡批次，结合梯度累积
	梯度累积步数	8	等效批次大小为 8，更稳定
	学习率	1.00E-04	较高学习率，加快 LoRA 参数收敛
	优化器	AdamW (8-bit)	高效优化器，结合 8-bit 量化降低显存
	权重衰减	0.01	防止过拟合

4.2.4 损失函数

本文采用交叉熵损失函数（Cross-Entropy Loss）作为主要训练目标函数。其核心思想是最小化模型生成的概率分布与参考答案分布之间的差异，从而提升模型在需求-测试用例生成任务中的拟合能力。在形式化定义中，给定输入序列 x 和目标序列 $y=(y_1,y_2,\ldots,y_T)$，模型在每个时间步 t 预测条件概率分布$\hat{p}(y_t \mid x, y_{<t})$。交叉熵损失函数定义如下：

$$L = -\frac{1}{T}\sum_{t=1}^{T} log\,\hat{p}\,(y_t \mid x, y_{<t}) \tag{4-1}$$

其中，T 为目标序列的长度，$\hat{p}$为模型的预测概率分布。该损失函数在语言建模与文本生成任务中已被广泛采用，能够有效衡量预测结果与参考序列之间的差异。监督微调中交叉熵损失的计算流程如图 4-4 所示。

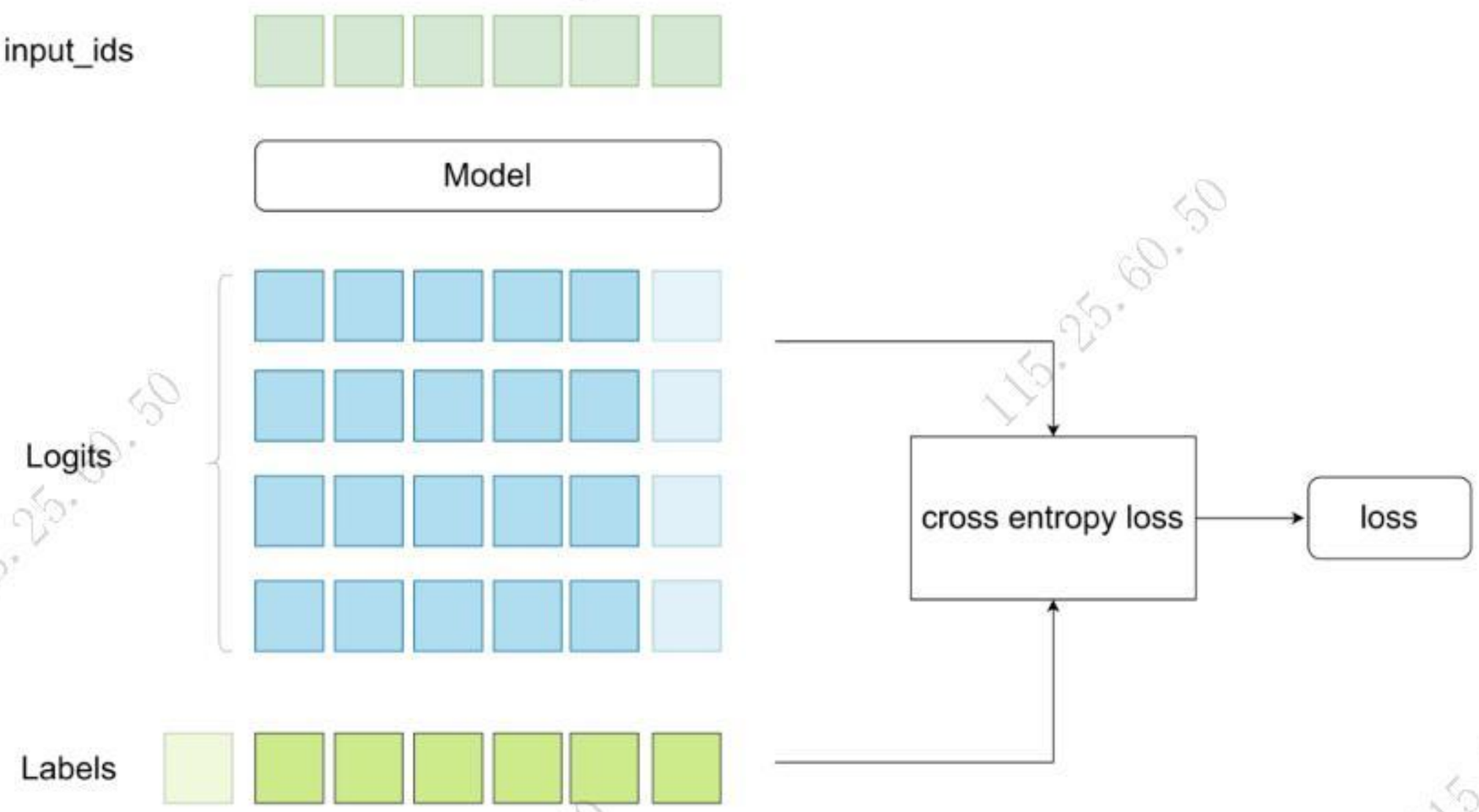


图 4-4 监督微调中交叉熵损失的计算流程

Fig.4-4 The computation process of cross-entropy loss in supervised fine-tuning

input_ids 表示输入的 token 序列（通常由 Prompt 与 Completion 拼接而成）。模型输出的 Logits 维度为[sequence_length, vocab_size]，其中第 t 列对应第 t 个位置的预测，行方向为词表中每个词的得分。Labels 由 input_ids 右移得到，用于对齐预测目标。交叉熵损失函数首先对 Logits 进行 softmax 转换得到概率分布，再与 Labels 的 one-hot 目标对比，逐位置计算损失，最后在所有有效 token 上取平均。其训练目标是最小化平均损失，从而最大化模型对参考答案序列的条件似然。

4.3 实验设计

4.3.1 实验环境配置

本文的微调实验基于表 4-7 的软硬件环境下完成。硬件配置方面，选用 NVIDIA RTX 4060Ti 16GB 显卡为模型训练提供了充足的显存空间，确保能够加载大型基础模型并进行梯度计算；32GB 系统内存保障了数据预处理和加载的流畅性；13 代 Intel Core i5 处理器提供基础计算能力。软件环境方面，采用 Ubuntu 22.04 LTS 作为操作系统，确保了深度学习框架的兼容性和稳定性。配合 CUDA 12.3 并行计算平台和 Python 3.11.8 编程环境，构建了完整的深度学习开发栈。该配置支持基于 Hugging Face Transformers 与 PEFT 库的大语言模型高效微调，满足 QLoRA 等参数高效微调方法的运行需求。

表 4-7 实验环境配置表

Tab.4-7 Experimental Environment Configuration Table

软件/硬件	配置/信息
处理器	13th Gen Intel(R) Core(TM) i5-13400F
显卡	NVIDIA RTX 4060Ti 16GB
内存	32GB
系统版本	Ubuntu 22.04.4 LTS
CUDA 版本号	12.3
Python 版本号	3.11.8

4.3.2 实验设计

本节围绕“微调是否显著提升测试用例生成质量”“提示工程与微调是否具有叠加增益”两个核心问题开展系统化实验设计。

(1) 实验目标

本实验旨在验证微调后的模型在测试用例生成任务上的整体表现相对基座模型的提升幅度。除此，还要验证“融合提示策略+微调”的协同效应，相较于仅用微调或仅用提示工程的效果增益。

(2) 实验流程

实验采用“由浅入深、逐层验证”的设计，以消除混杂因素、突出关键变量的因果作用。首先是基座模型验证，在原始 Qwen3-4B 上采用 Zero-shot Prompt，获得测试集上的模型基线；然后，在保持提示不变

（Zero-shot）的前提下，引入 QLoRA 进行参数高效微调，评估微调带来的质量提升；最后，在 QLoRA 微调模型上，采用 CoT + Few-shot +Template-based+ Adaptive 的多策略融合提示，验证提示工程与微调的协同最优效果。

(3) 实验方案与分组

根据上述目标与思路，将实验划分为三大分组。A 组建立未微调的基线；B 组验证微调的效果；C 组探究微调与提示工程的协同提升效果。详见表 4-8。

表 4-8 实验分组表

Tab.4-8 Experimental GroupTable

实验组	模型配置	提示策略	说明
A	原始 Qwen3-4B	Zero-shot	基于零样本提示获取未微调模型的效果，建立原始模型的基准数据
B	QLoRA 微调 Qwen3-4B	Zero-shot	在零样本提示的基础上，引入 QLoRA 微调，验证效果
C	QLoRA 微调 Qwen3-4B	融合提示策略	采用多提示策略融合的方法，验证提示工程与微调协同的最优效果

4.4 实验评估方法

4.4.1 评估方法选择

在自然语言生成任务中，传统指标如 ROUGE 和 BLEU 被广泛用于评估生成文本与参考文本之间的表面相似性。ROUGE 侧重于召回率，通过计算 n-gram 或最长公共子序列等单元的重叠情况，强调对参考文本内容的覆盖程度。BLEU 侧重于精确率，基于 n-gram 匹配的精度进行评估，通常使用 1-gram 至 4-gram，强调生成表达的准确性。

然而，上述指标存在显著局限：它们依赖字面匹配，难以捕捉语义层面的等价性。在测试用例自动生成任务中，功能逻辑等价的测试用例可能因表达方式、句式结构或术语选择的不同，导致 ROUGE 或 BLEU 得分偏低，从而无法真实反映其质量。例如，两个测试用例虽在语义上完全等效，但若一个使用“点击登录按钮”，另一个使用“触发登录操作”，传统指

标可能判定其相似度较低，这显然与实际评估目标相悖。

为克服这一不足，本文选择 BERTScore 作为核心评价指标。BERTScore 利用预训练语言模型的深层语义表示计算嵌入空间余弦相似度[76]，其核心优势在于能够超越词汇表面的匹配，捕捉语义层面的对应关系，从而更贴合本任务对“语义等价性”而非“字面重合度”的评估需求。因此，与依赖表层重叠的传统指标相比，BERTScore 能够更科学地衡量测试用例生成任务中的语义一致性，故被本文选为自动化评估指标。

尽管 BERTScore 在语义建模方面具有显著优势，但其评估维度仍存在局限：自动化指标无法全面衡量测试用例在工程实践中的关键属性，如可执行性、边界覆盖能力、异常场景处理、可读性及与目标系统的兼容性等。例如，一个语义得分较高的测试用例，却未能覆盖关键异常路径，导致测试有效性不足。因此，为构建全面、稳健的评估体系，本文在采用 BERTScore 作为核心自动指标的基础上，引入人工评估作为必要补充。通过自动化评估+人工评估双重评估方式，确保实验结论的正确性与科学性。

4.4.2 自动化评估

4.4.2.1 BERTScore 原理及语义评估机制

BERTScore 通过预训练语言模型的上下文嵌入来评估文本相似性，而不是依赖表层匹配。其核心思想如下：

(1) 上下文向量提取：将候选文本（生成用例）和参考文本（标准数据集）分别输入预训练语言模型（本文采用 BERTChinese），获取每个 token 的上下文感知向量表示。

(2) 相似度矩阵构建：计算候选文本与参考文本所有 token 对之间的余弦相似度，形成相似度矩阵。

(3) 精确率（Precison，P）计算：设候选文本（生成用例）的 token 集合为 $c=\{c_1,c_2,\ldots,c_n\}$，参考文本（标准数据集）的 token 集合为 $r=\{r_1,r_2,\ldots,r_m\}$，$cos(c_i,r_j)$表示候选文本第 i 个 token 与参考文本第 j 个 token 的余弦相似度，则 P 的公式为：

$$P = \frac{1}{n}\sum_{i=1}^{n} \max_{j=1,\ldots,m} cos(c_i, r_j) \tag{4-2}$$

(4) 召回率（Recall，R）计算：对于参考文本中的每个 token，找到与

候选文本最相似的 token，计算平均值作为 Recall；衡量参考内容中有多少语义被生成文本覆盖，计算公式为：

$$R = \frac{1}{m}\sum_{j=1}^{m} \underset{i=1,\ldots,n}{max} cos(r_j, c_i) \tag{4-3}$$

(5) F1 分数合成：采用 Precision 与 Recall 的调和平均作为最终评估指标，计算公式为：

$$F1 = 2 \times \frac{P \times R}{P + R} \tag{4-4}$$

在测试用例生成的场景中，Precision 得分高代表生成用例中冗余较少，关键语义点与参考一致；Recall 得分高代表参考用例中的信息大部分被覆盖；F1 分数高表示生成用例整体上与人工用例在语义层面高度对齐，是本文采纳的主要性能指标。

4.4.2.2 评估流程与实施细节

本文设计如下四步自动化评估流程：

(1) 文本规范化：统一大小写、空白符、标点格式，去除占位符以及不影响语义的装饰性字符（如多余空格、换行符）。

(2) 输入结构划分：对每个测试用例分别评估标题、前置条件、步骤和预期结果。标题为测试用例名称；前置条件是测试用例的先决条件，满足条件才可做操作；测试用例中，每个操作步骤应有与其对应的预期结果。各部分独立计算 BERTScore，最终加权聚合为用例级别得分。

(3) 计算指标：使用 BERTChinese 模型生成词嵌入，计算每对文本 Precision、Recall、F1。

(4) 结果汇总与对比：对所有测试用例样本进行调和平均 F1，获得实验组的整体表现；在不同实验组间进行横向对比，分析微调策略对语义生成质量的影响。

4.4.3 人工评估

人工评估方法同 3.3 节，采用 Likert 1-5 评分量表，从正确性、可执行性、完备性和创新性四个维度对生成的测试用例进行人工评价，评判的数据范围是随机选取微调测试集中的 10 条需求和与之对应的测试用例。

4.5 实验结果与分析

4.5.1 微调效果评估

如图 4-5 所示，训练损失与验证损失在整个过程中均持续下降，并在约 500 step 后由快速下降转入长尾收敛。末段两者的差值维持在较小水平，且未出现验证损失回升，表明本次微调未出现显著过拟合。考虑到收益递减，在验证损失达到最低点附近（约 1.5k–2k step）作为早停与模型选择的依据。图中同时给出原始曲线与平滑曲线以抑制随机波动，结论不受平滑方式影响。该损失曲线能够证明训练过程的稳定性与收敛性。

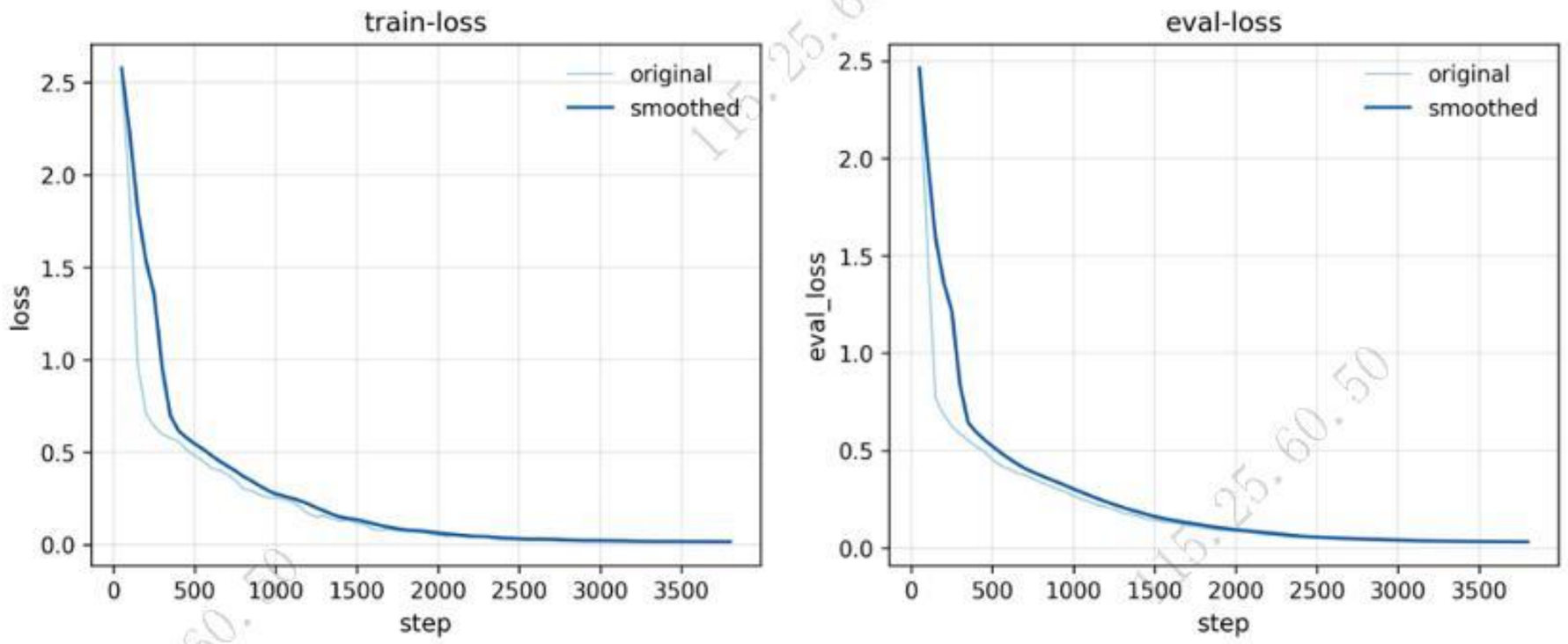


图 4-5 Qwen3-4B 模型微调实验 loss 图

Fig.4-5 Experimental loss graph for Qwen3-4B Model fine-tuning

4.5.2 自动化评估结果

本文对生成用例的四个描述项：title、step、precondition、expected 计算各自的 BERTScore F1-Score，并按任务重要度进行加权聚合，加权聚合得分的计算公式如下：

$$BS_{weighted} = \sum_{i\in\{t,s,p,e\}} w_i * BS_i \tag{4-5}$$

其中，t 表示 title，s 表示 step，p 表示 precondition，e 表示 expected。

在对微调模型生成的测试用例进行评估时，根据重要程度对用例中的 4 项描述指标设置的权重值如表 4-9。

表 4-9 测试用例描述指标权重表

Tab.4-9 Test case description indicator weight table

测试用例描述项	权重
title	0.1
precondition	0.1
step	0.4
expected	0.4

基于上述权重，Qwen3-4B 微调模型的测试数据集得分情况如表 4-10 所示。与原始 Qwen3-4B 相比，微调后的模型在加权聚合指标上由 69.65% 提升至 73.58%，整体增幅约 3.9 个百分点，相对提升幅度达到 5.6%。其中，step 与 title 两个维度提升最为明显（分别提升 5.11 和 5.59 个百分点），表明微调有效增强了模型对测试步骤逻辑与用例命名规范的理解与生成能力。在微调的基础上引入提示工程后，模型的加权聚合得分进一步提升至 74.02%。尤其在 Title 维度上，分数达到 76.64%，较原始模型提升超过 7 个百分点，说明提示策略在强化用例标题的准确性和一致性方面效果显著。同时，在 expected 维度也有一定提升。但是在 precondition 维度上，提示工程的引入使得分数由 71.40%降至 68.79%，出现一定的性能回落。这表明复杂提示可能对前置条件生成的稳定性带来干扰，需要进一步优化提示设计与微调结合的方式。

总体而言，参数高效微调是性能提升的核心因素，在整体语义贴合度和结构化质量方面带来显著改善；提示工程则在部分维度上提供了补充增益，尤其在标题生成和期望结果描述上效果突出，但在前置条件生成方面仍存在优化空间。两者结合虽存在局部波动，但整体呈现协同优势，为后续构建更稳定、全面的测试用例生成方案奠定了基础。

表 4-10 微调模型实验结果

Tab.4-10 Experimental Results of Fine-Tuned Models

实验组	加权聚合 (F1%)	Expected (F1%)	Precondition (F1%)	Step(F1%)	Title(F1%)
A	69.65	71.38	67.15	68.74	69.34
B	73.58	73.51	71.40	73.26	74.93
C	74.02	73.96	68.79	73.65	76.64

4.5.3 人工评估结果

从正确性、可执行性、完备性和创新性四个维度对生成结果进行评分，并按需求类型计算算术平均分，计算公式同公式(3-2)。结果如表 4-11 所示，其与自动化评估加权聚合得分的对比图如图 4-6 所示。

表 4-11 人工评估结果

Tab.4-11 Manual Evaluation Results

实验组	正确性	可执行性	完备性	创新性	综合得分
A	4.63	3.33	3.13	2.27	3.34
B	4.53	3.93	4.4	4.00	4.22
C	4.70	4.70	4.60	3.90	4.48

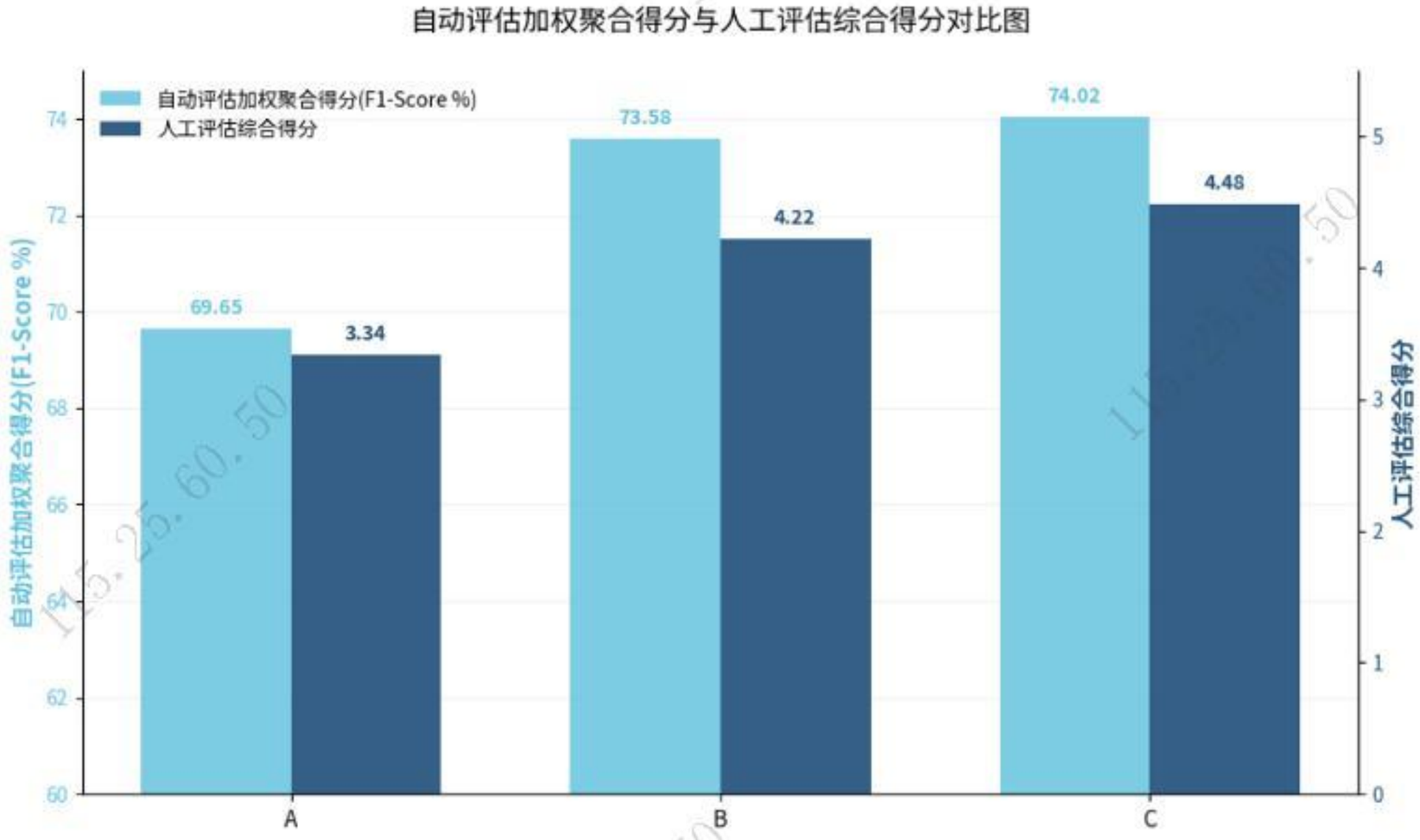


图 4-6 自动化评估加权聚合得分与人工评估综合得分对比图

Fig.4-6 Comparison of Automated vs. Human Evaluation Overall Scores

结果表明，融合提示策略与 QLoRA 微调相结合的协同策略可显著提升测试用例的生成质量。相较于仅依赖微调的方案，融合结构化提示后，生成用例在可执行性与完备性上进一步优化，综合得分达到 4.48，为三组中最高。正确性上，微调后因场景延伸出现微小偏差，提示工程修正偏差并补充细节，使之提高；可执行性从最初 3.33 的操作模糊，经微调补全参数，再到提示工程与微调的量化标准，解决了微调前操作方式模糊、验证标准不明确的关键痛点；完备性从基础场景覆盖，到微调后补全边界/异常场景，

最终融合提示工程与微调技术实现全维度覆盖。模型学会了生成更全面的测试场景，从仅验证功能是否能用升级为验证功能在各种场景下是否都能用，用例类型包括边界条件、异常流程和多路径覆盖，显著增强了缺陷发现能力。在创新性上也有显著进步，微调后模型能够提出传统测试设计可能忽略的新颖场景，体现了 LLM 在探索性测试中的独特价值。提示工程与微调协同方案的评分虽小幅下降，但能更精准地聚焦高价值场景并有效剔除冗余内容。

微调是质量提升基础，解决场景不全、操作模糊等问题；提示工程是进阶关键，修正偏差、优化细节、聚焦价值创新，推动用例从能用到优质。提示工程与微调的协同方案综合得分 4.48，达专业优质水平，可满足生产级测试需求，显著降低用例设计成本，提升测试效率与缺陷发现率。

4.6 本章小结

本章完成了从领域数据构建到模型微调验证的全流程研究：首先构建多源融合的测试领域数据集，通过一系列数据预处理操作，结合 Prompt-Completion 格式标准化，形成高质量训练数据；随后基于硬件资源约束与结构化输出能力评估，选定 Qwen3-4B 为基座模型，采用 QLoRA 技术实现参数高效微调，并设计 Answer-Only 损失策略（仅对模型生成的测试用例片段计算损失），提升训练效率与目标任务适配性。最后，在测试集上，以 BERTScore 为自动化指标并辅以人工盲评（正确性、可执行性、完备性、创新性四维）进行评估，结果表明微调模型在语义贴合与用例结构化质量上优于未微调基线。

同时，本章方法仍存在待优化方向：一是评估指标仍有盲区，BERTScore 虽能衡量语义相似性，但无法直接评估测试用例的可执行性、边界覆盖度、异常路径完整性等工程关键属性，仍需依赖人工评审。二是消费级 GPU（如 RTX 4060Ti）上的 QLoRA 训练需精细调参，且迭代速度较慢。为此，后续可从两方面推进，一方面是将大语言模型生成的测试用例（尤其是步骤与预期结果）自动转换为可运行的测试脚本（如 Pytest），通过实际执行结果（通过/失败/异常）来量化“可执行性”与“正确性”。另一方面则是在更高算力平台上对比 DoRA、PiSSA 等新型 PEFT 方法的量化兼容性，并探索模型蒸馏技术，实现消费级设备上的高效推理与增量微调。

5 系统设计与实现

本章节将围绕系统的整体构建过程展开，系统性地阐述从需求分析到系统实现的过程。内容主要包括需求分析、架构设计、数据库设计以及系统具体实现四个部分。

5.1 需求分析

5.1.1 系统的功能性需求分析

(1) 登录功能

当用户访问系统时，系统应自动验证其认证状态。若未发现有效身份令牌（Token），则需重定向至登录界面。系统仅允许处于“启用”状态的账户进行登录操作。如账户状态为“禁用”，应向用户明确提示“用户账户已被禁用，请联系管理员”。对于用户名或密码错误、或账户不存在等情形，系统应统一返回提示信息“用户名或密码错误，请重新登录”，以避免暴露账户是否存在等敏感信息，增强系统安全性。

(2) 用户管理功能

本系统在用户管理方面需要实现对用户账户的全生命周期控制。管理员应具备创建新用户、编辑、删除现有用户信息以及启用或禁用账户的权限。创建用户时，管理员需填写用户名和密码等必要信息，系统需对用户名进行唯一性验证，确保无重复后方可成功创建；新用户账户默认状态应为“启用”，并可立即登录系统。对于已存在的用户，允许管理员在任何状态下修改其账户信息（包括用户名与密码）。出于系统安全考虑，管理员应能够随时禁用指定账户以阻止其登录，防止未授权访问，并在必要时重新启用该账户以恢复其访问权限。系统还需提供用户查询功能，支持按用户名的模糊搜索，并可结合账户状态进行筛选，结果以分页列表形式展示用户名和状态等关键信息。

(3) 项目管理功能

项目管理模块的核心目标是为测试用例提供分类与分组管理机制。系统用户在创建项目时需填写项目名称、负责人、项目状态（包括“未开始”、“进行中”及“已完成”）和项目描述，系统将对项目名称实施唯一性校

验以避免重复。项目创建成功后，系统以列表形式展示项目名称、负责人、创建时间、状态及需求对应的测试用例数，并支持通过快速访问功能跳转至对应项目的测试用例列表。对于已创建的项目，用户可修改其名称、负责人、状态与描述内容；若项目中尚未创建任何测试用例，则允许删除该项目，如已关联测试用例则禁止删除，以确保数据完整性。

(4) 测试用例管理功能

测试用例管理功能涵盖用例生成与用例管理两大核心子功能。在用例生成界面，用户首先选择目标项目，随后将需求文本粘贴至输入框；点击“生成”按钮后，系统将用户需求持久化至数据库，并调用大语言模型服务自动生成初步的结构化测试用例。生成结果以可编辑表格形式展示，用户可对内容进行调整与确认，最终批量保存至系统。在用例列表界面，系统支持按所属项目、用例状态或关键词对已保存的用例进行检索，并提供对单条用例的编辑、删除及批量导出操作（支持 CSV 与 Excel 格式），以满足测试执行与文档归档的实用需求。

(5) 日志功能

为满足系统审计与安全运维要求，需实现完备的日志模块。该模块应记录关键用户操作行为，包括但不限于用户登录、用户账户的创建/编辑/删除/启用/禁用、项目的创建/编辑/删除、测试用例的创建/编辑/删除等事件。鉴于日志数据量较大，需提供多维度查询功能。管理员应可基于以下条件进行筛选：操作类型、日志级别（如：信息、警告、错误）、用户名、资源类型（通过下拉菜单选择，选项包括：用户、项目、测试用例）、时间范围（支持按起始时间与结束时间筛选，精确至时分），此外，日志模块应提供基本统计功能，支持实时展示最近 24 小时、最近 7 天及日志总数，以便管理员监控系统使用状况与活跃度。

(6) 基于角色的访问控制功能

为保障系统安全与权限最小化原则，本系统实现三层角色权限模型：管理员、测试人员、开发人员。管理员拥有全系统操作权限，涵盖用户管理、项目管理、测试用例管理及操作日志审计；测试/开发人员：仅授权访问项目管理与测试用例相关功能，禁止操作用户管理与日志模块；所有权限判定均在后端服务层强制执行，确保无论前端界面或 API 调用，均无法绕过权限校验，实现纵深防御的安全架构。系统用例图如图 5-1 所示。

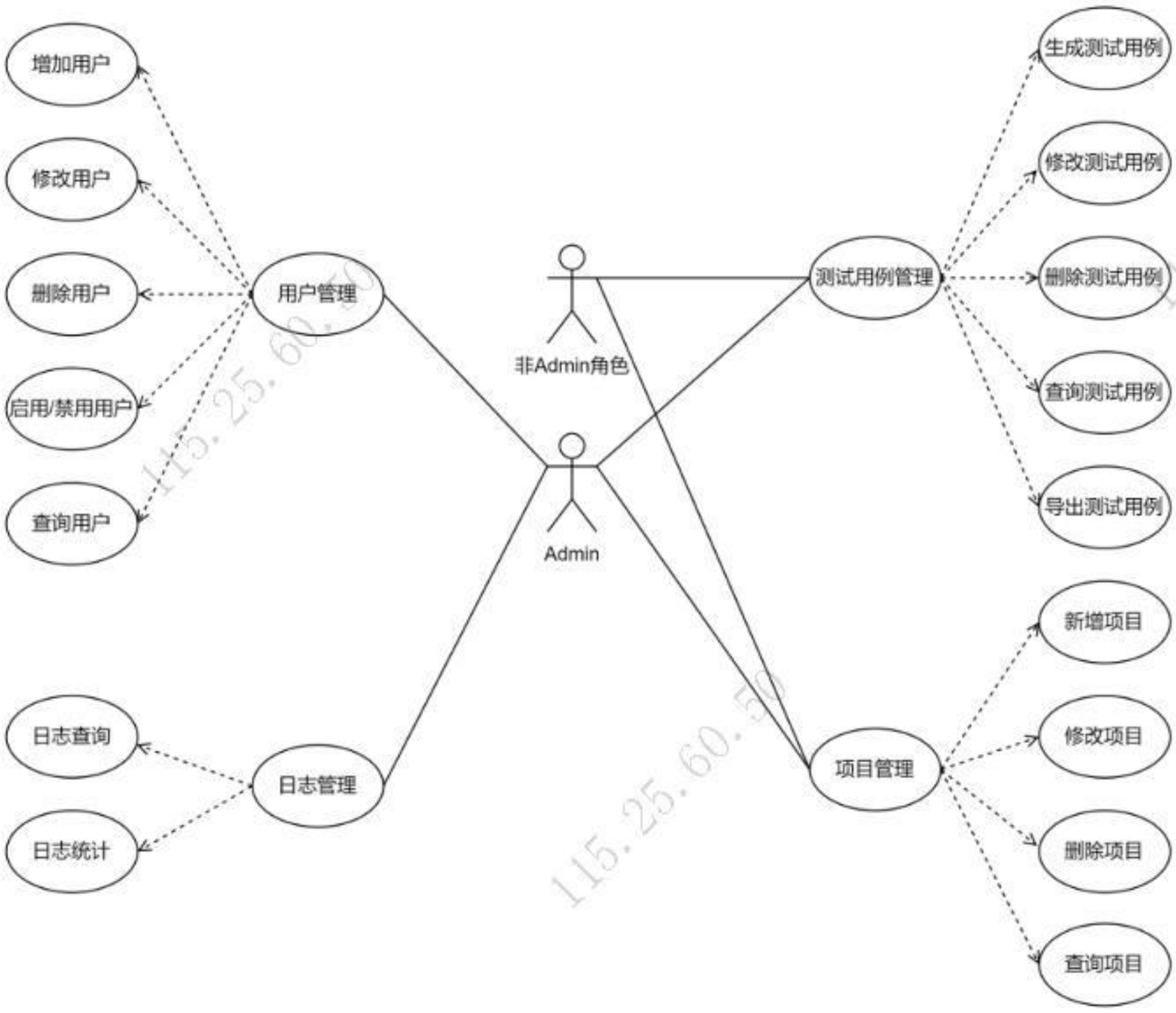


图 5-1 系统用例图

Fig.5-1 Use Case Diagram of the System

5.1.2 系统的非功能性需求

系统的非功能性需求指的是指软件产品为满足用户业务需求而必须具有且除功能需求以外的特性。非功能性需求也可称为软件开发的“约束”，这主要是因为从最简单的到最复杂的软件系统，都有反映软件系统质量和特性的额外要求，它从各个角度对所考虑的可能采取的解决方案起约束和限制作用。软件的非功能性需求主要是软件系统的性能、可靠性和可用性、出错处理等多个方面[77]。本系统的非功能性需求涵盖以下几个方面：

(1) 易用性与兼容性需求

用户界面应保持风格统一、交互简洁、表单校验及时、错误提示清晰，并在关键页面提供操作指引或帮助文档。系统需兼容主流桌面浏览器（Chrome、Firefox、Edge、Safari），字符编码统一为 UTF-8，以保证多语言文本和特殊符号的正确显示与输入。

(2) 安全性

用户密码禁止明文存储；对所有用户输入（如测试用例名称、描述、提示词）实施严格的校验与过滤，防范 SQL 注入、跨站脚本、命令注入等

常见攻击，输出到前端的数据应进行编码或转义。此外，系统还需重点关注性能与可靠性等，以满足高效、可靠运行的要求。非功能性需求是对功能性需求的有力补充，二者相辅相成，方能保障系统的正常运行。

5.2 架构设计

系统采用分层架构设计，如图 5-2 所示，整体划分为用户界面层、业务逻辑层、模型服务层及数据存储层，各层之间职责清晰、耦合度低，保证了系统的可维护性与可扩展性。

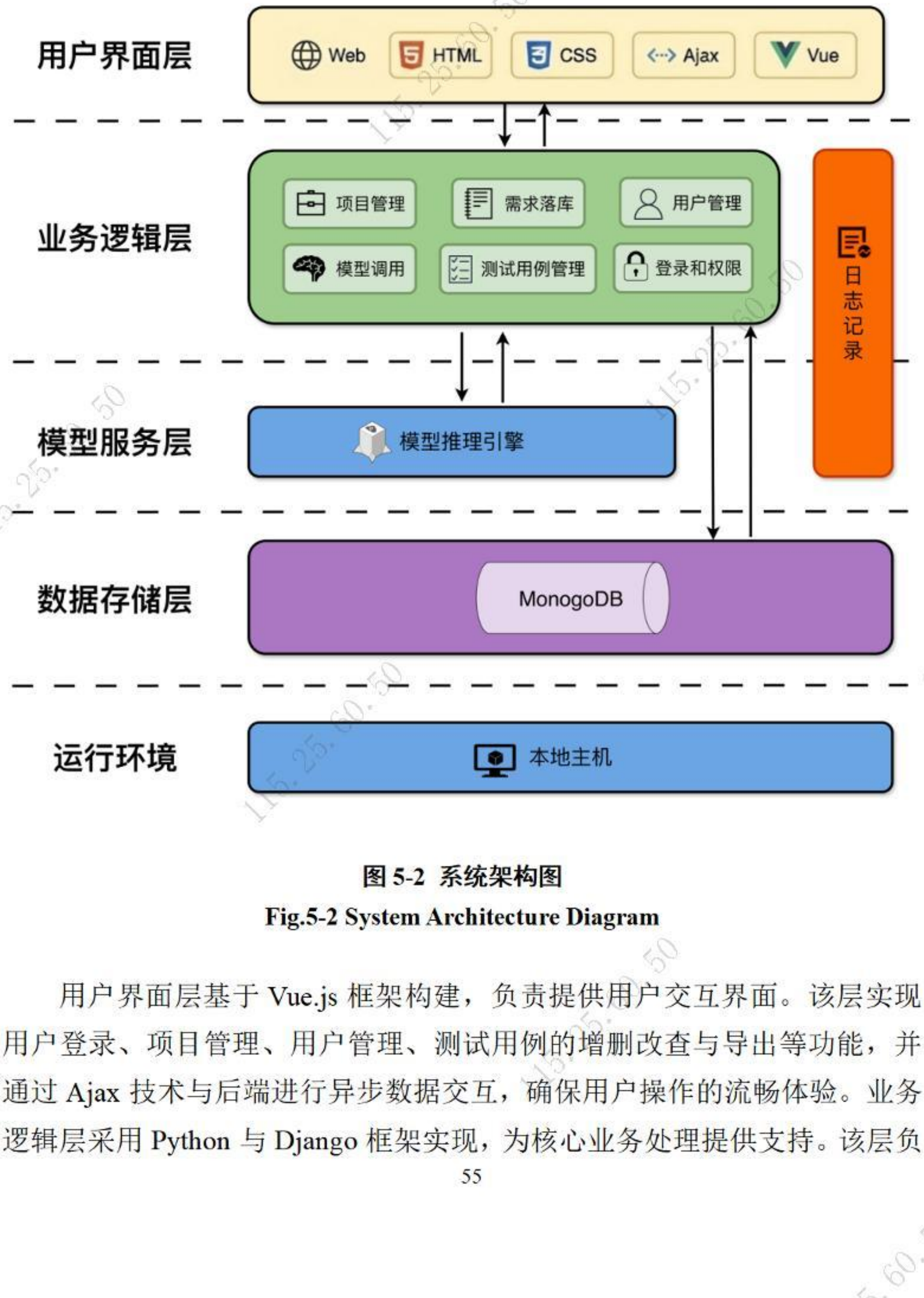


图 5-2 系统架构图

Fig.5-2 System Architecture Diagram

用户界面层基于 Vue.js 框架构建，负责提供用户交互界面。该层实现用户登录、项目管理、用户管理、测试用例的增删改查与导出等功能，并通过 Ajax 技术与后端进行异步数据交互，确保用户操作的流畅体验。业务逻辑层采用 Python 与 Django 框架实现，为核心业务处理提供支持。该层负

责用户身份认证、角色权限校验、项目管理、用户管理、测试用例管理等核心功能，同时协调下层模型服务完成测试用例的生成与处理，承担系统主要的业务规则与流程控制。模型服务层集成了模型推理引擎，专门处理与大语言模型的交互及测试用例生成任务。该层接收业务逻辑层发送的请求（如用户需求文档），经推理计算后返回结构化的测试用例数据，向上层提供统一的模型调用接口。数据存储层使用 MongoDB 数据库管理系统，利用其 BSON（Binary JSON）格式天然支持半结构化数据的特性，高效存储与检索需求文档、生成的测试用例及相关评估结果。

各层之间通过明确定义的接口进行通信，形成清晰的数据流与控制流，共同支撑系统的稳定运行与功能实现。该架构在保证系统功能完备性的同时，也为后续功能扩展与性能优化提供了良好基础。

5.3 数据库设计

5.3.1 数据库概要设计

数据库概要设计旨在阐明测试用例生成系统的整体数据结构，为后续开发提供理论与实践依据。设计内容涵盖用户管理、项目管理、需求管理和测试用例管理四大模块。鉴于 MongoDB 原生支持 BSON 格式，能够高效地存储和检索半结构化数据，其文档模型的灵活性及嵌入式文档特性非常适合保存需求说明与生成的测试用例。本系统选用 MongoDB 作为核心数据存储。系统涉及 4 个核心实体，实体说明见表 5-1，系统 ER 图如图 5-3 所示。

表 5-1 实体说明表

Tab.5-1 Entity Description Table

实体	实体说明
Users	用户，负责登录并执行操作
Projects	项目，可由多名用户协作管理
UserStory	需求，用于描述业务场景
TestCase	测试用例，由模型根据 UserStory 生成，并可编辑导出

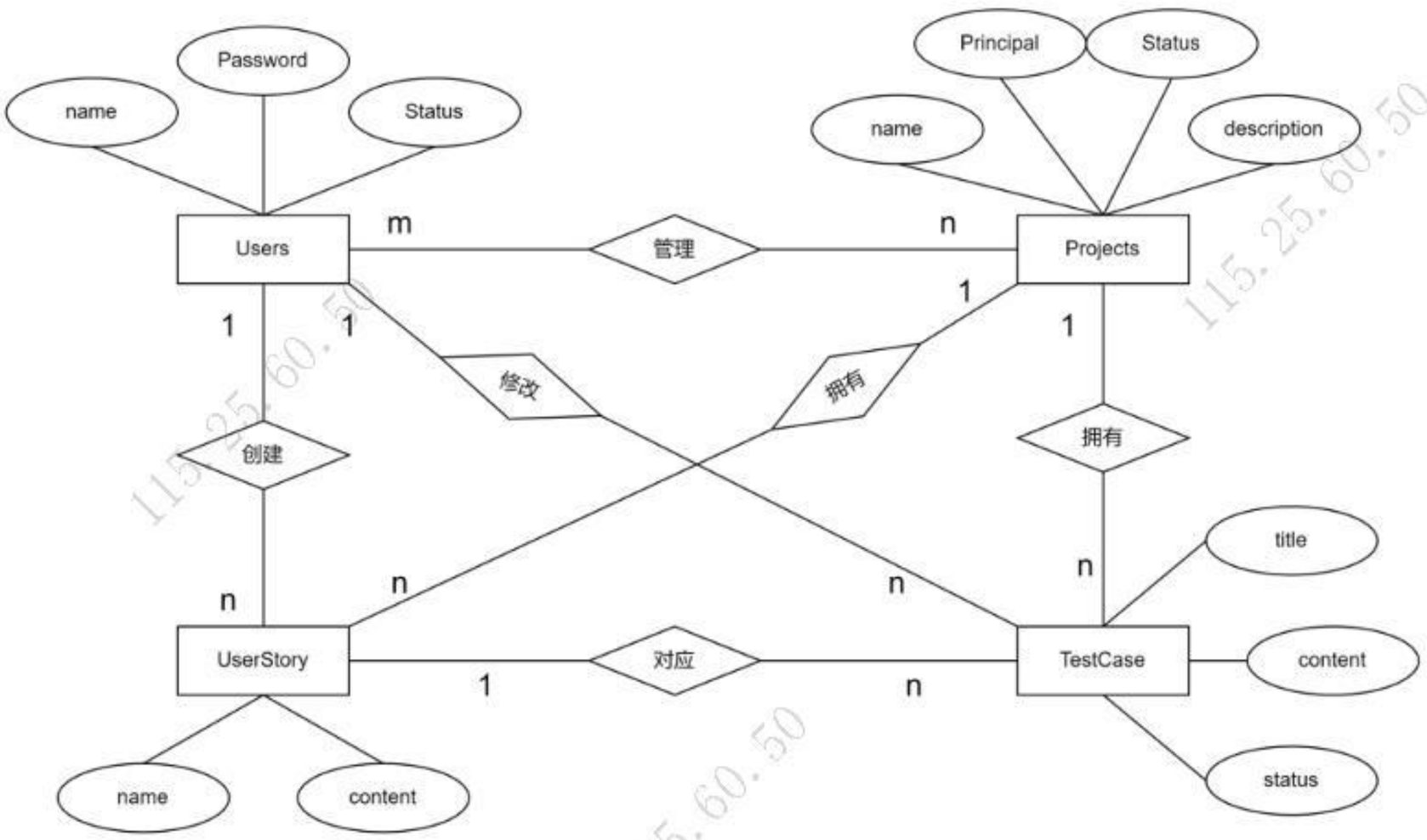


图 5-3 系统 E-R 图

Fig.5-3 E-R Diagram of the System

5.3.2 数据库详细设计

本文基于引用式建模策略组织核心数据。项目、需求与测试用例之间通过存储对方的 _id 建立关联，测试用例同时记录 projectId 与 userStoryId，分别指向所属项目与需求，需求文档则维护一个 testcaseIds 数组，记录其关联的测试用例 ID 列表，以支持从需求侧快速获取完整测试集；所有对象均通过 creatorId 关联创建用户。核心数据的关系类型与建模方式如表 5-2 所示。

表 5-2 核心业务对象建模关系表

Tab.5-2 Relationship Table of Core Business Objects and Their Modeling

业务对象	集合名	关系类型	建模方式
用户	Users	1 ↔ N （项目、需求、用例）	单向引用
项目	Projects	1 ↔ N （需求、用例）	双向引用
需求	UserStory	1 ↔ N （用例）	引用数组
测试用例	TestCase	N ↔ 1 （需求）	单向引用

(1) Users 集合的设计

Users 集合用于存储系统用户的基础信息，支撑用户身份认证、权限关联等核心功能，其字段定义及说明如表 5-3 所示。

表 5-3 用户信息集合

Tab.5-3 User Information Collection

字段名称	字段类型	描述说明
_id	ObjectId	用户 ID，自动生成
Name	String	用户名，必填
Password	String	用户密码，必填
Status	Boolean	true=启用， false=禁用，初始状态：false
Email	String	用户邮箱
Role	String Enum{admin, qa, rd}	用户角色，admin：管理员、qa：测试工程师，rd：开发人员
CreatedOn	Date	创建时间
LastModifiedOn	Date	修改时间

(2) Projects 集合的设计

Projects 集合用于存储项目的基础属性与关联信息，实现项目与用户、需求、测试用例的关联管理，其字段定义及说明如表 5-4 所示。

表 5-4 项目信息集合

Tab.5-4 Project Information Collection

字段名称	字段类型	描述说明
_id	ObjectId	项目 ID，自动生成
Name	String	项目名称，必填
Principal	String	项目负责人
Status	String Enum {new,in_progress,completed}	项目状态：未开始/进行中/已完成，初始状态：未开始
Description	String	项目描述
CreatedOn	Date	创建时间
LastModifiedOn	Date	修改时间
creatorId	ObjectId	引用用户表 id
userStory_ids	Array	关联需求 ID 列表
testCaseCount	Int	关联测试用例总数

(3) UserStory 集合的设计

UserStory 集合用于存储用户提出的功能需求，是大语言模型生成测试用例的核心数据源。当前端提交包含 project_id、user_id、需求名称及需求内容的请求后，系统会将需求信息写入该集合，其字段定义及说明见表 5-5。

表 5-5 需求信息集合

Tab.5-5 User Story Information Collection

字段名称	字段类型	描述说明
_id	ObjectId	需求 ID，自动生成
Name	String	需求名字，必填
Content	String	需求内容，作为大语言模型生成测试用例的主要输入。
CreatedOn	Date	创建时间
LastModifiedOn	Date	修改时间
creatorId	ObjectId	引用用户表 id
Project_id	ObjectId	引用项目表 id
Testcase_ids	[ObjectId]	一个数组字段，存储由该用户故事生成的所有测试用例的 ID。

(4) TestCase 表的设计

TestCase 集合用于存储大语言模型生成的测试用例，当模型生成测试用例后，系统会将每条用例与对应的需求 ID（us_id）关联，并批量写入该集合。该集合同时关联 Users、UserStory、Projects 三个集合，实现测试用例的全链路溯源，其字段定义及说明如表 5-6 所示。

表 5-6 测试用例集合

Tab.5-6 Test Case Collection

字段名称	字段类型	描述说明
_id	ObjectId	用例 ID，自动生成
Title	String	用例标题
Precondition	String	前置条件
Steps	Array	包含 index、action、expected
Status	Boolean	用例状态：true=已评审， false=未评审
StepsText	String	派生字段：测试步骤（优化查询）
ExpectedText	String	派生字段：预期结果（优化查询）
CreatedOn	datetime	创建时间
LastModifiedOn	datetime	修改时间
UserStory_id	ObjectId	关联需求表 id
Project_id	ObjectId	关联项目表 id
creatorId	ObjectId	关联用户表 id

5.4 系统实现

本文设计并实现了一个基于 B/S 架构的测试用例可视化系统，采用了前后端分离的技术方案。前端以 Vue.js 作为核心框架进行构建，后端则基于 Python 与 Django 技术栈实现，前后端之间通过 HTTP RESTful API 进行数据交互与通信。系统中的用例信息及项目和用户数据均统一存储于 MongoDB 非关系型数据库中，该数据库采用分布式文件存储方式，具有灵活的 JSON 类文档存储结构，为系统提供了高性能的数据存储解决方案。

5.4.1 登录功能

登录功能采用前后端协作的安全认证架构，前端负责交互逻辑与初步校验，后端依托 Token 机制实现无状态身份验证，为系统提供统一的认证基础。前端通过数据绑定实时管理登录表单状态，结合预校验机制减少无效请求。登录请求封装于异步流程中，依据响应结果处理 Token 持久化、页面跳转或异常反馈。后端执行分层安全验证：首先查询用户信息，验证账户存在性；其次进行密码哈希比对，确保凭证安全；最后校验账户状态。验证通过后，签发含用户 ID 与时间戳的 Token，并返回过滤敏感信息后的用户数据。系统严格遵循 RESTful 规范，针对不同结果返回符合语义的状态码与结构化响应。登录业务流程图如图 5-4 所示，时序图如图 5-5 所示。

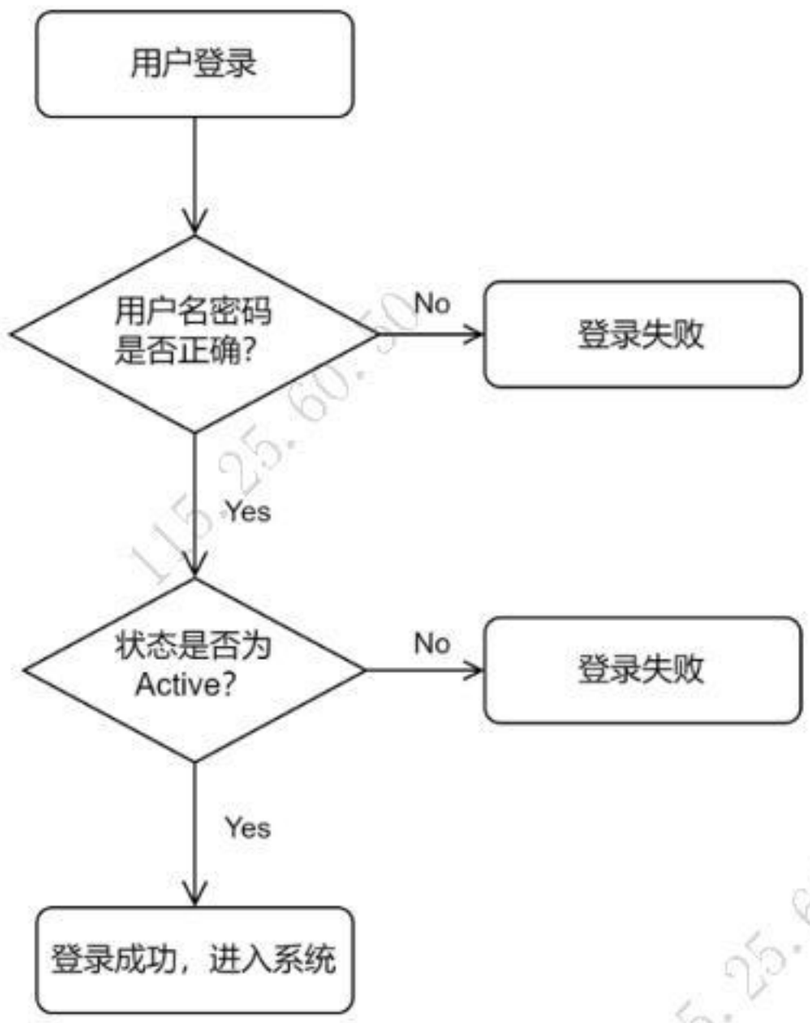


图 5-4 系统流程图

Fig.5-4 System Flow Chart

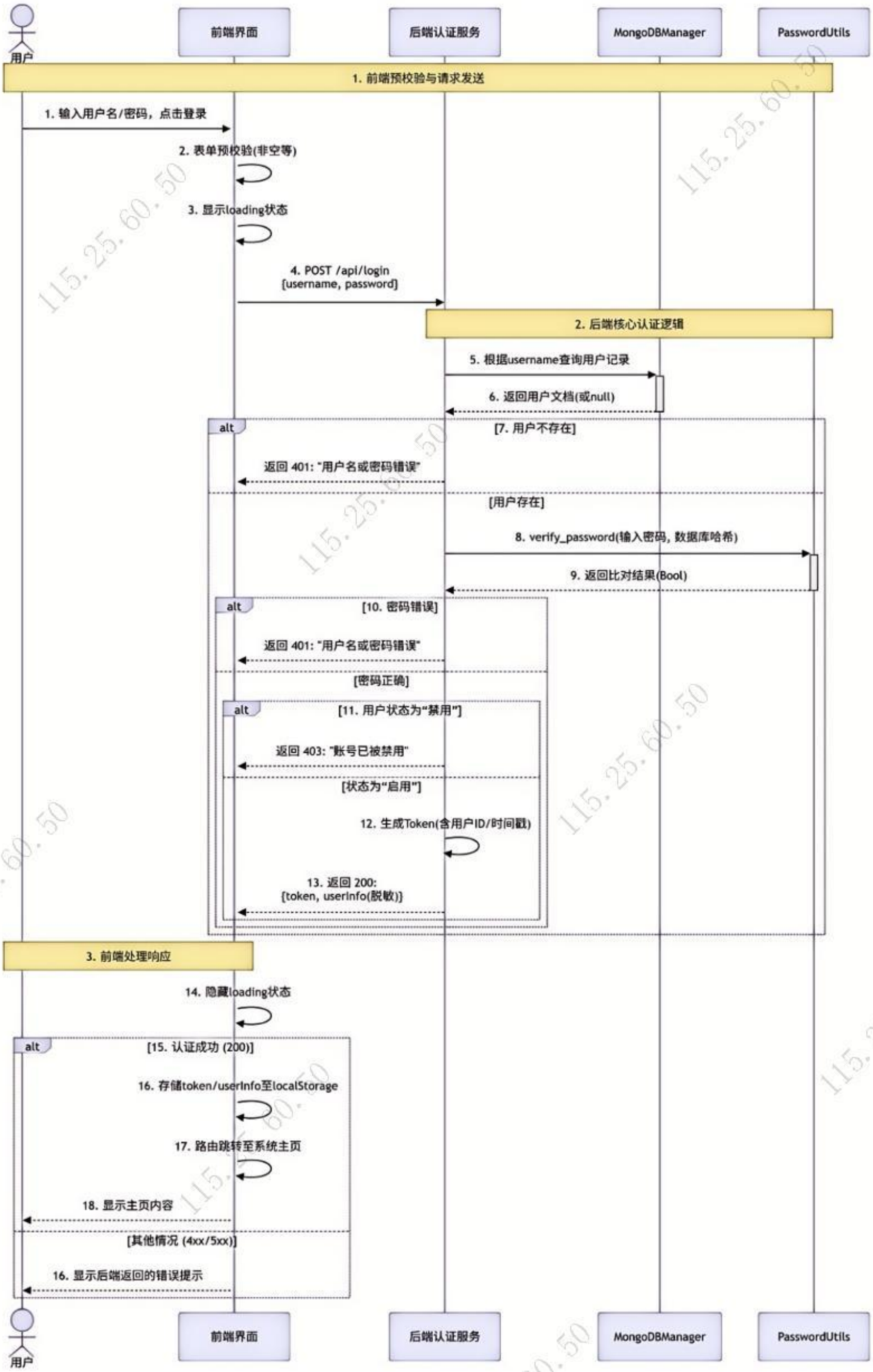


图 5-5 登录时序图

Fig.5-5 Login Sequence Diagram

系统前端展示效果如图 5-6 至图 5-8 所示。

图 5-6 账户禁用操作界面（用户 CQ）

Fig.5-6 User CQ: Operation Interface for Account Disabling

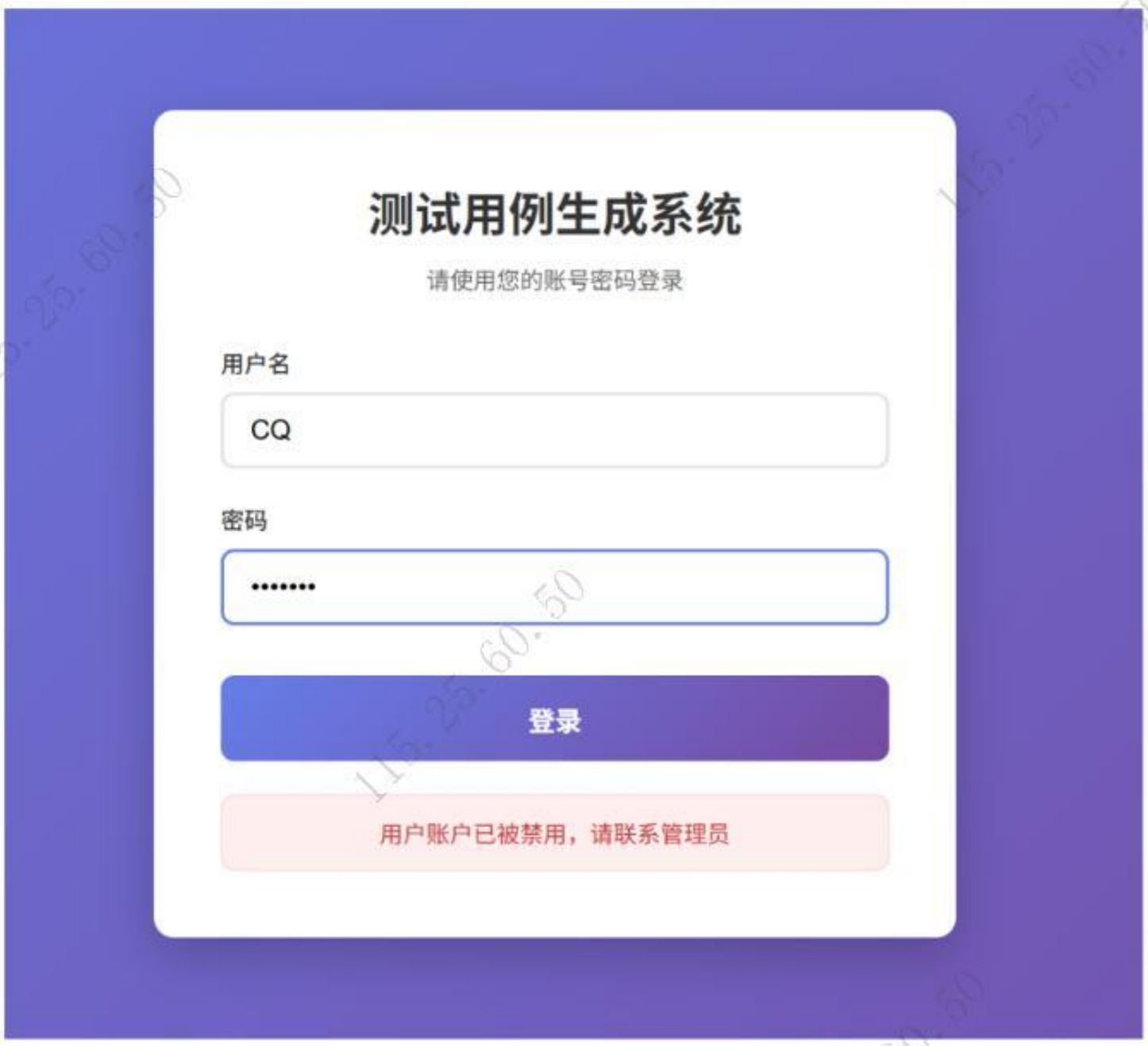


图 5-7 禁用账户访问控制验证

Fig.5-7 Access Control Verification for Disabled Accounts

图 5-8 无效认证响应机制

Fig.5-8 Response Mechanism for Invalid Authentication

5.4.2 用户管理

系统实现了覆盖用户全生命周期的管理功能，包括查询、创建、更新、状态切换与删除等核心操作。用户管理模块采用分层架构设计，由路由层、业务逻辑层、数据模型层和工具层协同完成。

路由层（urls.py）负责定义符合 RESTful 规范的 API 端点，将 HTTP 请求映射至对应的视图函数。业务逻辑层（views.py）负责处理用户管理中的业务流程与控制逻辑。数据模型层（models.py）负责定义用户数据结构及数据库交互规范。工具层（utils.py）提供通用支撑功能，包括：MongoDBManager 实现安全数据库连接；PasswordUtils 提供密码哈希、验证与强度检查；UserValidationUtils 封装用户名、邮箱等字段的格式验证逻辑，确保校验规则的模块化与复用性。

以创建用户为例，客户端发送 POST 请求至路由层，该请求被映射到相应的视图函数。视图函数调用工具层完成用户名、邮箱格式校验及密码强度验证，并利用 PasswordUtils 进行哈希加密。随后通过 MongoDBManager 执行唯一性检查，最终将用户数据写入数据库。用户管理模块的时序图如图 5-9 所示。

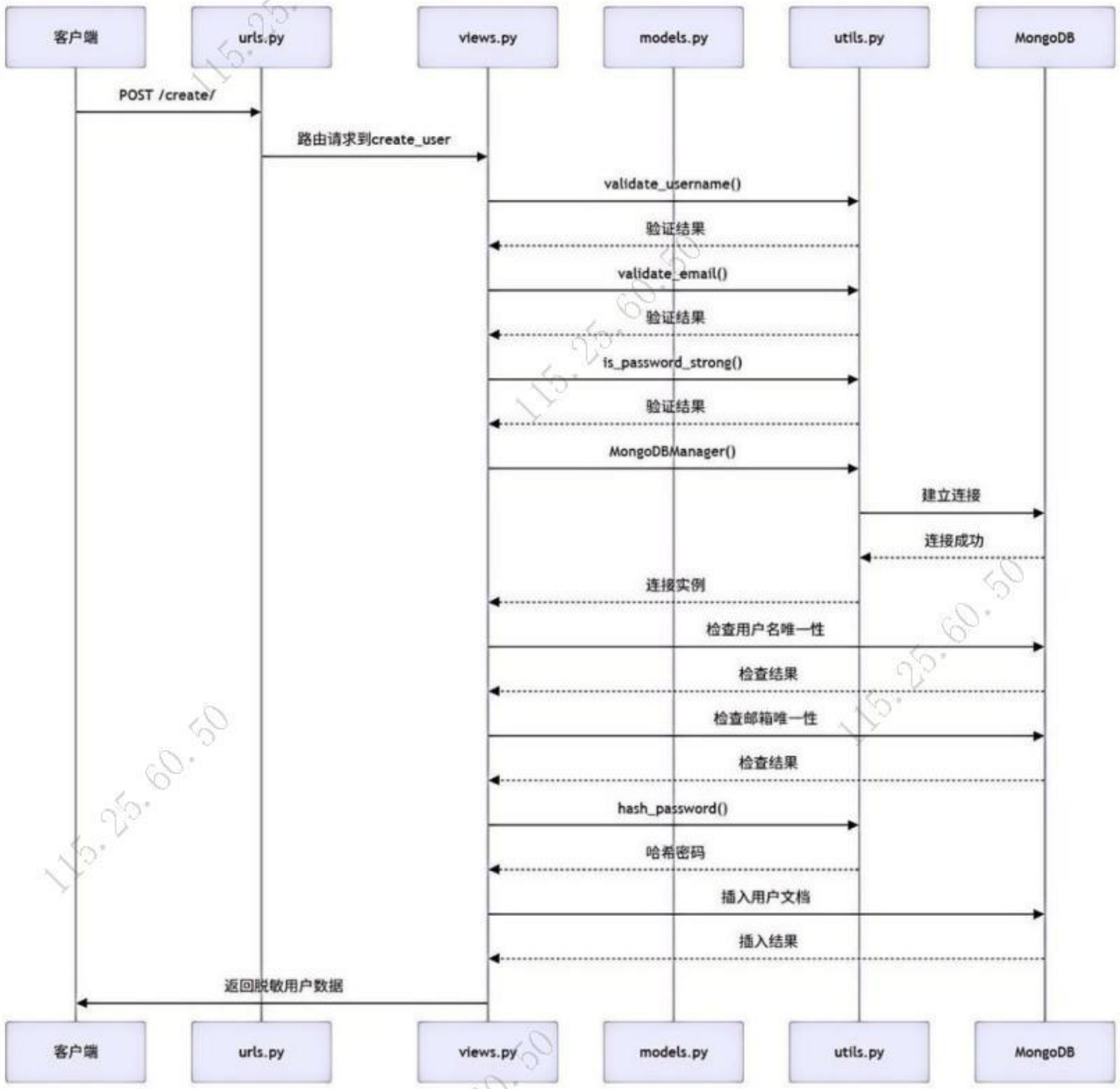


图 5-9 用户创建流程时序图

Fig.5-9 Sequence Diagram for User Creation Process

前端用户管理界面基于 Vue.js 框架实现，主要功能模块包括用户列表展示、多维度检索筛选及用户数据的增删改查操作，具体如下：

(1) 用户列表可视化呈现：系统以结构化表格形式展示用户核心属性，包括用户名、电子邮箱、角色标识、账户状态，并提供对应操作入口，便于管理员进行后续管理。用户管理系统页面见图 5-10 所示。

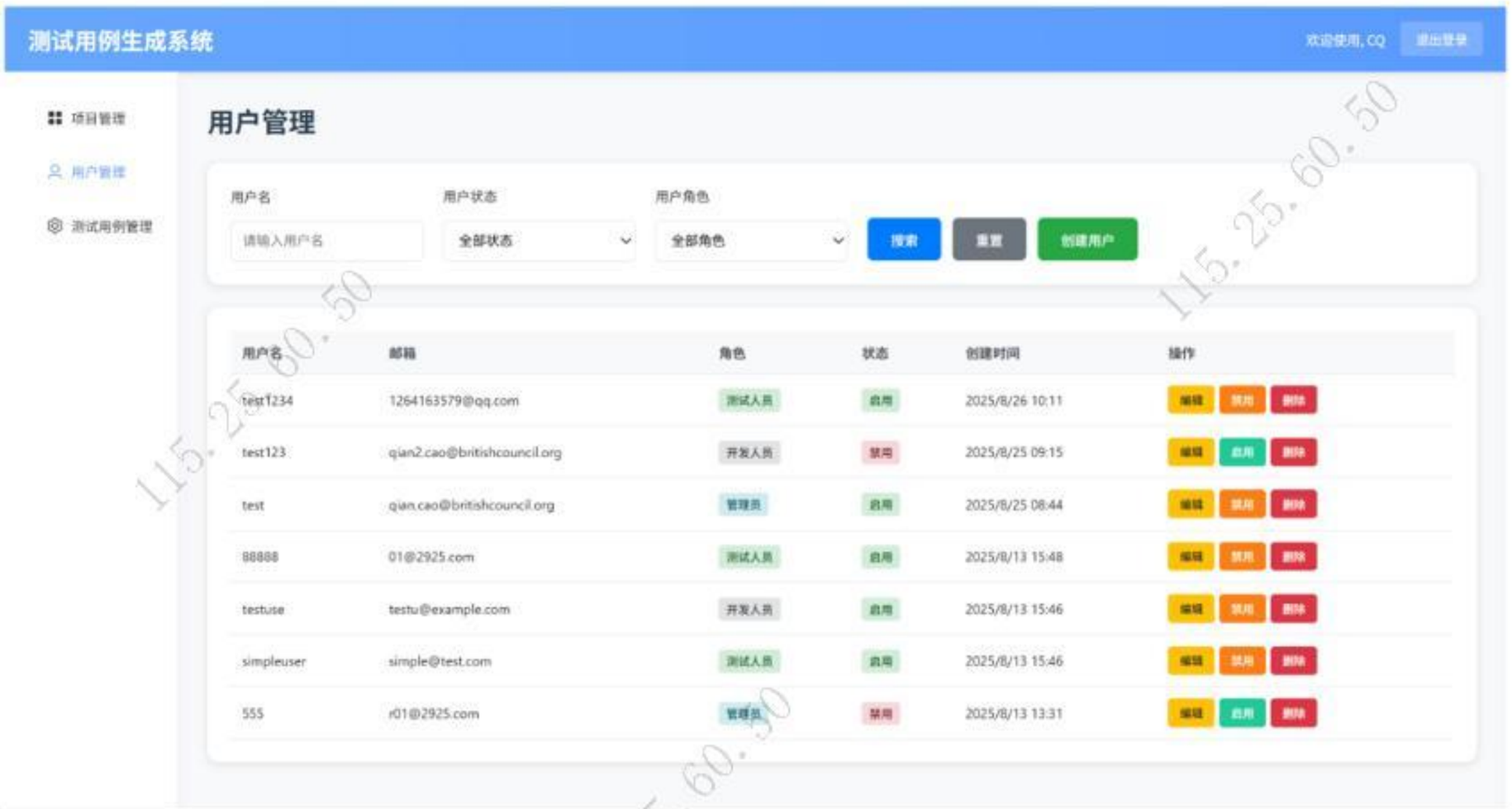


图 5-10 用户列表

Fig.5-10 User List

(2) 用多条件组合筛选与检索：支持基于用户名、账户状态及角色类型三类关键字段的动态筛选功能。图 5-11 展示了系统在设定账户状态为“启用”与角色为“测试人员”的双重筛选条件下的查询结果界面，验证了筛选机制的有效性与交互响应的准确性。

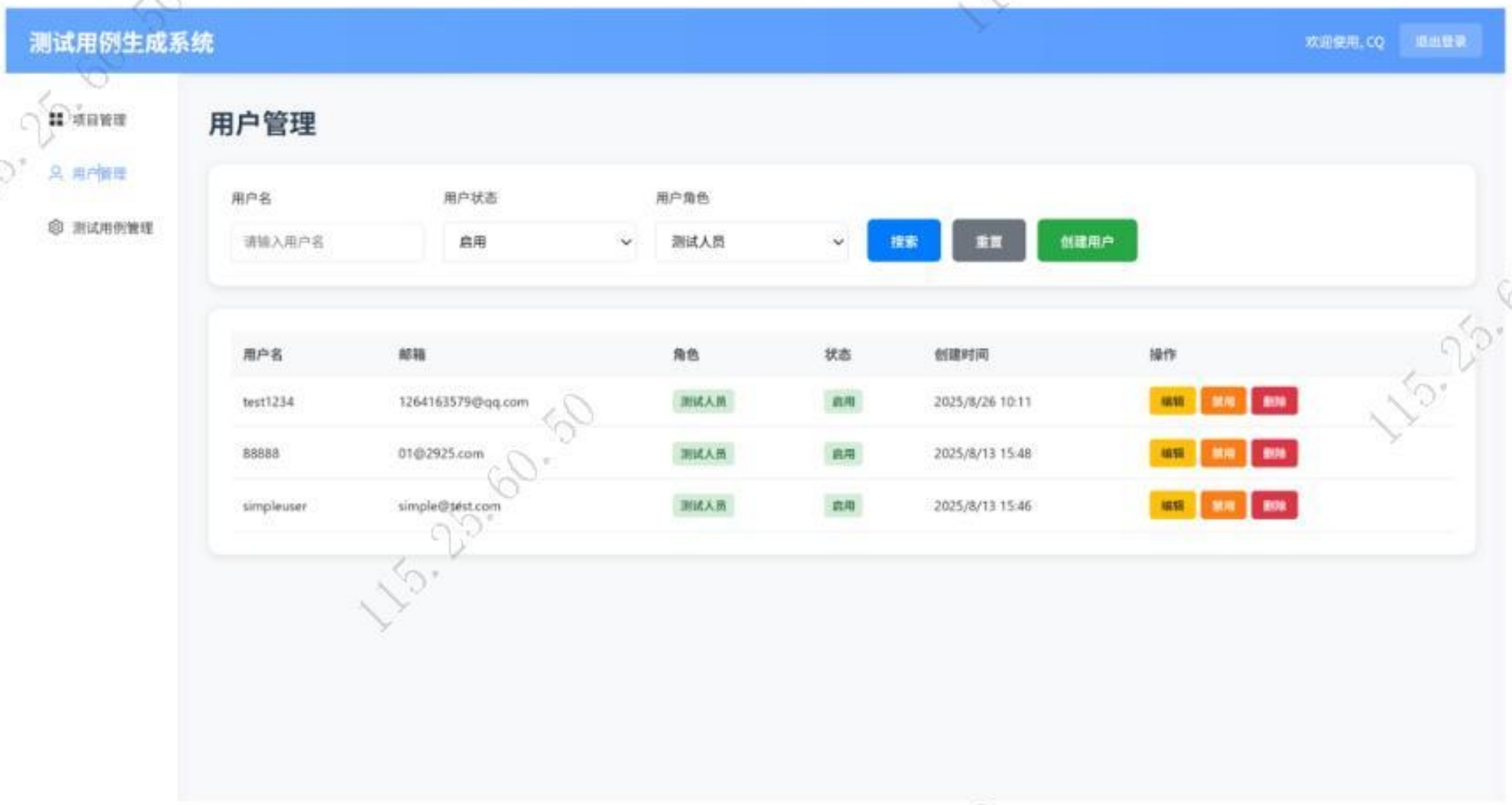


图 5-11 用户筛选结果图

Fig.5-11 User Filtering Result Diagram

(3) 用户增删改查操作：系统提供完整的用户信息增删改查操作入口，确保管理员可对用户数据实施闭环管理。图 5-12 至图 5-15 展示了新增用户、

编辑用户信息、删除账户的前端效果。

图 5-12 创建用户

Fig.5-12 User Creation

图 5-13 更新用户信息

Fig.5-13 User Information Update

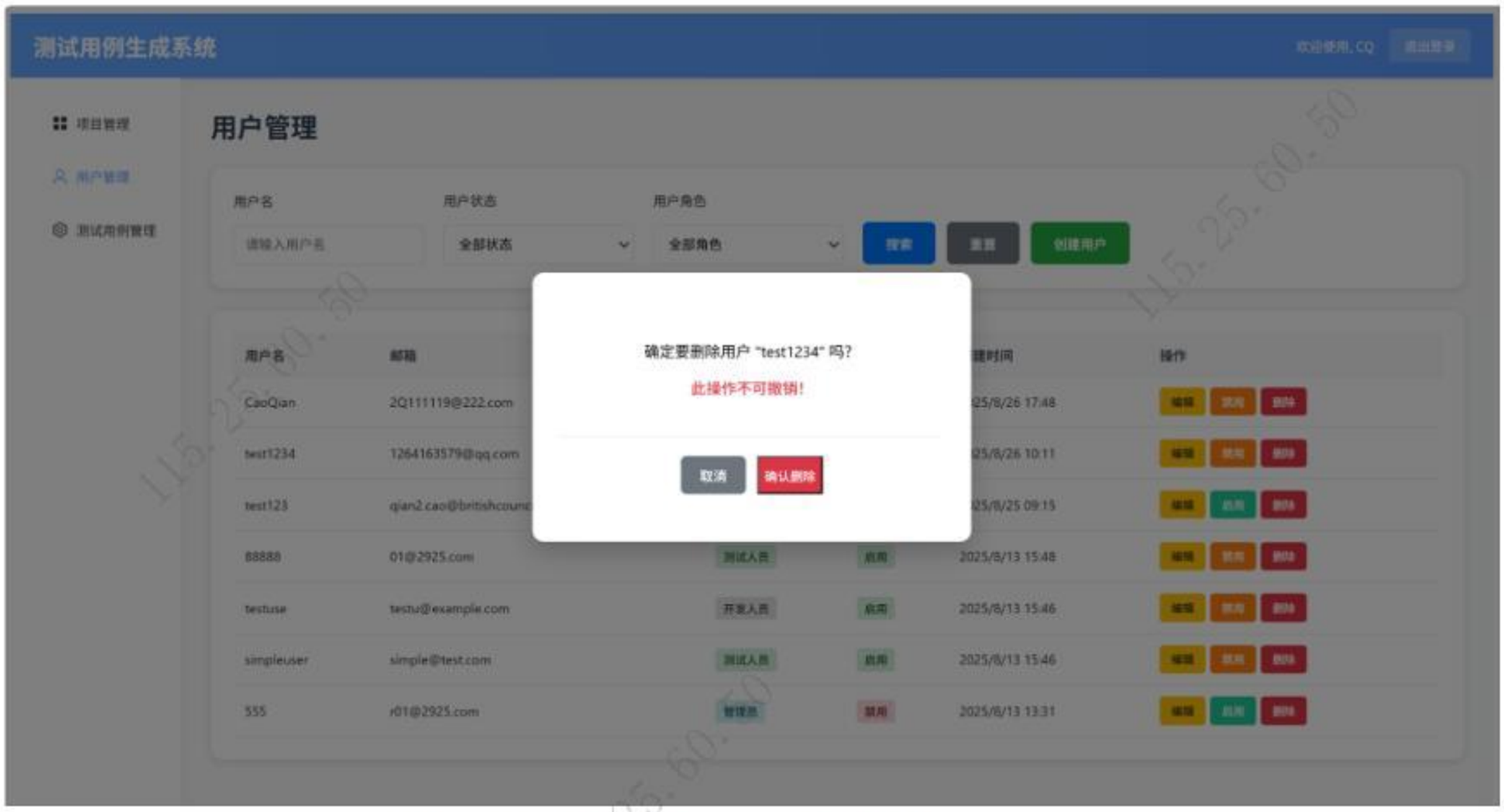


图 5-14 删除用户 - 二次确认弹窗

Fig.5-14 User Deletion - Double Confirmation Pop-up

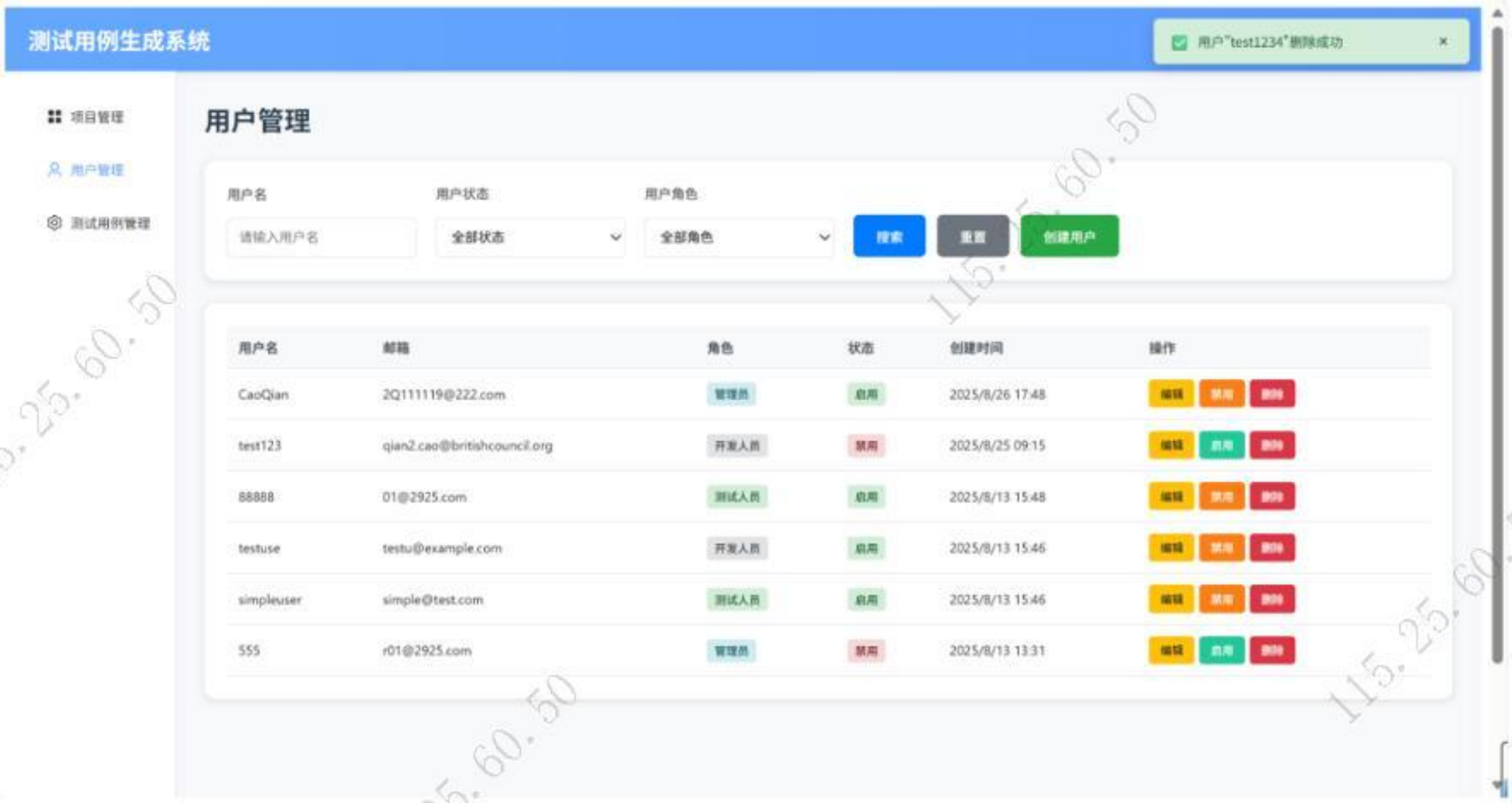


图 5-15 删除用户

Fig.5-15 User Deletion

5.4.3 项目管理

项目管理模块负责项目的全生命周期管理，采用分层架构确保数据一致性和系统性能。该模块的核心业务逻辑包括：唯一性约束：通过MongoDBManager建立数据库连接，在projects集合中执行find_one({"name": name}）查询，确保项目名称全局唯一。状态管理：使用ProjectStatus枚举

类严格定义三种状态（"new", "in_progress", "completed"）。状态变更时通过update_one操作更新数据库。关联数据统计：项目实体通过test_case_count字段实时聚合其下的测试用例总数，优化列表查询性能。

创建项目的时序图如下图 5-16 所示：创建项目时，客户端发送 POST 请求至路由层（urls.py），该请求被映射到相应的视图函数（views.py）。视图函数解析请求参数并验证必填字段，随后调用数据模型层完成业务对象构建。工具层（utils.py）提供 MongoDB 数据库连接管理，执行项目名称唯一性校验等操作，确保数据一致性。最终，新项目数据通过数据库管理器写入 MongoDB。

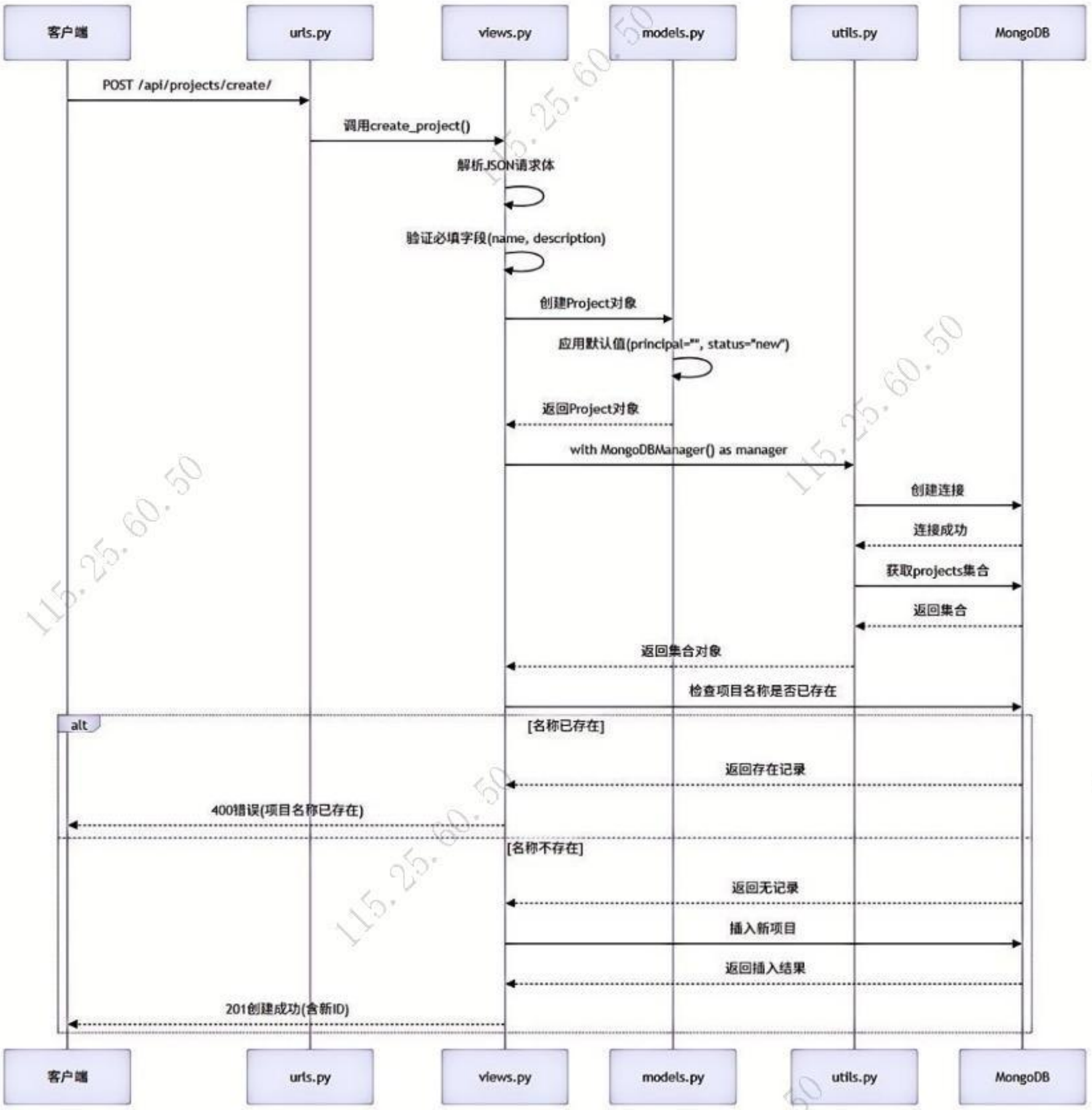


图 5-16 创建项目时序图

Fig.5-16 Sequence Diagram for Project Creation Process

前端基于 Vue.js 框架实现项目管理界面，主要包括以下功能：

(1) 项目列表展示：系统以结构化表格形式呈现项目列表，展示字段包括项目名称、负责人、项目状态、关联测试用例数量及操作控件。用户点击“测试用例”操作入口后，系统将自动跳转至测试用例管理页面，并将当前项目名称作为预设筛选条件传入，实现项目维度下的测试用例快速定位与上下文关联，提升管理效率与操作连贯性。系统页面见图 5-17 所示。

图 5-17 项目列表

Fig.5-17 Project List

(2) 项目搜索与筛选：支持按项目名称、状态等条件进行搜索；图 5-18 是根据项目名称和项目状态进行双重条件筛选的结果图。

图 5-18 根据条件筛选项目

Fig.5-18 Project Filtering by Conditions

(3) 项目增删改查操作：为实现项目数据的全生命周期管控，系统在前

端界面集成标准化 CRUD（Create, Read, Update, Delete）操作接口，支持管理员执行项目创建、属性编辑、逻辑删除及详情查阅等核心管理行为，保障数据操作的完整性与可追溯性。项目管理相关页面见图 5-19 至图 5-22。

图 5-19 创建项目

Fig.5-19 Project Creation

图 5-20 编辑项目

Fig.5-20 Project Editing

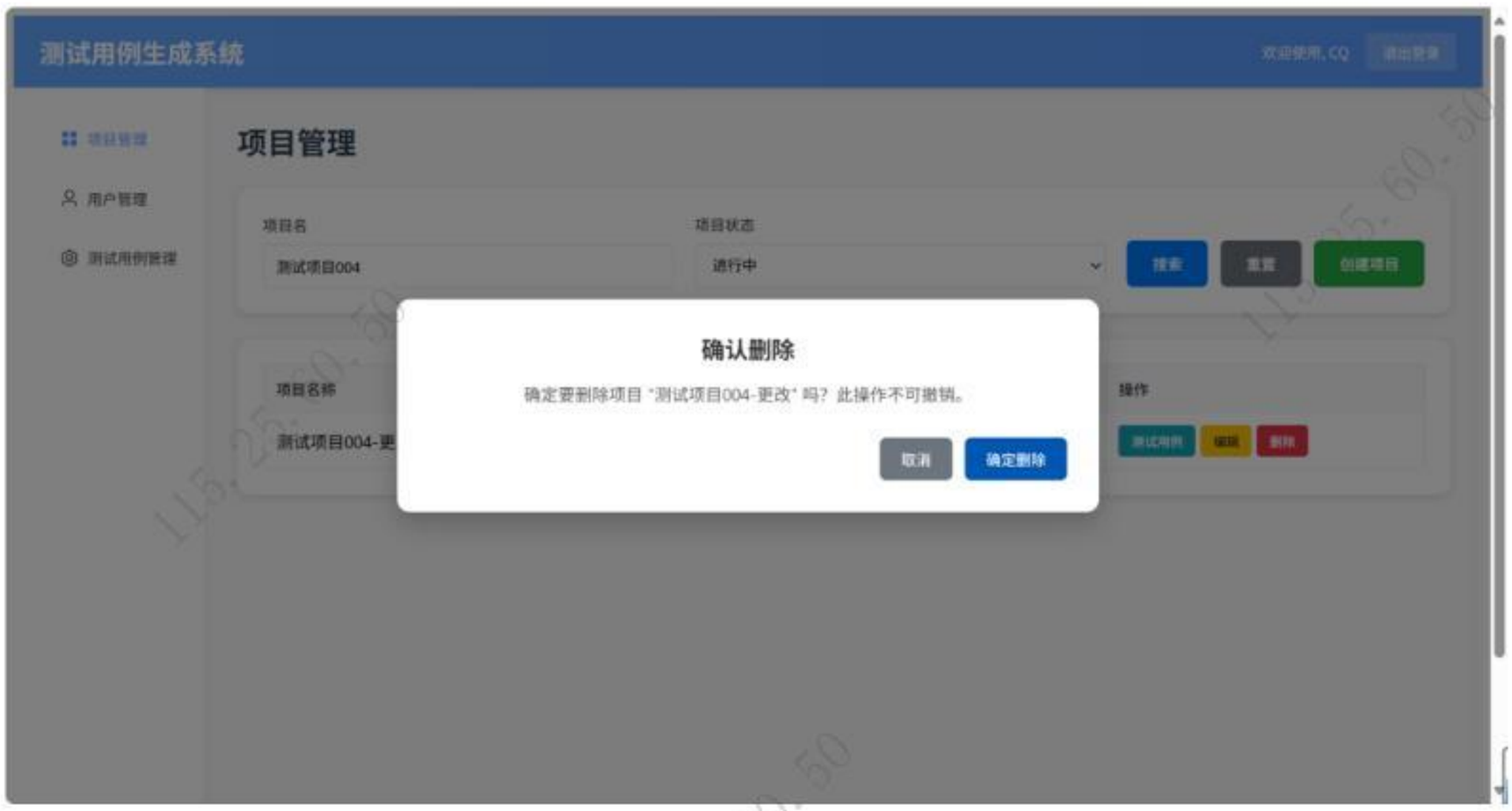


图 5-21 删除时的二次确认弹窗

Fig.5-21 Double Confirmation Pop-up for Deletion

图 5-22 删除项目

Fig.5-22 Project Deletion

(4) 用户交互与提示：通过 Element-UI 组件实现弹窗、表单验证、提示信息等交互效果。

5.4.4 测试用例管理

测试用例管理模块是本系统的核心，负责将用户需求（用户故事）转化为结构化、可执行的测试用例，并对其进行全生命周期管理。本节将深入阐述该模块的详细设计与实现。

5.4.4.1 测试用例生成

测试用例的自动化生成是本文的重点，旨在通过大语言模型实现从用户故事到结构化测试用例的智能转换，显著提升测试资产构建效率与一致性。该模块的业务流程如图 5-23 所示。

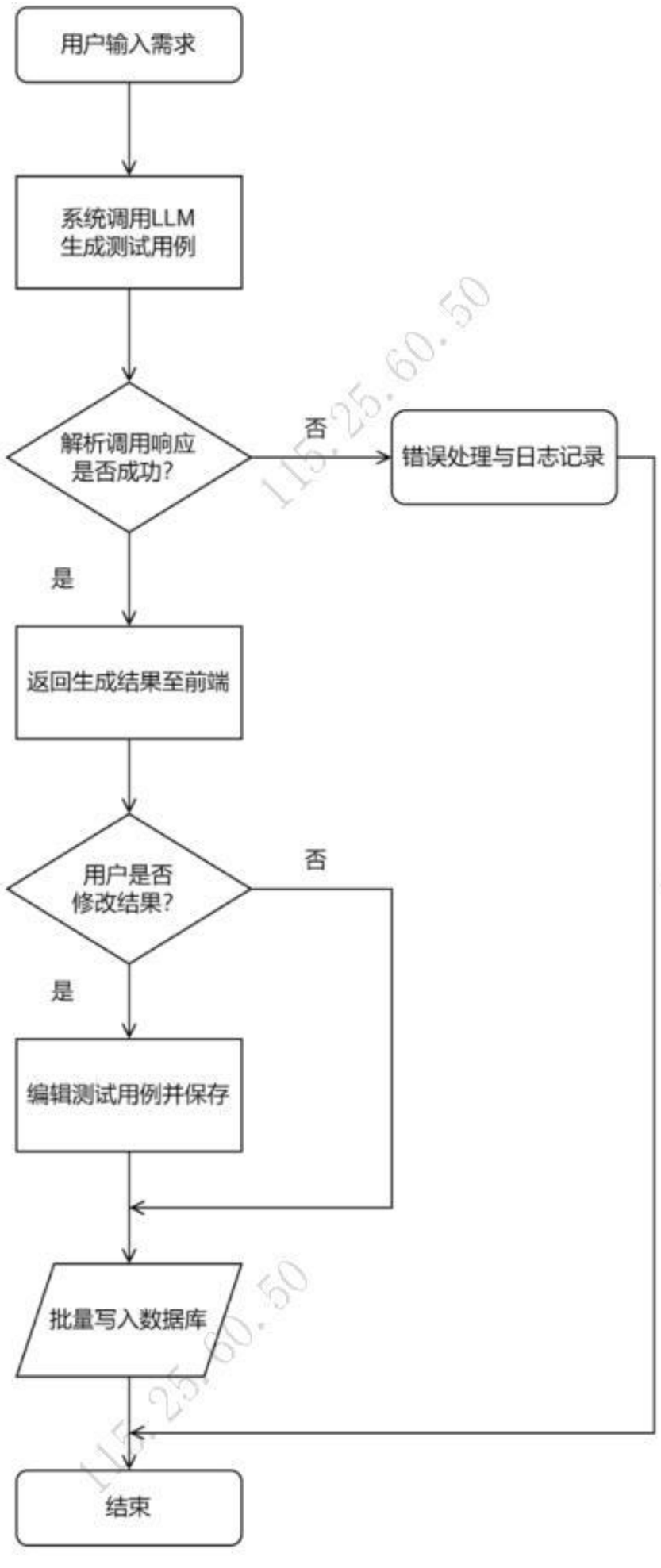


图 5-23 生成用例流程图

Fig.5-23 Use Case Generation Flow Chart

测试用例生成模块通过多服务协同实现从用户需求到结构化测试用例的完整转换流程。图 5-24 展示了该模块的详细时序关系与数据处理流程：

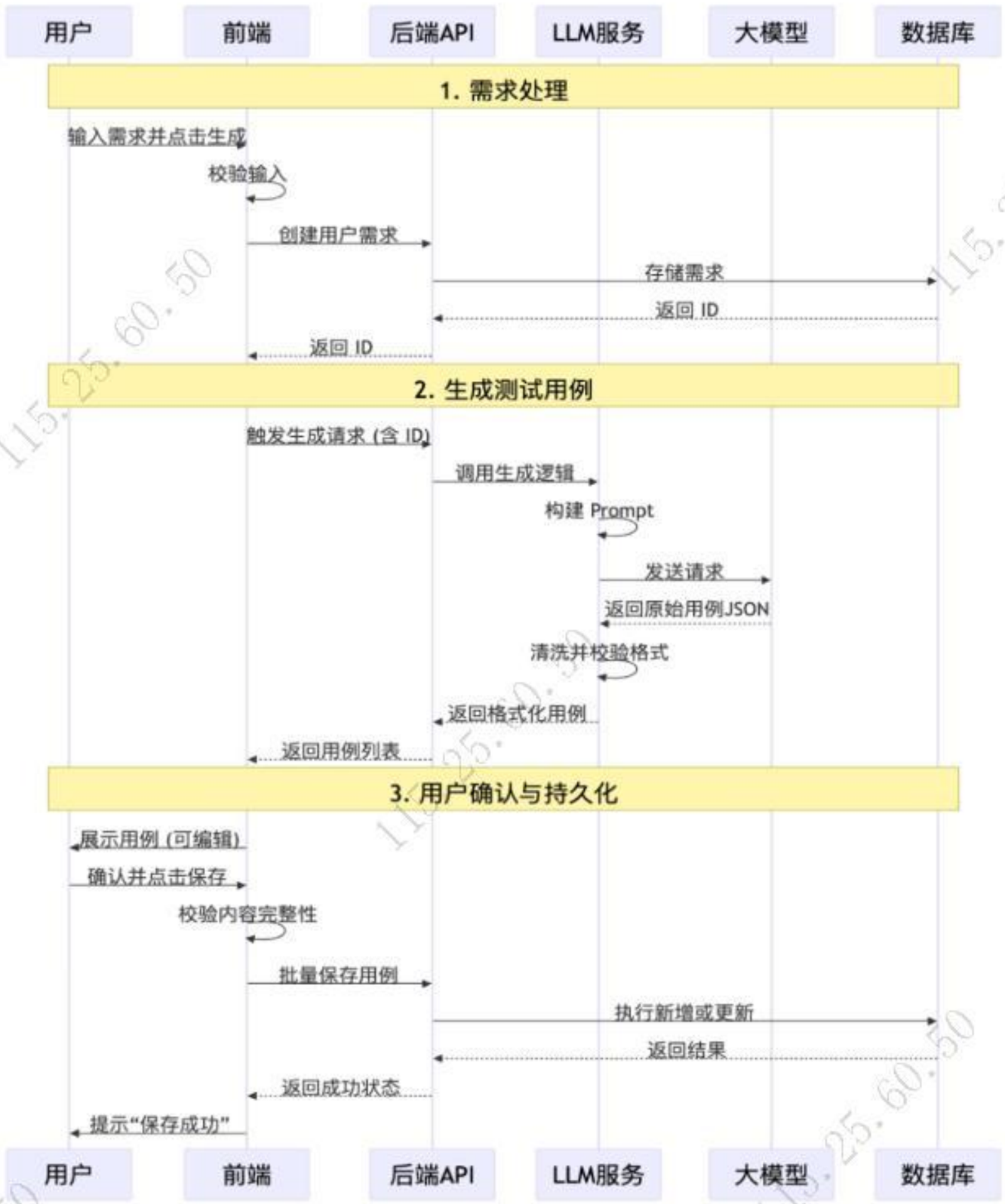


图 5-24 生成用例时序图

Fig.5-24 Use Case Generation Sequence Diagram

核心算法与关键技术实现如下：

(1) 双阶段请求处理：前端依次调用 POST /api/userstory/create 和 POST /api/testcase/generate 接口，需求先持久化至 MongoDB，获得唯一标识符（比如 US_123）后触发测试用例生成。

(2) 大语言模型交互层：采用 vLLM 框架进行高效推理，支持高吞吐、低延迟的连续批处理请求，底层服务端点通过环境变量 LLM_BASE_URL 动态配置，便于部署环境迁移与模型服务切换；基于预定义、经多轮调优的语义引导型提示词模板，动态填充用户故事的 name 与 content 字段，构造结构化输入，以引导模型输出符合预期的 JSON 格式测试用例；支持多模型架构，通过标准化 API 接口实现模型无关性。

(3) 测试步骤标准化算法：针对模型输出中步骤与预期结果数量不一致的问题，系统实现鲁棒性标准化算法，智能处理以下四类边界情况：

- 情况 A（长度相等）：直接逐项合并。

- 情况 B（预期结果多于步骤）：将多余的预期结果用分号连接，合并到最后一步的预期结果中。
- 情况 C（步骤多于预期结果）：为多余的步骤填充空的预期结果。
- 情况 D（均为空）：返回空列表。

该算法确保无论模型输出如何波动，最终持久化数据均具备统一、规范的结构，为前端展示与后续分析提供稳定数据基础。

(4) 响应解析与清洗机制：为确保大语言模型输出数据的程序可读性，本文设计了专用的响应解析器，用于处理LLM返回内容中的非结构化信息。该解析器采用多阶段清理流程：首先剥离 Markdown 代码块标记（```json）和模型内部思考标签（<think>...</think>），随后基于括号匹配原理，通过栈式结构定位首个完整 JSON 对象的起止边界。该方法有效消除了 LLM 输出中常见的解释性文本干扰，确保了 JSON 提取的完整性与准确性，最终输出符合标准格式、可直接反序列化的结构化数据。代码片段如下表 5-7 所示：

表 5-7 响应解析器代码片段

Tab.5-7 Code Snippet of the Response Parser

代码片段 1：

```
# 1. 提取模型生成的核心文本内容
content = response_data['choices'][0]['message']['content'].strip()
logger.debug(f"LLM 原始返回内容: {content[:500]}...")

# 2. 清洗 Markdown 代码块标记
if content.startswith('```json'):
    content = content[7:]
if content.endswith('```'):
    content = content[:-3]
content = content.strip()

# 3. 清洗模型内部思考标签 (<think>...</think>)
if content.startswith('<think>'):
think_end = content.find('</think>')
if think_end != -1:
    content = content[think_end + 8:].strip()   # 移除 Think 标签及内容
else: ..... # 后备策略：如果没有结束标签，直接寻找 JSON 起始位置
```

表 5-7 续

Tab.5-7 Continued

代码片段 2：

```
# 4. 精准定位 JSON 字符串的边界
    json_start = content.find('{')
    if json_start == -1:
        raise LLMClientError("LLM 返回内容中未找到 JSON 起始标记 '{' ")
    brace_count = 0
    json_end = -1
    # 遍历字符串，匹配大括号，以找到最外层 JSON 的结束位置
    for i, char in enumerate(content[json_start:], start=json_start):
        if char == '{':
            brace_count += 1
        elif char == '}':
            brace_count -= 1
            if brace_count == 0:  # 当括号计数器归零，找到匹配的结束括号
                json_end = i + 1
                break
    if json_end == -1:
        raise LLMClientError("LLM 返回的 JSON 括号不匹配，格式不完整")
    json_content = content[json_start:json_end]  # 精确截取 JSON 字符串

    # 5. 解析 JSON 并处理可能的转义错误
    try:
        return json.loads(json_content)  # 尝试解析 JSON
    except json.JSONDecodeError as e:
        logger.warning(f"JSON 首次解析失败，尝试修复转义字符: {e}")
        # 修复常见的转义字符问题
        json_content = json_content.replace('\\"', '"').replace('\\\\', '\\')
        return json.loads(json_content)  # 重试解析
```

(5) 派生字段计算与持久化：系统为每个测试用例自动计算派生字段 stepsText 和 expectedText，这两个字段通过在写入时预先计算，避免了在每次查询时进行实时转换（如通过数据库聚合管道），以空间换时间，显著提升了列表查询接口的响应性能。

(6) 异常处理与日志记录：模块集成了完整的错误处理与日志记录机制。对 LLM 网络请求超时、返回非标准 JSON、数据库操作失败等异常情况均

进行了捕获和处理，并向客户端返回友好的错误信息，保证了系统的稳定性。系统的最终实现效果如图 5-25~图 5-29 所示。

图 5-25 测试用例生成-初始状态

Fig.5-25 Test Case Generation - Initial State

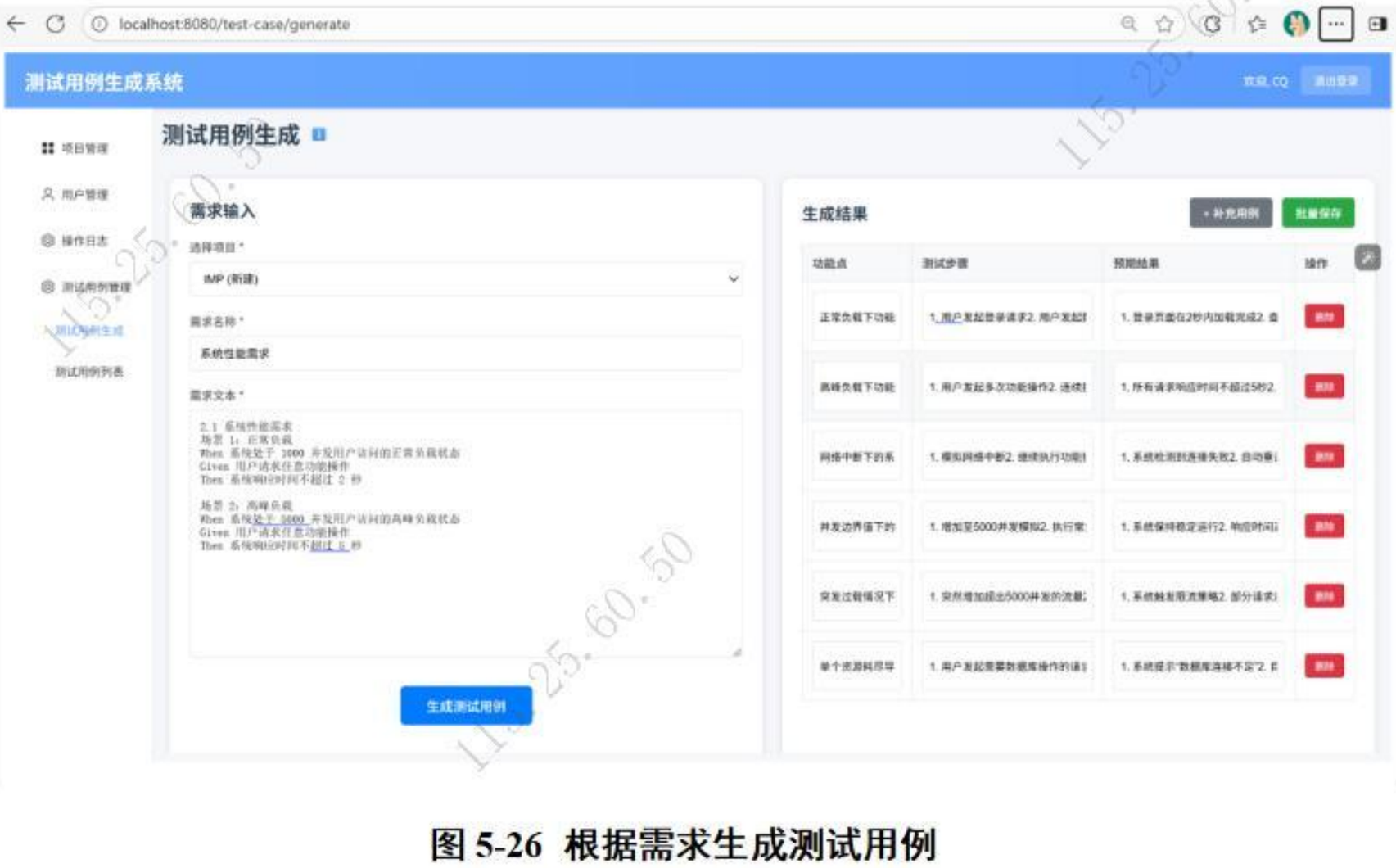


图 5-26 根据需求生成测试用例

Fig.5-26 Test Case Generation Based on Requirements

图 5-27 补充测试用例

Fig.5-27 Supplementary Test Case

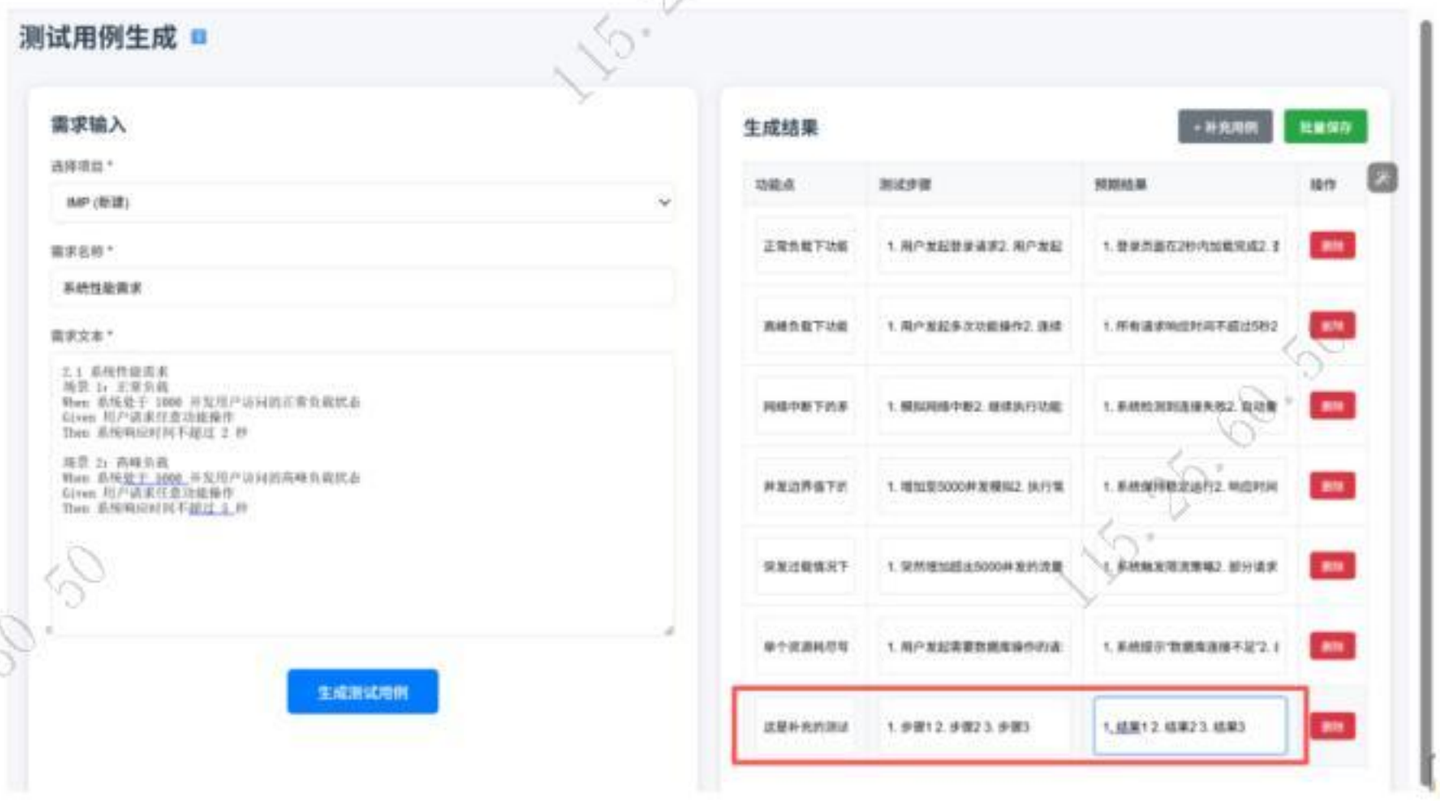


图 5-28 添加用例成功

Fig.5-28 Successful Test Case Addition

图 5-29 新增用例在列表页的展示效果

Fig.5-29 Display Effect of Newly Added Test Cases on the List Page

5.4.4.2 测试用例列表

测试用例列表管理模块为生成的测试用例提供了全面的增删改查功能。该模块通过 GET /api/testcase/list 接口提供高效的数据检索能力，支持基于项目 ID、用例状态的精确过滤，并可对标题、步骤文本及预期结果文本等关键字段进行模糊搜索。查询结果采用分页机制返回，通过 page 与 page_size 参数控制数据量，有效保障了前端列表的渲染性能与大数据集下的查询效率。为优化查询性能，数据库中对常用过滤字段建立了索引。

数据更新通过 PUT /api/testcase/update 接口实现，允许用户对已生成的测试用例内容进行修正或更新其评审状态。删除功能则由 DELETE /api/testcase/delete 接口提供，在执行物理删除操作前，会校验并维护与用户故事之间的关联引用，确保数据的一致性。所有操作均记录操作时间戳，为系统审计提供依据。该模块通过上述设计，实现了测试用例的规范化与高效管理。系统页面如图 5-30 所示。

图 5-30 测试用例列表

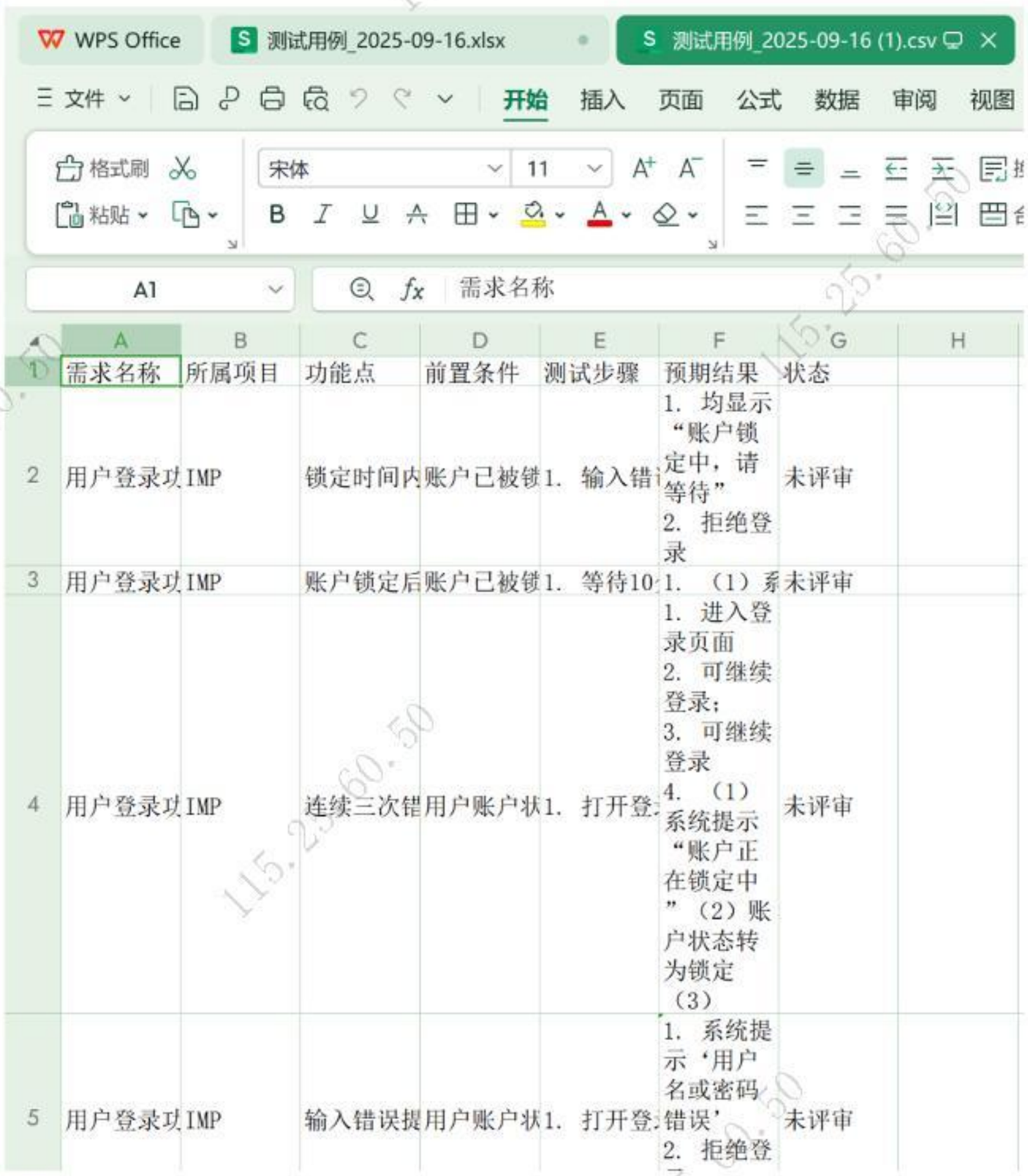
Fig.5-30 Test Case List

5.4.4.3 测试用例导出

测试用例导出模块支持CSV与Excel两种通用格式的客户端导出功能，无需服务器端参与即可高效生成标准化测试文档。该模块从前端状态管理中获取结构化测试用例数据，通过差异化的数据处理流程生成相应格式文件。

CSV 格式导出采用文本序列化方案，遍历用例数组并按 CSV 规范格式化每条记录。系统自动将需求名称、项目名称、功能点、前置条件、测试步骤、预期结果及状态等字段转换为文本行，其中包含特殊字符（如换行符）的文本字段使用双引号包裹并进行转义处理，有效防止格式错乱。最终通过 Blob 对象构造 UTF-8 编码文件内容，并借助动态创建的隐藏<a>标签触发浏览器下载流程，文件名按“测试用例_日期.csv”模式自动生成。

Excel 格式导出基于 xlsx 库实现，在浏览器内存中动态构建工作簿对象。系统将 JavaScript 对象数组映射为工作表所需的二维数组结构，首行生成标题行，后续各行填充具体测试数据。该库自动处理单元格格式，对含换行符的文本字段智能启用"自动换行"属性，保持内容的可视性完整性。最终通过编码生成二进制数据流，利用 HTML5 Blob API 创建临时下载链接，实现高性能的客户端文件导出功能。实现效果见图 5-31 和图 5-32。



	A	B	C	D	E	F	G	H
1	需求名称	所属项目	功能点	前置条件	测试步骤	预期结果	状态	
2	用户登录功	IMP	锁定时间内	账户已被锁	1. 输入错	1. 均显示“账户锁定中，请等待” 2. 拒绝登录	未评审	
3	用户登录功	IMP	账户锁定后	账户已被锁	1. 等待10	1. （1）系	未评审	
4	用户登录功	IMP	连续三次错	用户账户状	1. 打开登	1. 进入登录页面 2. 可继续登录； 3. 可继续登录 4. （1）系统提示“账户正在锁定中”（2）账户状态转为锁定（3）	未评审	
5	用户登录功	IMP	输入错误提	用户账户状	1. 打开登	1. 系统提示‘用户名或密码错误’ 2. 拒绝登	未评审	

图 5-31 测试用例导出结果图 - csv 文件

Fig.5-31 Test Case Export Result Diagram - CSV File

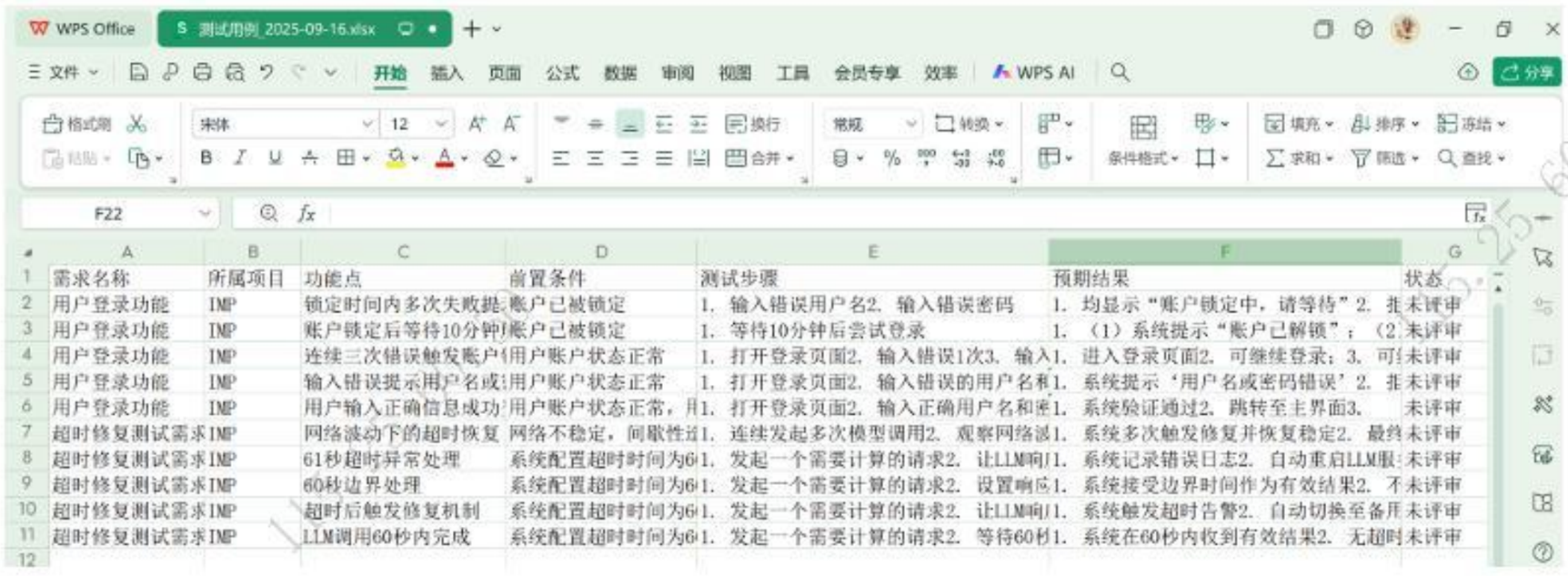


图 5-32 测试用例导出结果图 - xlsx 文件

Fig.5-32 Test Case Export Result Diagram - XLSX File

5.4.5 日志管理

日志管理模块的交互流程如图 5-33 所示。

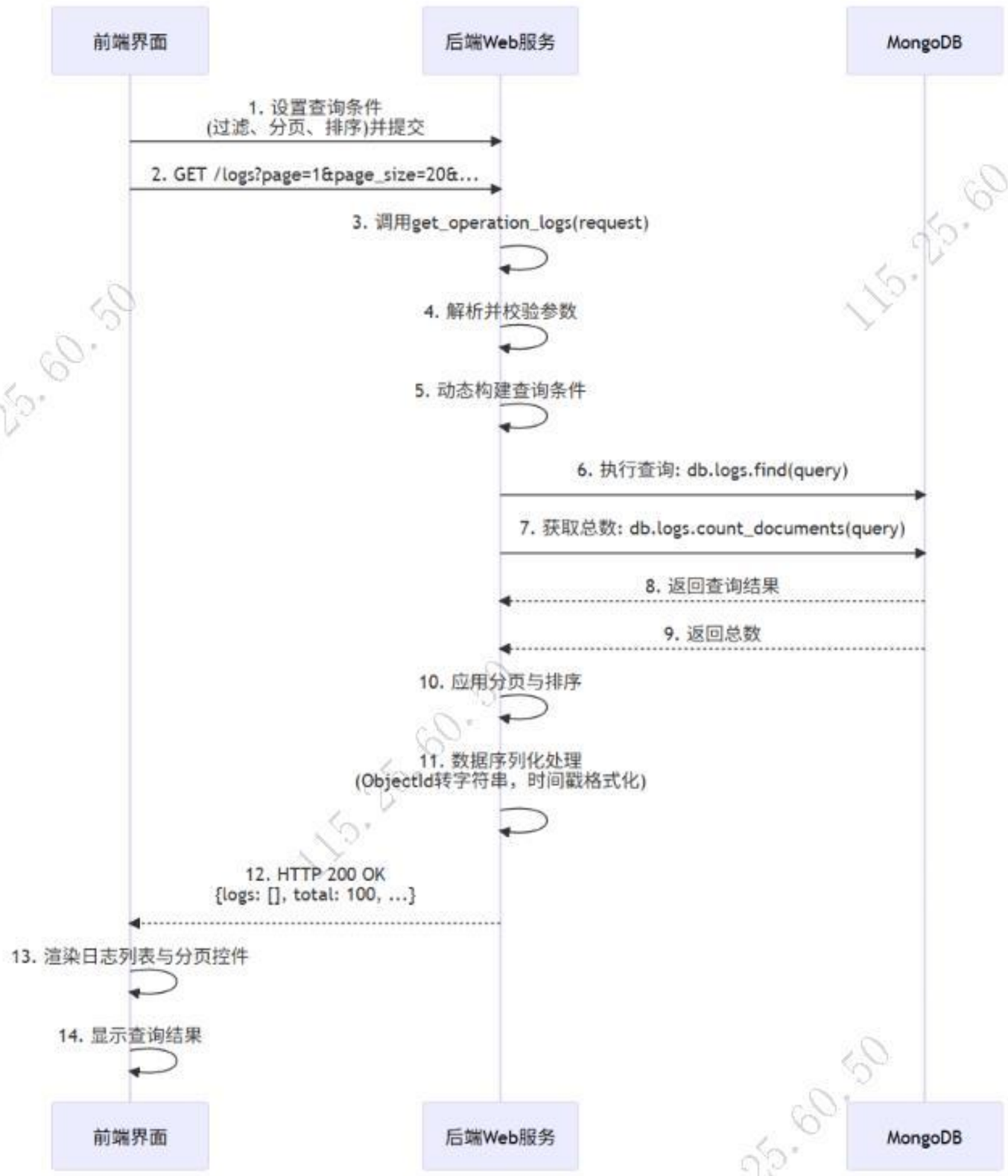


图 5-33 日志功能时序图

Fig.5-33 Sequence Diagram for Log Function

该模块基于 MongoDB 构建，为系统提供完整的操作审计与安全追踪能力。通过专用日志集合持久化记录用户操作行为，并对外提供两个核心 RESTful 接口：一个支持多条件组合的日志检索（包括操作类型、用户标识、资源对象及时间范围等维度），另一个用于获取关键统计指标，如日志总量、近 24 小时及近 7 日的操作日志数量。在实现层面，日志查询功能通过动态构建 MongoDB 查询条件并结合游标机制实现高效分页与结果返回；统计功能则依托数据库的聚合能力完成指标计算。所有操作均包含异常处理机制，以保障服务的稳定性与鲁棒性。系统运行效果见图 5-34 与图 5-35。

图 5-34 日志管理列表

Fig.5-34 Log Management List

图 5-35 日志管理-按照时间段搜索

Fig.5-35 Log Management - Search by Time Period

5.4.6 权限管理

本系统基于角色访问控制模型实现精细化的权限管控，通过前后端协同保障系统安全。权限验证流程如图 5-36 所示。

图 5-36 基于角色访问控制的权限验证时序图

Fig.5-36 RBAC-Based Permission Verification Sequence Diagram

后端采用装饰器模式实现权限与业务逻辑解耦，装饰器内部完成用户身份提取与权限验证。通过解析请求头中的 Bearer Token 获取用户信息，查询 MongoDB 验证用户状态与角色权限。验证通过后将用户身份绑定至请求对象，供后续业务使用；验证失败返回相应的 HTTP 状态码（401 未认证/403 禁止访问）。

前端通过请求拦截器自动附加认证令牌（Authorization: Bearer ${token}），实现无状态认证。响应拦截器统一处理权限异常：针对 403 状态码清除本地存储并跳转登录页，针对 401 状态码提示权限不足，确保用户获得清晰的操作反馈。系统最终实现效果如图 5-37 到图 5-39 所示。

图 5-37 管理员角色权限

Fig.5-37 Administrator Role Permissions

图 5-38 测试人员角色权限

Fig.5-38 Tester Role Permissions

图 5-39 开发人员角色权限

Fig.5-39 Developer Role Permissions

5.5 本章小结

本章完成了测试用例自动化生成系统的设计与实现工作。系统采用前后端分离架构，前端使用 Vue.js 搭建操作界面，后端基于 Django 框架提供 API 服务，数据存储选用 MongoDB，以适应需求与用例这类半结构化数据的灵活存储。

在功能实现上，系统覆盖了用户登录、用户与项目管理、测试用例的生成与编辑、日志记录及权限控制等完整流程。其中，针对大语言模型输出结果不稳定的问题，设计了一套标准化处理流程，包括清洗无关标记、提取有效 JSON、对齐步骤与预期结果等，确保了生成数据的可用性；为提升查询效率，在保存用例时预先计算并存储了步骤和预期结果的文本摘要，避免了每次查询时的重复计算；在权限与安全方面，实现了基于角色的访问控制，确保不同角色用户只能操作其权限范围内的功能。

6 系统测试

系统测试是软件工程生命周期中的关键阶段，其目的在于在真实或接近真实的环境中检验完整的软件配置项能否和系统正确连接，并满足系统设计文档的要求[78]。本章将依据第 5.1 节所定义的需求，对实现的测试用例生成系统进行测试，主要包括功能性测试（6.1 节）与非功能性测试（6.2 节）两部分，确保研究成果不仅在理论上可行，而且能够在实际应用中稳定运行。

6.1 功能性测试

功能性测试旨在验证系统的各项功能是否按照需求规格说明正确实现。其核心是检验软件系统的输入-输出行为是否符合预期[79]。本节将依据功能性需求，针对系统的项目管理、测试用例管理（包括生成、存储、导出等功能）以及用户管理等核心模块设计测试用例。测试将采用黑盒测试方法，如等价类划分和边界值分析，通过执行预先设计的测试用例，来验证系统功能点的正确性、完整性与一致性，并据此评估系统是否达到了预期的设计目标。表 6-1 至表 6-3 为系统各个模块的功能测试用例表。

表 6-1 用户管理模块功能测试用例表

Tab.6-1 Functional Test Case Table for User Management Module

No.	用例标题	前置条件	测试步骤	预期结果	Result
User 01	（正向）验证管理员可成功新增启用状态用户并登录	1.管理员已登录系统 2.数据库中无名为 alice 的账户	1. 进入用户管理页面，点击“新增用户” 2. 输入用户名 alice，密码 Passw0rd!，点击“保存” 3. 使用 alice 和 Passw0rd!登录系统	1.页面提示用户创建成功 2.用户列表显示 alice，状态为启用 3.登录成功，跳转至用户主界面	Pass
User 02	（正向）验证管理员可成功编辑用户信息并保存	1. 登录系统 2. 存在启用状态用户 alice 密码为 Passw0rd!	1.将 alice 用户名修改为 a_new，密码更新为 New6!，点击保存 2.使用新账户登录	1. (1)页面提示“用户信息更新成功”(2)用户列表中该用户姓名为 a_new 2. 登录成功	Pass

表 6-1 续

Tab.6-1 Continued

No.	用例标题	前置条件	测试步骤	预期结果	Result
User 03	（反向）验证新增用户时用户名重复或为空的处理	1. 管理员已登录系统 2. 数据库中已存在用户名为 "alice" 的账户	1.输入用户名为空、密码 Passw0rd!,点击保存 2. 输入用户名 alice 、密码 Passw0rd!，点击保存	1.(1)系统阻止提交，提示“用户名不能为空” (2)“保存”按钮不可点击 2.(1)系统提示“用户名已存在” (2)显示“返回修改”链接，可点击跳回编辑页	Pass
User 04	（异常）验证编辑用户信息时密码为空或含非法字符的处理	1.管理员已登录系统 2.存在启用状态用户 alice，初始密码 Passw0rd!	1.编辑该用户，将密码留空，点击“保存” 2.编辑该用户，输入含非法字符密码 "pass<>"，点击“保存”	1.(1)系统提示“密码不能为空”； (2)保存按钮置灰不可点击； 2.(1)系统提示“密码包含非法字符”； (2) 光标自动聚焦密码框，允许重新输入	Pass
User 05	（状态）验证禁用/启用用户状态切换及登录权限控制	1. 管理员已登录系统 2. 存在启用状态用户 alice，密码 Passw0rd! 3.用户当前未登录	1.禁用用户 alice 2.尝试使用 alice 登录 3.启用该用户 4.再次尝试登录	1. 用户列表状态更新为禁用，操作成功提示 2.(1)登录失败，提示账户已被禁用，请联系管理员； (2) 显示返回按钮，可点击 3. 状态更新为“启用”，操作成功提示 4.登录成功，跳转至主界面	Pass
User 06	（边界）验证用户名和密码输入最大长度及特殊字符支持	1.管理员已登录系统 2.用户名最大长度为 20 字符，密码最大长度为 50 字符	1.输入用户名为 20 个字符，密码为 50 个字符 2.点击“保存” 3.使用该账户尝试登录	1.输入框完整显示内容，无截断 2.提示户创建成功 3.登录成功，系统正确处理长密码	Pass
User 07	（集成）验证模糊搜索与状态筛选联动分页展示功能	1. 数据库中存在：alice（启用）、bob（停用）carol（启用） 2. 数据分 2 页展示	1.在第二页搜索框输入 a，状态筛选选启用，点击查询 2.修改状态筛选为“全部”，点击“查询”	1. 列表显示 alice_active 和 carol_active 2.列表显示全部 3 个用户	Pass

表 6-2 项目管理模块功能测试用例表

Tab.6-2 Functional Test Case Table for User Management Module

No.	用例标题	前置条件	测试步骤	预期结果	Result
Proj 101	（正向）验证管理员可成功创建项目并跳转至测试用例列表	1. 管理员已登录系统 2. 数据库中无 ProjectAlpha 的项目	1. 进入项目管理页面，点击“新建项目” 2.项目名填 PAlpha、负责人张工、状态为进行中、描述为首个测试项目，点击“保存" 3. 在项目列表中点击 ProjectAlpha 的“查看用例”链接	1. 出现创建项目弹窗； 2.项目列表新增 PtAlpha，显示负责人、创建时间、状态“进行中" 3. 跳转至该项目的测试用例列表页，页面项目搜索条件显示 PAlpha	Pass
Proj 102	（正向）验证管理员可成功编辑项目信息并保存	1. 管理员已登录系统 2. 存在项目 "ProjectBeta"，负责人 "李工"，状态“未开始”，描述为空	1.在项目列表中点击 ProjectBeta 的“编辑”按钮 2.修改负责人为 "王工"，状态为“已完成”，描述为 "已结项项目"，点击“保存”	1. 编辑页预填充原数据 2.(1)提示“项目信息更新成功”(2)项目列表中负责人、状态、描述字段同步更新	Pass
Proj 103	（反向）验证创建项目时名称为空或重复的处理	1. 管理员已登录系统 2. 数据库中有项目 ExiProject	1. 项目名称留空，填写其他字段，点击“保存” 2. 输入项目名称 ExiProject，点击“保存”	1.(1)系统提示“项目名称不能为空”(2)“保存”按钮置灰不可点击 2.(1) 系统提示项目名称已存在； (2)显示“返回修改”链接，可点击跳回编辑页	Pass
Proj 104	（异常）验证删除项目时存在/不存在测试用例的不同处理	1. 管理员已登录系统 2. 存在项目 Empty（无任何测试用例） 3. 存在项目 Populated（含至少 1 条测试用例）	1.点击 Empty 的删除按钮，确认删除 2.点击 Populated 的删除按钮	1.(1) 系统提示项目删除成功 (2) 项目列表中 Empty 消失 2.(1)系统提示“该项目下存在测试用例，无法删除” (2)显示“查看用例”按钮，可点击跳转至用例列表	Pass
Proj 105	（边界）验证项目名称和描述输入最大长度及特殊字符支持	1. 管理员已登录系统 2.项目名称最大长度 50 字符，描述最大长度 500 字符	1. 输入名称为 50 字符 2. 描述输入 500 字符（含换行、标点、中文、特殊符号） 3. 点击“保存”	1. 输入框完整显示内容，无截断 2.文本域完整显示，支持换行与特殊字符 3.(1)提示“项目创建成功” (2)列表中名称和描述字段完整显示，无丢失或乱码	Pass

表 6-2 续

Tab.6-2 Continued

No.	用例标题	前置条件	测试步骤	预期结果	Result
Proj 106	（状态）验证项目状态从“未开始”→“进行中”→“已完成”的合法流转	1.管理员已登录系统 2.存在项目 StatusProject，初始状态未开始	1. 编辑项目，将状态改为“进行中”，保存 2. 再次编辑，将状态改为“已完成”，保存	1. 状态更新成功，列表显示“进行中” 2. 状态更新成功，列表显示“已完成”	Pass
Proj 108	（集成）验证项目创建后自动关联空用例列表并支持后续用例归属	1. 管理员已登录系统 2. 无同名项目	1. 创建新项目 NProject，状态进行中 2.点击“查看用例”进入用例列表 3. 在用例管理模块创建一条新用例并选择归属到 NProject	1. 项目创建成功 2.(1)用例列表页显示“暂无测试用例”(2)提供“新建用例”按钮，可点击 3.(1)用例保存成功 (2)返回项目用例列表，显示该用例，项目名称字段正确关联	Pass

表 6-3 测试用例管理模块测试用例表

Tab.6-3 Test Case Table for Test Case Management Module

No.	用例标题	前置条件	测试步骤	预期结果	Result
TC 101	（正向）验证用户可成功选择项目并粘贴需求文本触发用例生成	1.用户已登录 2.有 X 项目 3.大语言模型服务正常	1.进入用例生成页面，选择项目 X，粘贴需求文本 2.点击"生成测试用例"按钮	1. (1) 项目下拉框选中 X；(2)输入框完整显示粘贴内容 2.(1)显示"生成中..."提示；(2)3 秒内返回结构化用例的可编辑表格，字段完整无缺失	Pass
TC 102	（正向）验证用户可对生成的用例表格编辑后批量保存至数据库	1.用户已登录 2.已在用例生成页成功生成 3 条临时用例（未保存） 3.目标项目为 X	1.修改第 1 条用例标题为"邮箱格式校验" 2.勾选全部 3 条用例，点击批量保存 3.点击"查看用例列表"跳转	1.表格内编辑实时生效，无报错；2.(1)提示 3 条用例保存成功(2)批量保存按钮变为"已保存"状态；3.用例列表页显示 3 条新增用例，内容与编辑后一致，归属 X 项目	Pass
TC 103	（正向）验证用户可按项目筛选用例并执行单条编辑操作	1.用户登录系统 2.A 下存在用例 TC1001：标题"登录校验"，状态"未执行"	1.进入用例列表，筛选 A 项目 2.点击 TC1001 的编辑按钮 3.修改标题为"邮箱登录校验"，点击"保存"	1.列表仅显示属于 A 的用例，包含 TC1001；2.编辑页加载 TC1001 原始数据；3.(1)提示"用例更新成功"(2)列表中 TC1001 标题更新为"邮箱登录校验"，状态仍为"未执行"	Pass

表 6-3 续

Tab.6-3 Continued

No.	用例标题	前置条件	测试步骤	预期结果	Result
TC104	（正向）验证用户可按状态/关键词组合检索并执行单条删除操作	1.用户已登录系统 2.项目 A 下存在： •TC102（状态"未执行"，标题含支付） •TC103（状态"已通过"，标题含支付）	1.选择状态未执行，关键词输入"支付"，点击"查询" 2.点击 TC102 的删除按钮，确认删除 3.清空筛选条件，刷新列表	1.列表仅显示 TC1002 2.(1) 弹出确认框，点击"确认"后提示"删除成功" (2)TC102 从当前列表消失 3.列表显示 TC103，TC102 不再出现，总数-1	Pass
TC105	（正向）验证用户可勾选多条用例执行批量导出为 CSV/Excel 文件	1.用户已登录系统 2.项目 ProjectA"下存在至少 2 条用例：TC1003、TC1004	1.勾选 TC1003 和 TC1004 2.点击"导出"，选择 CSV 格式 3.重复操作，选择 Excel 格式	1.选中行高亮显示 2.(1) 浏览器自动下载 .csv 文件；(2)文件内容包含 2 条用例所有字段，编码 UTF-8 无乱码； 3.(1) 浏览器自动下载 .xlsx 文件；(2)文件可被 Excel 打开，格式对齐，数据完整无缺失	Pass
TC106	（反向）验证生成用例时未选择项目或需求为空的处理	1.用户已登录系统 2.至少存在 1 个项目	1.不选择项目，直接点击"生成" 2.选择项目但需求输入框为空，点击"生成"	1.(1)系统提示"请选择目标项目"；（2)生成按钮置灰不可点击 2.(1)系统提示"需求文本不能为空"；（2)光标自动聚焦输入框，允许粘贴内容	Pass
TC107	（异常）验证大语言模型服务超时或返回空结果时的容错处理	1. 用户已登录系统 2. 项目"X"已选中 3. 模拟大语言模型服务返回超时或空响应	1.粘贴需求文本，点击"生成" 2.等待 15 秒无响应后，点击"取消" 3.重新点击"生成"，模拟返回空结果	1.显示"生成中..."加载动画；2.(1)加载动画消失，提示"生成超时，请重试"(2)显示"重试"按钮，可点击 3.(1)提示"未能生成有效用例，请检查需求或手动补充"(2)显示空表格，但"手动添加行"按钮可用	Pass
TC108	（边界）验证粘贴超长需求文本（≥5000 字符）的处理与生成稳定性	1. 用户已登录系统 2. 项目已选中 3. 大语言模型服务支持长文本输入	1.粘贴 5000 字符需求文本含中英文、换行、标点） 2.点击生成测试用例 3.生成后编辑其中一条用例标题为最大长度 255 字符，点击保存	1.输入框完整显示内容，无截断或卡顿 2.正常返回结构完整用例，无字段丢失 3.(1)保存成功提示（2)用例列表中该用例标题完整显示，无截断	Pass

6.2 非功能性测试

非功能性测试关注的是系统在运行过程中所表现出来的、除功能以外的质量属性，这些属性同样是衡量系统成功与否的关键指标。如第 5.1.2 节所分析，本系统需具备良好的性能、可用性、安全性等非功能性特性。本节将重点对系统的兼容性和安全性进行测试与评估。以确保系统在实际部署后能够提供友好、高效且安全的服务。

兼容性测试是软件测试的关键类型之一，旨在验证软件系统在不同硬件、软件平台或环境（如不同浏览器）下能否正常工作。为验证系统跨平台兼容性，本文针对 Chrome、Edge、Firefox 及 Safari 等主流浏览器进行了测试。由表 6-4 可知，系统功能与用户界面在各测试环境中均表现出良好的兼容性。

表 6-4 兼容性测试结果表

Tab.6-4 Compatibility Test Result Table

功能	Chrome	Edge	Firefox	Safari
用户管理	√	√	√	√
项目管理	√	√	√	√
日志管理	√	√	√	√
测试用例生成	√	√	√	√
测试用例列表	√	√	√	√

为确保系统数据的机密性、完整性与可用性，本文依据开放网络应用安全项目（Open Web Application Security Project, OWASP）应用安全验证标准（Application Security Verification Standard, ASVS）与 OWASP Top 10 最新清单，对系统进行了安全测试，覆盖身份认证、输入认证、数据安全。

表 6-5 为安全测试结果，测试表明，系统在这三个关键安全项上表现符合预期。图 6-1 和图 6-2 为系统实际展示。

表 6-5 安全性测试用例表

Tab.6-5 Security Test Case Table

测试类别	测试标题	测试步骤	预期结果	实际结果
身份认证	未授权访问控制失效	在用户未登录状态下，于浏览器中直接输入需授权的项目 URL（如 /projects/123）尝试访问。	拒绝访问，返回 403	✓
输入验证	存储型跨站脚本	在名称输入框中输入<script>alert(1)</script>并保存。随后浏览展示页面，检查警报是否弹出。	输入被转义，显示为文本	✓
数据安全	密码加密存储	检查数据库用户表密码字段	通过哈希算法将密码加密存储	✓

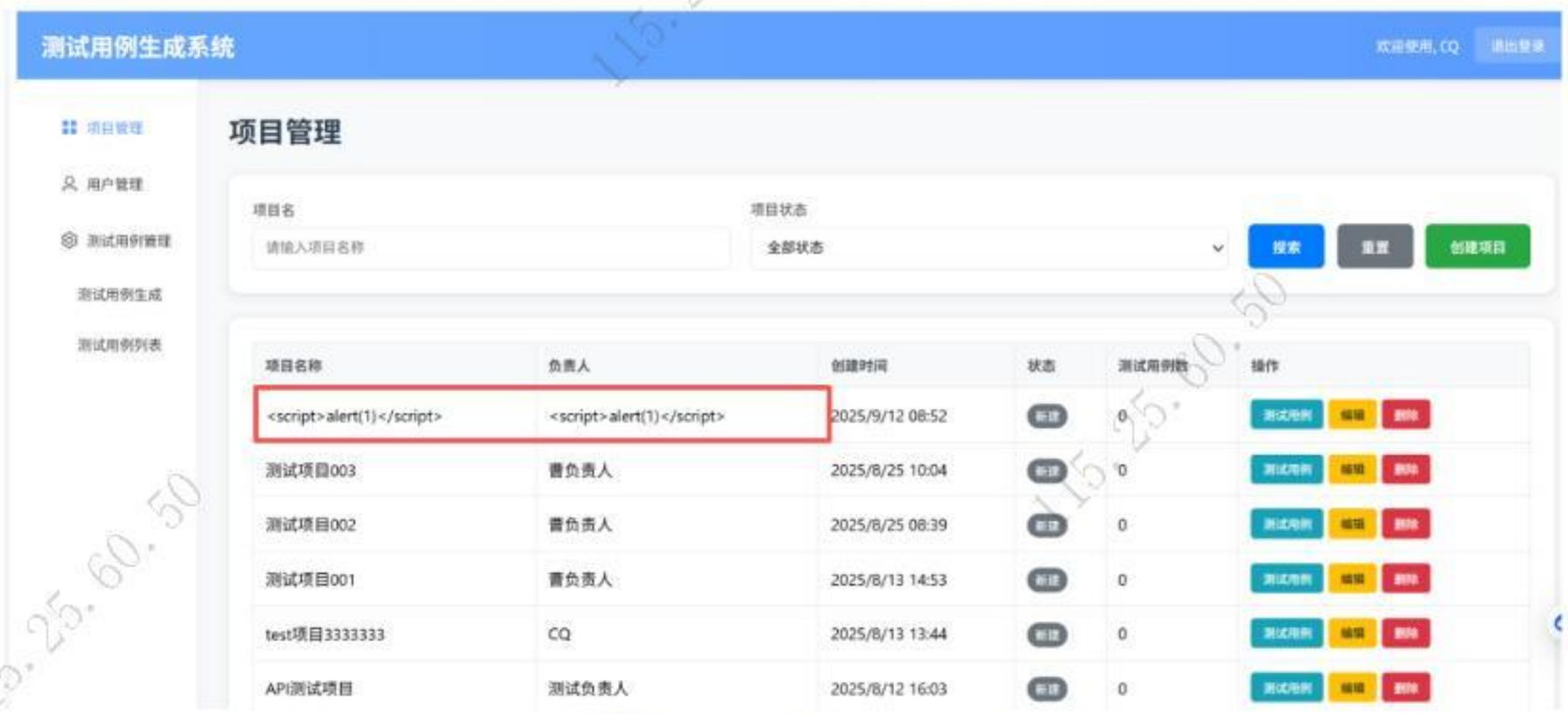


图 6-1 跨站脚本被成功转义结果图

Fig.6-1 Result Diagram of Successful Cross-Site Scripting Escaping

图 6-2 数据库用户密码的哈希加密存储示例图

Fig.6-2 Example Diagram of Hashed Encryption Storage for Database User Passwords

6.3 本章小结

本章围绕系统的功能性与非功能性测试展开，全面验证了系统的质量。功能性测试针对项目管理、测试用例管理及用户管理等核心模块进行了系统性验证。通过采用黑盒测试方法（包括等价类划分与边界值分析），全面检验了系统在输入输出逻辑、操作流程及数据一致性方面的表现。测试结果表明，系统各模块均能按照需求规格说明正确实现预期功能，具备良好的功能完整性与稳定性。非功能性测试进一步从兼容性与安全性角度对系统进行了质量评估。兼容性测试依据 ISO/IEC/IEEE 29119 系列标准开展，结果显示系统在 Chrome、Edge、Firefox 及 Safari 等主流浏览器中均能稳定运行，界面与交互一致性良好。安全性测试则基于 OWASP ASVS 与 OWASP Top 10 标准进行，验证了系统在身份认证、输入校验与数据保护等关键环节的安全性，结果均符合设计预期。综上，系统在功能实现、兼容适配与安全防护等方面均达到预期目标。

7 结论与展望

本章旨在对全文的研究工作与成果进行系统性总结，并在此基础上展望未来的研究方向。

7.1 结论

本研究围绕“需求驱动的测试用例自动生成”这一核心问题，在已有研究基础上，针对大语言模型在软件测试领域的特定应用场景进行了系统性研究。主要工作与创新成果如下：

(1) 针对大语言模型在测试用例生成中输出结构混乱、可执行性差的问题，本研究分析了测试任务的特殊约束（如步骤-预期严格对齐、字段完整性、BDD 规范等），设计了一种融合思维链、少样本示例、模板化输出与自适应机制的提示策略。在包含简单功能、非功能性及复杂业务需求场景上的实验结果表明，该方法在正确性、可执行性、完备性与创新性方面均显著优于单一提示策略，尤其在可执行性和正确性维度表现突出，为测试用例的结构化生成提供了有效方法。

(2) 针对通用大模型缺乏领域知识、对业务规则与边界条件理解不足的问题，通过构建包含企业真实项目、教材案例与开源数据的高质量“需求-测试用例”对齐数据集，并采用标准化的 Prompt-Completion 格式与多阶段清洗流程（去重、结构校验、字段对齐、语言规范化），设计了基于 QLoRA 的微调方案，在 Qwen3-4B 模型上实现了领域知识的高效注入。自动化评估与人工盲评结果均表明，微调后模型在语义贴合度、用例结构化质量与工程可执行性方面均有显著提升。

(3) 针对当前缺乏面向测试工程师的专用自动化生成工具的问题，通过集成提示工程与微调模型，设计并实现了一个前后端分离的测试用例自动生成系统，支持项目和用户管理、需求录入、用例生成与编辑、导出等完整流程。在功能性测试与非功能性测试上的实验结果验证了：系统功能完整、运行稳定、安全合规，能够切实满足测试人员在实际项目中快速生成高质量、可直接执行测试用例的需求。

整体而言，本文证明了大语言模型在需求驱动的测试用例自动生成领域的潜力，为软件测试自动化提供了一种新的解决思路。

7.2 展望

尽管取得了一定成果，仍存在一些有待深入研究的问题与发展方向：

(1) 需求理解的鲁棒性：复杂需求文档中常包含模糊、冗余甚至矛盾的信息，未来可结合需求解析、语义一致性检测与上下文对齐方法，进一步提升模型对复杂需求的理解能力。

(2) 跨领域与多场景适应性：实验主要集中于典型功能需求，未来需扩展至金融、法律、医疗等更垂直、专业性更强的测试场景，提升模型的跨领域泛化能力。

(3) 提示工程的自动化与自适应优化：当前提示策略的设计与调优依赖人工经验。未来可构建提示词自动进化框架，基于测试执行反馈驱动 LLM 自我迭代提示模板，实现动态优化与上下文感知的提示生成。

(4) 评价体系的完善：现有提示工程评估以人工指标为主，后续可建立更系统的混合评估框架，引入需求覆盖率、可执行率与业务相关性等多维度度量，提升评估的客观性与可复现性。

综上，需求驱动的测试用例自动生成仍处于探索阶段，提示工程与微调的结合展现了良好潜力。未来若能在需求解析、生成约束、跨场景适配与系统化评估等方面持续突破，将有望推动测试用例生成走向更加智能化、自动化与实用化。

参考文献

[1] 尚冬娟, 郝克刚, 葛玮,等. 软件测试中的测试用例及复用研究[J]. 计算机技术与发展, 2006, (1): 69-72.

[2] 董齐兴. 基于动态符号执行的测试用例生成技术研究[D]. 合肥: 中国科学技术大学, 2014.

[3] 刘扬, 李亚芬, 王普. 模型驱动的测试用例自动生成框架[J]. 计算机工程, 2011, 37(1): 39-40.

[4] 周瑞杰, 江国华. 基于遗传算法的面向对象类测试用例生成研究[C]//2010 通信理论与技术新发展——第十五届全国青年通信学术会议论文集 (下册). 2010.

[5] 邱晓康, 李宣东. 一个面向路径的软件测试辅助工具[J]. 电子学报, 2004, 32(12A): 231-234.

[6] 丁蕊, 董红斌, 张岩, 冯宪彬. 基于关键点路径的快速测试用例自动生成方法[J]. 软件学报, 2016, 27(4): 814-827.

[7] Kennedy J, Eberhart R. Particle swarm optimization[C]//Proceedings of the IEEE Conference on Neural Networks. Perth, Australia, 1995, 4: 1942-1948.

[8] 包晓安, 杨亚娟, 张娜, 林青霞, 俞成海. 基于自适应粒子群优化的组合测试用例生成方法[J]. 计算机科学, 2017, 44(6): 177-181.

[9] 王秀玄. 基于 UML 状态图的列控中心软件测试路径生成方法[J]. 铁路计算机应用, 2016, 25(8): 9-13.

[10] 苗俊磊. 基于 UML 状态图的软件测试研究[D]. 西安: 西安电子科技大学, 2014.

[11] 赵卫东, 李有俊, 张丽. 一种基于高阶 Markov 使用模型的测试用例自动生成方法[J]. 现代电子技术, 2019, 42(06): 26-29.

[12] 栾家伟, 吴陈. 基于 UML 状态图测试用例生成的策略研究[J]. 计算机与数字工程, 2020, 48(2): 409-411, 432.

[13] Manès V J M, Han H S, Han C, et al. The art, science, and engineering of fuzzing: A survey[J]. IEEE Transactions on Software Engineering, 2019, 47(11): 2312-2331.

[14] Bu L, Liang Y, Xie Z, et al. Machine learning steered symbolic execution framework for complex software code[J]. Formal Aspects of Computing, 2021, 33(3): 301-323.

[15] Mathur A, Pradhan S, Soni P, et al. Automated test case generation using t5 and GPT-3[C]//2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS). IEEE, 2023, 1: 1986-1992.

[16] Kang S, Yoon J, Yoo S. Large language models are Few-shot testers: Exploring LLM-based general bug reproduction[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 2312-2323.

[17] Rahman T, Zhu Y. Automated user story generation with test case specification using large language model[J]. arXiv preprint arXiv:2404.01558, 2024.

[18] 杨浠, 熊盼, 郑旭飞, 等. AIGC 辅助软件单元测试的研究[J]. 人工智能科学与工程, 2024, (04): 31-41.

[19] 李博诺, 王博, 陈明达, 等. 基于大语言模型的C程序单元测试方法[J]. 空间控制技术与应用, 2025, 51(01): 115-122.

[20] 曹鹏, 温广琪, 杨金柱, 等. 面向测试用例生成的大模型高效微调方法[J]. 计算机应用, 2025, 45(03): 725-731.

[21] Johnsson N. An in-depth study on the utilization of large language models for test case generation[D]. Umeå: Umeå University, 2024.

[22] Kalla D, Smith N, Samaah F, et al. Study and analysis of chat GPT and its impact on different fields of study[J]. International Journal of Innovative Science and Research Technology, 2023, 8(3). 827-833.

[23] Lemeš S. Prompt Engineering[J]. Artificial Intelligence in Industry, 2024, 4: 159-170.

[24] Liu P, Yuan W, Fu J, et al. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing[J]. ACM Computing Surveys, 2023, 55(9): 1-35.

[25] Sahoo P, Singh A K, Saha S, et al. A systematic survey of prompt engineering in large language models: Techniques and applications[J]. arXiv preprint arXiv: 2402.07927, 2024.

[26] 黄峻, 林飞, 杨静, 等. 生成式 AI 的大模型提示工程:方法, 现状与展望[J]. 智能科学与技术学报, 2024, 6(2): 115-133.

[27] OpenAI. Best practices for prompt engineering with the OpenAI API[EB/OL]. (2023-03)[2025-12-08]. https://platform.openai.com/docs/guides/prompt-engineering.

[28] Feng G, Zhang B, Gu Y, et al. Towards revealing the mystery behind chain of thought: a theoretical perspective[C]//Advances in Neural Information Processing Systems, 2023, 36: 70757-70798.

[29] Yao S, Yu D, Zhao J, et al. Tree of thoughts: Deliberate problem solving with large language models[C]//Advances in Neural Information Processing Systems, 2023, 36: 11809-11822.

[30] White J, Hays S, Fu Q, et al. ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design[M]//Generative AI for Effective Software Development. Cham: Springer Nature Switzerland, 2024: 71-108.

[31] Kojima T, Gu S S, Reid M, et al. Large language models are Zero-shot reasoners[C]//Advances in Neural Information Processing Systems, 2022, 35: 22199-22213.

[32] Brown T B, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, et al. Language models are Few-shot learners[C]//Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.

[33] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, volume 1 (long and short papers). 2019: 4171-4186.

[34] Ruder S, Peters M E, Swayamdipta S, et al. Transfer learning in natural language processing[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Tutorials. 2019: 15-18.

[35] Howard J, Ruder S. Universal language model Fine-tuning for TextClassification[C]//Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2018:

328-339.

[36] Sun C, Qiu X, Xu Y, et al. How to Fine-Tune BERT for Text Classification?[C]//China National Conference on Chinese Computational Linguistics. 2019: 194-206.

[37] Lee J, Yoon W, Kim S, et al. BioBERT: a pre-trained biomedical language representation model for biomedical text mining[J]. Bioinformatics, 2020, 36(4): 1234-1240.

[38] Hendrycks D, Burns C, Basart S, et al. Measuring massive multitas language understanding[J]. arXiv preprint arXiv:2009.03300, 2020.

[39] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[C]//Advances in Neural Information Processing Systems, 2022, 35: 27730-27744.

[40] Dettmers T, Pagnoni A, Holtzman A, et al. QLoRA: Efficient finetuning of quantized LLMs[C]//Advances in Neural Information Processing Systems, 2023, 36: 10088-10115.

[41] Lester B, Al-Rfou R, Constant N. The power of scale: Parameter-efficient adaptation for pretrained language models[C]//Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP). 2021: 3045-3059.

[42] Li X L, Liang P. Prefix tuning: Optimizing continuous prompts for Generation[C]//Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). 2021: 4582-4597.

[43] Houlsby N, Giurgiu A, Jastrzebski S, et al. Parameter-efficient transfer learning for NLP[C]//International Conference on Machine Learning. PMLR, 2019: 2790-2799.

[44] Hu E J, Wallis P, Allen-Zhu Z, et al. LoRA: Low-Rank adaptation of large language models[C]//International Conference on Learning Representations. 2022.

[45] Liu H, Tam D, Muqeeth M, et al. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning[C]//Advances in Neural Information Processing Systems, 2022, 35: 1950-1965.

[46] Maia Polo F, Xu R, Weber L, et al. Efficient multi-prompt evaluation of LLMs[C]//Advances in Neural Information Processing Systems, 2024, 37: 22483-22512.

[47] Zhu K, Wang J, Zhou J, et al. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts[C]//Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis. 2023: 57-68.

[48] KardkovĂĄcs Z T, Djennane L, Field A, et al. BTC-SAM: Leveraging LLMs for generation of bias test cases for sentiment analysis models[J]. arXiv preprint arXiv:2509.24101, 2025.

[49] Tam Z R, Wu C K, Tsai Y L, et al. Let me speak freely? A study on the impact of format restrictions on large language model performance[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. 2024: 1218-1236.

[50] Zhang Y, Diddee H, Holm S, et al. Noveltybench: Evaluating creativity and diversity in language models[J]. arXiv preprint arXiv:2504.05228, 2025.

[51] Korraprolu B R, Pinninti P, Reddy Y R. Test case generation for requirements in natural language-An LLM comparison study[C]//Proceedings of the 18th Innovations in Software Engineering Conference. 2025: 1-5.

[52] Liu N F, Lin K, Hewitt J, et al. Lost in the middle: How language models use long contexts[J]. arXiv preprint arXiv: 2307. 03172, 2023.

[53] Norheim J J, Rebentisch E, Xiao D, et al. Challenges in applying large language models to requirements engineering tasks[J]. Design Science, 2024, 10: e16.

[54] Geng S, Cooper H, Moskal M, et al. Generating structured outputs from language models: Benchmark and studies[J]. arXiv e-prints, 2025: arXiv: 2501. 10868.

[55] Park K, Wang J, Berg-Kirkpatrick T, et al. Grammar-aligned decoding[C]// Advances in Neural Information Processing Systems, 2024, 37: 24547-24568.

[56] Shojaee P, Mirzadeh I, Alizadeh K, et al. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity[J]. arXiv preprint arXiv:2506.06941, 2025.

[57] Hsiao V, Fine-Morris M, Roberts M, et al. A critical assessment of LLMs for solving multi-step problems: Preliminary results[C]//AAAI 2025 Workshop LM4Plan.

[58] Ouédraogo W C, Kaboré K, Li Y, et al. Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation[J]. arXiv preprint arXiv: 2407. 00225, 2024.

[59] Holtzman A, Buys J, Du L, et al. The curious case of neural text degeneration[C]. 8th International Conference on Learning Representations (ICLR 2020), Addis Ababa, Ethiopia, April 26-30, 2020.

[60] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]. Advances in Neural Information Processing Systems, 2022, 35: 24824-24837.

[61] Fei Y, Hou Y, Chen Z, et al. Mitigating label biases for in-context learning[J]. arXiv preprint arXiv:2305.19148, 2023.

[62] Turpin M, Michael J, Perez E, et al. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting[C]. Advances in Neural Information Processing Systems, 2023, 36: 74952-74965.

[63] Wu T, Jiang E, Donsbach A, et al. Promptchainer: Chaining large language model prompts through visual programming[C]//CHI Conference on Human Factors in Computing Systems Extended Abstracts. 2022: 1-10.

[64] Willard B T, Louf R. Efficient guided generation for large language models[J]. arXiv preprint arXiv:2307. 09702, 2023.

[65] Beurer-Kellner L, Fischer M, Vechev M. Prompting is programming:A query language for large language models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(PLDI): 1946-1969.

[66] Likert R. A technique for the measurement of attitudes[J]. Archives of Psychology, 1932, 22(140): 1-55.

[67] OpenAI. Preparing your dataset for fine-tuning[EB/OL]. 2023[2025-09-19]. https://platform.openai.com/docs/guides/fine-tuning/preparing-your

-dataset

[68] Taori R, Gulrajani I, Zhang T, et al. Alpaca: a strong, replicable instruction-following model[EB/OL]. (2023-03-13)[2025-10-26]. https://crfm.stanford.edu/2023/03/13/alpaca.html.

[69] Farooq M S, Omer U, Ramzan A, et al. Behavior driven development: A systematic literature review[J]. IEEE Access, 2023, 11: 88008-88024.

[70] Xu L, Li A, Zhu L, et al. Superclue: A comprehensive chinese large language model benchmark[J]. arXiv preprint arXiv:2307. 15020, 2023.

[71] Wang H, Chen B K, Siquan L, et al. Prefix tuning+: Modernizing prefix tuning by decoupling the prefix from attention[C]//Second Workshop on Test-Time Adaptation: Putting Updates to the Test! at ICML 2025. 2025.

[72] He P. Parameter efficient instruction tuning: An empirical study[J]. arXiv preprint arXiv: 2411. 16775, 2024.

[73] Razuvayevskaya O, Wu B, Leite J A, et al. Comparison between parameter-efficient techniques and full fine-tuning: A case study on multilingual news article classification[J]. PLoS One, 2024, 19(5): e0301738.

[74] Liu S Y, Wang C Y, Yin H, et al. Dora: Weight-decomposed low-rank adaptation[C]//Forty-first International Conference on Machine Learning. 2024.

[75] Meng F, Wang Z, Zhang M. Pissa: Principal singular values and singular vectors adaptation of large language models[C]//Advances in Neural Information Processing Systems, 2024, 37: 121038-121072.

[76] Zhang T, Kishore V, Wu F, Weinberger K Q, Artzi Y, et al. BERTScore: Evaluating text generation with BERT[C]//2020 8th International Conference on Learning Representations (ICLR 2020). OpenReview.net, 2020.

[77] 张海藩. 软件工程导论[M]. 6 版. 北京: 清华大学出版社, 2013: 56-57.

[78] 陶幸辉, 宋志刚. 软件系统测试类型及测试用例设计[J]. 科技经济市场, 2011,(06): 3-5.

[79] Ammann P, Offutt J. Introduction to software testing[M]. Cambridge: Cambridge University Press, 2017.

附录 A 提示词

角色设定的作用是确定身份和任务边界，减少幻觉。详细提示词见表 A-1。

表 A-1 角色设定提示词

Tab.A-1 Role Instruction Prompt Strategy

方法	完整提示语
角色设定	你是一名资深软件测试架构师。请根据以下需求文档生成结构化测试用例，仅输出 Markdown 表格，不要任何分析、说明或额外文字。

需求输入模块提供待处理的自然语言需求，作为模型输入。详细提示词见表 A-2。

表 A-2 需求输入提示词

Tab.A-2 Requirement Input Prompt Strategy

方法	完整提示语
需求输入	需求文档：{需求文本}

自适应策略模块作用是根据需求复杂度动态调整覆盖范围，确保不同场景下的覆盖合理性。设计目标在于实现覆盖范围与生成效率的自适应平衡。详细提示词见表 A-3。

表 A-3 自适应策略提示词

Tab.A-3 Adaptive Prompt Strategy

方法	完整提示语
Adaptive	- 若功能点 ≤2 → 简单需求（5-7 个核心用例） - 若功能点 >2 → 复杂需求（模块化 + 集成 + 并发/状态场景）

思维链模块的作用是引导模型逐步推理，确保生成过程覆盖全面、逻辑清晰。设计目标在于提升逻辑覆盖率与生成深度。详细提示词见表 A-4。

表 A-4 思维链提示词

Tab.A-4 CoT Prompt Strategy

方法	完整提示语
CoT	请在内部执行以下处理流程： 1. 需求分析 - 拆解核心功能点（功能点 = 独立用户操作或业务能力） - 识别输入/输出、业务规则、状态流转、数据依赖 - 若功能点 ≤2 → 简单需求；>2 → 复杂需求 2. 测试策略设计 基于功能点选择： - 等价类划分：覆盖有效类与无效类输入 - 边界值分析：覆盖长度临界/数值极限/时间临界（比如锁定/解锁、活动开始/结束的瞬时毫秒级操作）等边界值 - 状态转换法：设计合法迁移与非法跳转，包含完整状态闭环（触发状态→状态存续中操作→状态过期后操作） - 错误推测法：覆盖空值、超时、重复提交等风险点，每个错误类型单独设计场景，验证异常发生后的系统状态、用户提示信息以及可进行的后续操作（如重试、修改、返回等） - 正交实验法：当输入参数 ≥3 且组合复杂时使用 - 并发与竞争场景：如“最后 1 库存，多用户同时下单” - 时间敏感场景：覆盖对称场景，包括 “活动开始前停留至开始后操作"活动结束前停留至结束后操作”，以及网络延迟导致请求到达时超过有效期、前端显示未及时同步后端时效状态等瞬时状态转换 - 性能需求：是连续性指标，不适用于传统的边界值测试（如 n-1/n+1），需关注系统在标称负载下的稳定性等 覆盖：正向、反向、异常、边界、状态、并发、集成 7 类场景。 3. 用例生成 - 用例标题标注类型：（正向）/ （反向）/（异常）/ （边界）/ （状态）/ （并发）/ （集成） - 场景隔离原则：严禁将业务流程中不同阶段、不同触发原因、不同处理逻辑的异常场景合并到同一条用例中。每个独立的异常原因（如：库存不足、支付失败、地址无效、商品下架等）都必须设计为单独的测试用例，以确保测试意图明确、步骤清晰、结果可追溯。 - 使用具体值（如 "user@test.com"），禁止使用 <占位符> - 测试步骤需可执行，多步用 分隔 - 状态类场景必须至少包含三类：触发异常状态（如锁定）、状态到期恢复（如解锁）、状态瞬时转换（用户在有效状态期间进入页面，停留至状态过期后操作），需验证前端状态同步、后端时效校验、用户提示、后续操作入口及关联数据影响 - 结果完整性：对于异常、边界、状态类场景，预期结果必须严格对应需求文档，不仅包含提示信息，还必须明确描述系统提供的后续操作入口（如：‘显示’重试’按钮’、‘提供’返回修改’链接’）及其可用性（如：‘按钮可点击’、‘链接可正常跳转’）

少样本示例模块则是提供 1-2 个高精度参考用例，保证输出风格一致性与可控性。详细提示词见表 A-5。

表 A-5 少样本示例提示词

Tab.A-5 Few-shot Prompt Strategy

方法	完整提示语
Few-shot	测试用例参考示例： | 用例编号 | 用例标题 | 前置条件 | 测试步骤 | 预期结果 | | :--- | :--- | :--- | :--- | :--- | | **TC101** | 验证用户可正常登录（含普通/特殊符号账户）| 1. 系统运行正常，处于登录页面 2. 数据库中存在以下有效账户： - 普通用户名：U1，密码：P1（状态正常） - 含特殊符号用户名：U2（如 `user.name+dev@site`），密码：P2（状态正常）| 1. 在用户名输入框输入：U1 2. 在密码输入框输入：P1 3. 点击"登录"按钮 4. 在主界面点击"退出登录"按钮 5. 在用户名输入框输入：U2（含特殊符号） 6. 在密码输入框输入：P2 7. 点击"登录"按钮 | 1. 用户名 U1 输入成功 2. 密码 P1 输入成功 3. 登录成功，页面跳转至主界面，显示当前登录用户名为 U1 4. 退出成功，返回登录页面 5. 用户名 U2 输入成功（特殊字符被接受） 6. 密码 P2 输入成功 7. 登录成功，页面跳转至主界面，正确显示当前登录用户名 U2 | | **TC102** | 验证登录时输入错误场景 | 1. 系统处于登录页面 2. 存在有效账户：用户名 A，密码 P 3. 准备以下测试数据： - 不存在的用户名 - 错误密码（含大小写不符） - 空用户名和空密码 | 1. 输入不存在的用户名和任意密码 2. 点击"登录"按钮 3. 输入正确用户名 A 但错误密码（大小写不符） 4. 点击"登录"按钮 5. 清空用户名和密码输入框 6. 点击"登录"按钮 | 1. 用户名和密码输入完成 2. 登录失败，停留在登录页面，提示"用户名或密码错误" 3. 用户名 A 和错误密码输入完成 4. 登录失败，停留在登录页面，提示"用户名或密码错误" 5. 用户名和密码输入框为空 6. 登录失败，停留在登录页面；用户名/密码输入框显示"必填"提示，系统提示"请输入用户名和密码" |

模板化输出模块的设计目标在于确保输出结构化，详细提示词见表 A-6。

表 A-6 模板化输出提示词

Tab.A-6 Template Prompt Strategy

方法	完整提示语
Template	输出策略： - 简单需求：需覆盖所有功能点及隐含场景，生成 5-7 个核心用例，覆盖“正常流程 + 细分异常 + 完整边界” - 复杂需求：按模块组织，每模块 3-4 条 + 至少 2 条集成（覆盖跨模块交互场景，如“购物车选品后→结算页数据同步”）+ 至少 2 条并发/状态，覆盖 7 类场景 输出要求： - 使用以下格式的 Markdown 表格 - 所有字段必须完整填写 - 禁止输出表格以外的任何内容 - 每个用例为“主场景 + 子场景”结构，子场景需覆盖该主场景下的所有关键等价类 / 流转路径； - 前置条件包含共同前提与差异数据（如多账户信息）； - 测试步骤需分步骤执行所有子场景的操作，逻辑清晰； - 步骤-预期一一对应：expected.length === step.length，如果测试步骤只有一步，但是预期结果有多个，请用 1.（1）（2）标注，如果测试步骤有 2 步：1. 2.，而 1 步骤有多个预期结果，那预期结果应该写成 1.（1）（2），2. - 预期结果对应每步，必须明确包含：提示信息的具体内容、页面状态变化、按钮或链接的可用性（如‘重试’按钮是否显示并可点击），以及用户可选的后续操作路径（如‘返回修改’） 模板如下： | 用例编号 | 用例标题 | 前置条件 | 测试步骤 | 预期结果 | |---|---|---|---|---|

附录 B 需求文档

以下需求文档部分采用 When（当...时）– Given（假设...前提）– Then（那么...结果）格式 ，部分采用纯自然语言描述的形式。W–G–T 格式是一种基于行为驱动开发的需求表达方法，When 是触发条件，描述用户操作或系统事件；Given 是系统状态或前置条件；Then 则为系统的预期输出或行为。其核心思想是将系统需求转化为具体的场景，并以自然语言描述用户操作与系统期望行为。

1 简单功能性需求

1.1 用户登录功能

场景 1：正常登录

When 用户输入正确的用户名和密码

Given 用户账户状态正常且未被锁定

Then 系统允许用户成功登录并跳转至主界面

场景 2：输入错误

When 用户输入错误的用户名或密码

Given 用户账户状态正常

Then 系统提示“用户名或密码错误”，并拒绝登录

场景 3：账户锁定

When 用户连续三次输入错误的用户名或密码

Given 系统检测到错误次数达到 3 次

Then 系统锁定该账户 10 分钟，并在此期间禁止登录

2 非功能性需求

2.1 系统性能需求

场景 1：正常负载

When 系统处于 1000 并发用户访问的正常负载状态

Given 用户请求任意功能操作

Then 系统响应时间不超过 2 秒

场景 2：高峰负载

When 系统处于 5000 并发用户访问的高峰负载状态

Given 用户请求任意功能操作

Then 系统响应时间不超过 5 秒

3 业务流程需求

3.1 电商下单流程（验证模块拆分与集成测试能力）

场景 1：正常下单

When 用户将商品加入购物车并选择收货地址与支付方式

Given 系统校验库存充足且支付成功

Then 系统生成订单并向用户发送确认邮件

场景 2：库存不足

When 用户提交订单请求

Given 系统检测到库存不足

Then 系统提示“库存不足”，并允许用户修改订单或重试

场景 3：支付失败

When 用户提交支付请求

Given 支付渠道返回失败状态

Then 系统提示“支付失败”，并允许用户重试支付

3.2 抢购活动库存扣减（验证并发与竞争场景生成能力）

系统将上线一场限时抢购活动，商品为“限量款运动鞋”，总库存 100

件，活动时间为 2025 年 4 月 1 日 10:00–10:30。用户可通过商品详情页点击“立即抢购”按钮参与活动。系统需满足以下规则：

（1） 库存控制：每售出一件商品，库存实时扣减；当库存为 0 时，不再接受新的抢购请求。

（2） 限购策略：每位用户最多只能成功抢购 1 件，不可重复下单。

（3） 活动时效性：活动开始前不可抢购，活动结束后按钮置灰，无法提交请求。

（4） 结果反馈：抢购成功后跳转至订单确认页；失败则根据原因提示“库存已售罄”“您已抢购过该商品”或“活动未开始”等具体信息。

3.3 在线考试系统答题与提交流程功能（验证多模块深度交互+ 非功能性需求）

（1） 功能架构：包含 “登录→选题→答题→交卷→成绩统计”5 个模块，模块间存在严格流转顺序（如未登录无法选题，未答题无法交卷）；

（2） 关键规则：

选题模块：用户登录后可选择 “指定试卷”（需试卷状态为“已发布”），选题后进入答题页，答题时长不超过试卷规定时长（如试卷 A 时长 60 分钟，从选题成功开始计时）；答题模块：支持 “单题提交”（提交后不可修改）和“全部答完后统一提交”，断网时自动保存已答题数据（断网≤5 分钟重连可恢复，＞5 分钟视为考试超时）；交卷模块：①主动交卷：需确认“是否提交”，确认后不可修改；②自动交卷：时长结束前 10 秒提示，超时后强制提交；交卷后跳转至成绩页；成绩统计模块：根据答题正确数计算得分（每题分值固定，如单选 10 分/题），得分≥60 分为 “合格”，否则 “不合格”，成绩实时存入用户档案；

（3） 确保不同设备、浏览器 / 系统版本的用户都能正常查看答题页面、完成操作，避免因兼容性问题导致部分用户无法考试（如按钮遮挡、文字截断、操作无响应），覆盖主流用户使用场景。

设备与软件版本范围：需覆盖 PC 端浏览器（Chrome 120.0 及以上版本）、移动端 APP(iOS 17 及以上版本的 iPhone15 系列、Android14 及以上版本的华为 Mate 60 系列）；

屏幕分辨率要求：PC 端浏览器窗口分辨率≥1366×768（主流办公显示器尺寸），移动端设备屏幕分辨率≥720×1280（避免低分辨率导致布局错乱）；

附录 C 测试用例及评分

表 C-1 需求 1.1 对应测试用例：Zero-shot

序号	用例名称	前置条件	测试步骤	预期结果
1	正常登录-成功	1. 存在用户 testuser，密码 Pass123! 2.账户状态为正常”	1. 打开登录页面 2. 在用户名输入框输入：testuser 3. 在密码输入框输入：Pass123! 4. 点击“登录”按钮	1.系统验证通过 2.跳转至系统主界面 3.登录态被记录（如页面显示用户名）
2	登录-用户名错误	1. 存在用户 testuser，密码 Pass123! 2. 账户状态为“正常”	1. 打开登录页面 2. 输入用户名：wronguser 3. 输入正确密码：Pass123! 4. 点击“登录”按钮	1. 系统提示：用户名或密码错误 2.停留在登录页面，不清空输入框
3	登录-密码错误	1. 存在用户 testuser，密码 Pass123! 2.账户状态为正常	1. 打开登录页面 2. 输入正确用户名：testuser 3. 输入错误密码：WrongPass! 4. 点击“登录”按钮	1. 系统提示：“用户名或密码错误” 2. 停留在登录页面，密码框被清空，用户名保留（常见设计）
4	登录-用户名为空	1.账户状态为正常	1. 打开登录页面 2. 用户名留空 3. 输入任意密码（如：Pass123!） 4. 点击“登录”按钮	1. 系统在用户名输入框附近提示“用户名不能为空”之类的信息 2. 不发起网络请求，页面无跳转
5	登录-密码为空	1.账户状态为正常	1. 打开登录页面 2. 输入正确用户名（如：testuser）3. 密码留空 4. 点击“登录”按钮	1. 系统在密码输入框附近提示“密码不能为空”之类的信息 2. 不发起网络请求，页面无跳转
6	连续错误登录导致账户锁定	1. 存在用户 testuser，密码 Pass123! 2. 账户状态为“正常”，错误次数为 0	1. 打开登录页面 2. 使用用户名 testuser 和错误密码，点击“登录” 3. 重复步骤 2，共 3 次	1.前两次登录，系统提示“用户名或密码错误” 2.第三次登录后，系统提示：“因多次尝试失败，账户已被锁定 10 分钟” 3.该账户状态变为锁定
7	账户锁定后尝试登录	1. 用户 testuser 的账户状态为“锁定”	1. 打开登录页面 2. 输入用户名：testuser 3. 输入正确密码：Pass123! 4. 点击“登录”按钮	1. 系统提示：账户已被锁定，请 10 分钟后再试 2. 拒绝登录，停留在登录页面

续表 C-1

序号	用例名称	前置条件	测试步骤	预期结果
8	账户锁定超时后自动解锁	1. 用户因连续 3 次错误已被锁定 2. 等待超过 10 分钟	1. 等待 10 分钟以上 2. 打开登录页面 3. 输入正确的用户名和密码，点击“登录”按钮	1. 系统验证通过 2. 成功登录并跳转至主界面 3. 账户的错误次数被重置，状态恢复为“正常”
9	边界值-用户名最大长度	1. 规则：用户名最大支持 20 字符 2. 已存在用户，密码 123	1. 打开登录页面 2. 输入该 20 位的用户名 3. 输入对应密码 4. 点击“登录”按钮	系统允许登录成功
10	边界值-用户名超长	1. 规则：用户名最大支持 20 字符	1. 打开登录页面 2. 输入 21 位字符的用户名，任意密码，点击登录	1. 系统应在前端校验并提示“用户名长度超限”，或提交后后端返回错误信息 2. 登录失败
11	边界值-密码最大长度	1. 规则：密码最大支持 16 字符 2. 已存在用户密码为 16 位	1. 打开登录页面 2. 输入用户名和 16 位的正确密码，点击“登录”按钮	系统允许登录成功
12	边界值-密码超长	1. 规则：密码最大支持 16 字符	1. 打开登录页面 2. 输入正确用户名和 17 位字符的密码，点击登录	1. 系统应在前端校验并提示“密码长度超限”，或提交后后端返回错误信息 2. 登录失败

表 C-2 需求 1.1 对应测试用例：Few-shot

用例编号	用例标题	前置条件	测试步骤	预期结果
LOG-001	（正常）验证输入正确用户名和密码时登录成功	1. 系统运行正常 2. 用户账户状态正常且未被锁定	1.输入正确的用户名 2.输入正确的密码 3.点击"登录"按钮	1. 用户成功登录 2. 页面跳转至主界面
LOG-002	（异常）验证输入错误用户名时登录失败	1. 系统运行正常 2. 用户账户状态正常	1.输入错误的用户名 2.输入正确的密码 3.点击"登录"按钮	1. 系统提示"用户名或密码错误" 2. 登录失败
LOG-003	（异常）验证输入错误密码时登录失败	1. 系统运行正常 2. 用户账户状态正常	1.输入正确的用户名 2.输入错误的密码 3.点击"登录"按钮	1. 系统提示"用户名或密码错误" 2. 登录失败
LOG-004	（异常）验证用户名为空时登录失败	1. 系统运行正常 2. 用户账户状态正常	1.不输入用户名 2.输入密码 3.点击"登录"按钮	1. 系统提示"请输入用户名" 2. 登录失败
LOG-005	（异常）验证密码为空时登录失败	1. 系统运行正常 2. 用户账户状态正常	1.输入用户名 2.不输入密码 3. 点击"登录"按钮	1. 系统提示"请输入密码" 2. 登录失败

续表 C-2

用例编号	用例标题	前置条件	测试步骤	预期结果
LOG-006	（异常）验证连续三次输入错误时账户被锁定	1. 系统运行正常 2. 用户账户状态正常	1. 连续三次输入错误的用户名和密码 2. 点击"登录"按钮	1. 系统提示"账号已锁定，请 10 分钟后重试" 2. 账户被锁定
LOG-007	（异常）验证账户锁定 10 分钟后重新登录	1. 系统运行正常 2. 用户账户状态被锁定	1. 等待 10 分钟后 2. 输入正确的用户名和密码 3. 点击"登录"按钮	1. 用户成功登录 2. 页面跳转至主界面
LOG-008	（异常）验证账户锁定 9 分钟后重新登录	1. 系统运行正常 2. 用户账户状态被锁定	1. 等待 9 分钟后 2. 输入正确的用户名和密码 3. 点击"登录"按钮	1. 系统提示"账号已锁定，请 10 分钟后重试" 2. 登录失败
LOG-009	（异常）验证账户锁定 11 分钟后重新登录	1. 系统运行正常 2. 用户账户状态被锁定	1. 等待 11 分钟后 2. 输入正确的用户名和密码 3. 点击"登录"按钮	1. 用户成功登录 2. 页面跳转至主界面
LOG-010	（异常）验证用户名大小写敏感时登录失败	1. 系统运行正常 2. 用户账户状态正常	1. 输入大小写混合的用户名（如"Abc123"） 2. 输入正确的密码 3. 点击"登录"按钮	1. 系统提示"用户名或密码错误" 2. 登录失败
LOG-011	（边界）验证用户名长度过短时登录失败	1. 系统运行正常 2. 用户账户状态正常	1. 输入长度为 1 的用户名 2. 输入正确的密码 3. 点击"登录"按钮	1. 系统提示"用户名长度应为 2-20 位" 2. 登录失败
LOG-012	（边界）验证用户名长度过长时登录失败	1. 系统运行正常 2. 用户账户状态正常	1. 输入长度为 21 的用户名 2. 输入正确的密码 3. 点击"登录"按钮	1. 系统提示"用户名长度应为 2-20 位" 2. 登录失败
LOG-013	（边界）验证密码长度过短时登录失败	1. 系统运行正常 2. 用户账户状态正常	1.输入正确的用户名 2.输入长度=1 的密码 3.点击"登录"按钮	1. 系统提示"密码长度应为 6-20 位" 2. 登录失败
LOG-014	（边界）验证密码长度过长时登录失败	1. 系统运行正常 2. 用户账户状态正常	1. 输入正确的用户名 2. 输入长度为 21 的密码 3. 点击"登录"按钮	1. 系统提示"密码长度应为 6-20 位" 2. 登录失败

表 C-3 需求 1.1 对应测试用例：CoT+Few-shot

用例编号	用例标题	前置条件	测试步骤	预期结果
LOG-001	[正常]用户名和密码均正确时登录成功	1. 系统运行正常 2. 登录页面已打开 3. 用户账户状态正常且未被锁定	1. 输入正确的用户名 2. 输入正确的密码 3. 点击“登录”按钮	登录成功，系统跳转至主界面
LOG-002	[异常]用户名错误时登录失败	1. 登录页面已打开 2. 用户账户状态正常	1. 输入错误的用户名 2. 输入正确的密码 3. 点击“登录”按钮	登录失败，系统提示“用户名或密码错误”
LOG-003	[异常]密码错误时登录失败	1. 登录页面已打开 2. 用户账户状态正常	1. 输入正确的用户名 2. 输入错误的密码 3. 点击“登录”按钮	登录失败，系统提示“用户名或密码错误”
LOG-004	[异常]用户名和密码均为空时登录失败	登录页面已打开	1. 保持用户名为空 2. 保持密码为空 3. 点击“登录”按钮	登录失败，系统提示“请输入用户名和密码”
LOG-005	[异常]用户名为空时登录失败	登录页面已打开	1. 保持用户名为空 2. 输入正确的密码 3. 点击“登录”按钮	登录失败，系统提示“请输入用户名”
LOG-006	[异常]密码为空时登录失败	登录页面已打开	1. 输入正确的用户名 2. 保持密码为空 3. 点击“登录”按钮	登录失败，系统提示“请输入密码”
LOG-007	[异常]用户名包含非法字符时登录失败	登录页面已打开	1. 输入包含特殊字符的用户名 2. 输入正确的密码 3. 点击“登录”按钮	登录失败，系统提示“用户名格式错误”
LOG-008	[异常]密码包含非法字符时登录失败	登录页面已打开	1. 输入正确的用户名 2. 输入包含非法字符的密码 3. 点击“登录”按钮	登录失败，系统提示“密码格式错误”
LOG-009	[异常]账户已被锁定时登录失败	1. 登录页面已打开 2. 用户账户已被锁定	1. 输入正确的用户名 2. 输入正确的密码 3. 点击“登录”按钮	登录失败，系统提示“账户已被锁定，请10 分钟后再试”

续表 C-3

用例编号	用例标题	前置条件	测试步骤	预期结果
LOG-010	[边界]用户名达到最大长度时登录成功	1. 进入登录页； 2. 用户账户状态正常且未被锁定 3. 已创建用户名的长度等于系统允许的最大长度	1. 输入达到最大长度的有效用户名 2. 输入正确的密码 3. 点击“登录”按钮	登录成功，系统跳转至主界面
LOG-011	[边界]密码达到最大长度时登录成功	1. 进入登录页 3. 用户账户状态正常且未被锁定 4. 用户密码长度等于系统允许的最大长度	1. 输入正确的用户名 2. 输入达到最大长度的正确密码 3. 点击“登录”按钮	登录成功，系统跳转至主界面
LOG-012	[边界]连续三次错误登录后账户被锁定	1. 登录页面已打开 2. 用户账户状态正常	1. 输入错误的用户名或密码 2. 点击“登录”按钮 3. 重复步骤 1-2 两次（共三次错误尝试）	第三次登录失败后，系统提示“账户已被锁定，请 10 分钟后再试”
LOG-013	[边界]锁定账户等待 10 分钟后自动解锁	1. 用户账户因连续三次错误登录已被锁定 2. 锁定开始时间已记录	1. 等待超过 10 分钟锁定时限 2. 输入正确的用户名 3. 输入正确的密码 4. 点击“登录”按钮	登录成功，系统跳转至主界面，账户锁定状态已解除
LOG-014	[异常]尝试登录不存在的用户账户	登录页面已打开	1. 输入系统中不存在的用户名 2. 输入任意密码 3. 点击“登录”按钮	登录失败，系统提示“用户名或密码错误”

表 C-4 需求 1.1 对应测试用例：CoT+Few-shot+模板化提示+Adaptive

用例编号	用例标题	前置条件	测试步骤	预期结果
TC101	验证用户可正常登录（正向）	1. 系统运行正常，处于登录页面 2. 数据库中存在有效账户：用户名 user@test.com，密码 CorrectPass123，账户状态正常且未被锁定	1. 在用户名输入框输入：user@test.com 2. 在密码输入框输入：CorrectPass123 3. 点击"登录"按钮	1. 用户名输入成功 2. 密码输入成功 3. 登录成功，页面跳转至主界面，显示“欢迎，user@test.com”

续表 C-4

用例编号	用例标题	前置条件	测试步骤	预期结果
TC102	验证登录输入错误提示（反向）	1. 系统处于登录页面 2. 存在有效账户：用户名 user@test.com，密码 CorrectPass123 3. 准备错误数据： - 用户名正确，密码错误：WrongPass456 - 用户名错误：fake@test.com，密码任意	1. 输入用户名：user@test.com，密码：WrongPass456 2. 点击"登录"按钮 3. 输入用户名：fake@test.com，密码：任意值 4. 点击"登录"按钮	1. 用户名和密码输入完成 2. 登录失败，停留在登录页面，提示“用户名或密码错误” 3. 用户名和密码输入完成 4. 登录失败，停留在登录页面，提示“用户名或密码错误”
TC103	验证连续三次错误触发账户锁定（异常）	1. 系统处于登录页面 2. 存在有效账户：用户名 user@test.com，密码 CorrectPass123 3. 账户当前未被锁定，错误计数器为 0	1. 输入用户名：user@test.com，密码：Error1 2. 点击"登录"按钮 3. 输入用户名：user@test.com，密码：Error2 4. 点击"登录"按钮 5. 输入用户名：user@test.com，密码：Error3 6. 点击"登录"按钮	1. 输入完成 2. 登录失败，提示“用户名或密码错误”，错误计数器=1 3. 输入完成 4. 登录失败，提示“用户名或密码错误”，错误计数器=2 5. 输入完成 6. 登录失败，提示“账户已被锁定，请 10 分钟后重试”，登录按钮置灰不可点击
TC104	验证账户锁定期间禁止登录（状态）	1. 账户 user@test.com 已被系统锁定（因连续 3 次错误） 2. 锁定开始时间：T，当前时间在 T 至 T+10min 内 3. 系统时钟同步准确	1. 在用户名输入框输入：user@test.com 2. 在密码输入框输入：CorrectPass123 3. 点击"登录"按钮	1. 用户名输入成功 2. 密码输入成功 3. 登录失败，提示“账户已被锁定，请 10 分钟后重试”，登录按钮保持置灰不可点击
TC105	验证账户锁定 10 分钟后自动解锁（状态）	1.账户 u@t.com 已被锁定 2.当前时间≥锁定开始时间+10 分钟 3.系统时钟同步准确	1.用户名输入 u@t.com 2.输入密码 CorrectPass123 3.点击"登录"按钮	1. 用户名输入成功 2. 密码输入成功 3. 登录成功，页面跳转至主界面，显示“欢迎，u@t.com”，登录按钮恢复可点击状态

续表 C-4

用例编号	用例标题	前置条件	测试步骤	预期结果
TC106	验证账户锁定期间前端状态同步与用户引导(状态)	1. 账户 user@test.com 已被锁定 2. 用户停留在登录页面超过锁定生效时间 3. 页面未刷新或重新加载	1. 等待锁定生效后，观察页面提示 2. 尝试点击登录按钮 3. 手动刷新页面 4. 再次尝试输入正确凭据登录	1. 页面自动显示“账户已被锁定，请 10 分钟后重试” 2. 登录按钮置灰不可点击 3. 页面刷新后提示信息仍存在，按钮仍不可点击 4. 输入正确凭据后，在锁定期内仍无法登录，提示信息保持不变
TC107	验证并发多次错误登录触发锁定（并发）	1. 账户 user@test.com 初始状态正常，错误计数器=0 2. 模拟三个并发请求，分别使用错误密码 ErrorA、ErrorB、ErrorC 3. 系统支持并发请求处理	1. 并发发送三次登录请求（用户名均为 user@test.com，密码分别为 ErrorA、ErrorB、ErrorC） 2. 等待所有请求返回 3. 立即使用正确密码 CorrectPass123 尝试第四次登录	1. 三次请求均返回“用户名或密码错误” 2. 系统记录错误次数=3，触发锁定 3. 第四次登录失败，提示“账户已被锁定，请 10 分钟后重试”，登录按钮置灰不可点击

对于这 4 种提示词生成的测试用例，4 位测试工程师的评分如下：

表 C-5 需求 1.1 对应的 QA 打分 1

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	3	2	3.25
Few-shot	3	4	3	2	3
CoT+Few-shot	4	4	4	3	3.75
融合策略	5	5	5	4	4.75

表 C-6 需求 1.1 对应的 QA 打分 2

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	5	5	5	3	4.5
Few-shot	4	4	4	2	3.5
CoT+Few-shot	4	4	4	3	3.75
融合策略	5	5	5	4	4.75

表 C-7 需求 1.1 对应的 QA 打分 3

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3	5	5	4	4.25
Few-shot	3	4	4	3	3.5
CoT+Few-shot	4	4	5	4	4.25
融合策略	5	5	5	5	5

表 C-8 需求 1.1 对应的 QA 打分 4

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	5	4	3	4
Few-shot	3	3	4	3	3.25
CoT+Few-shot	4	4	5	4	4.25
融合策略	5	5	5	5	5

表 C-9 需求 2.1 对应测试用例：Zero-shot

序号	用例名称	前置条件	测试步骤	预期结果
1	正常下单流程-成功	1. 用户已登录且账户正常 2. 商品 A 库存充足(>0) 3. 用户有默认收货地址和支付方式	1. 将商品 A 加入购物车 2. 进入购物车，点击“结算” 3. 选择收货地址和支付方式 4. 点击“提交订单” 5. 完成支付流程	1. 系统生成状态为“待发货”的订单 2. 商品 A 库存相应减少 3. 系统向用户注册邮箱发送订单确认邮件
2	下单流程-库存不足提示	1. 用户已登录 2. 商品 B 库存为 0	1.将商品 B 加入购物车 2.进入购物车，点击“结算” 3. 选择收货地址和支付方式 4. 点击“提交订单”	1. 系统弹出提示:“库存不足” 2. 停留在当前页面，允许用户返回修改购物车或重新尝试
3	下单流程-支付失败提示	1. 用户已登录 2. 商品 A 库存充足 3. 支付渠道模拟失败(或网络异常)	1. 正常提交订单 2. 在支付页面，点击“确认支付”	1. 支付渠道返回失败信息 2. 系统弹出提示:“支付失败，请重试” 3.订单状态为待支付，允许用户重新发起支付

续表 C-9

序号	用例名称	前置条件	测试步骤	预期结果
4	边界值-购买数量为1（最小值）	1. 用户已登录 2. 商品 A 库存 >= 1	1. 将商品 A 购买数量设置为 1 2. 完成正常下单流程	1. 下单成功，库存减少 1
5	边界值-购买数量等于最大库存	1. 用户已登录 2. 商品 A 库存为 10	1. 将商品 A 购买数量设置为 10 2. 完成正常下单流程	1. 下单成功，库存减少为 0
6	边界值-购买数量超过最大库存	1. 用户已登录 2. 商品 A 库存为 10	1. 将商品 A 购买数量设置为 11 2. 尝试提交订单	1. 系统提示“库存不足，当前仅剩 10 件”
7	场景-支付失败后重试成功	1. 存在一个因支付失败而状态为“待支付”的订单	1. 在订单列表找到该订单 2. 点击“立即支付” 3. 完成支付流程	1. 支付成功 2. 订单状态更新为“待发货” 3. 库存在第一次提交订单时已锁定，此时扣减
8	场景-库存不足后移出商品并成功下单	1. 用户已登录 2. 购物车中有库存为 0 的商品 B 和库存充足的商品 A	1. 进入购物车，发现商品 B 显示“库存不足” 2. 移除商品 B 3. 对商品 A 进行结算并支付	1. 系统生成仅包含商品 A 的订单 2. 下单成功

表 C-10 需求 2.1 对应测试用例：Few-shot

用例编号	用例标题	前置条件	测试步骤	预期结果
PERF-001	（性能）验证1000 并发用户下系统响应时间不超过 2 秒	1. 测试环境已部署并运行正常 2. 负载测试工具（如 JMeter）已配置好 3. 系统处于待测试状态	1. 使用负载测试工具模拟 1000 个并发用户 2. 每个用户执行一个标准业务操作（例如：登录操作） 3. 记录所有用户请求的响应时间	1. 所有用户请求的平均响应时间 ≤2 秒 2. 系统无错误或崩溃

续表 C-10

用例编号	用例标题	前置条件	测试步骤	预期结果
PERF-002	（性能）验证5000 并发用户下系统响应时间不超过 5 秒	1. 测试环境已部署并运行正常 2. 负载测试工具已配置好 3. 系统处于待测试状态	1. 使用负载测试工具模拟 5000 个并发用户 2. 每个用户执行一个标准业务操作 3. 记录所有用户请求的响应时间	1. 所有用户请求的平均响应时间 ≤5 秒 2. 系统无错误或崩溃
PERF-003	（性能）边界 - 验证 999 并发用户下系统响应时间不超过 2 秒	1. 测试环境已部署并运行正常 2. 负载测试工具已配置好 3. 系统处于待测试状态	1. 使用负载测试工具模拟 999 个并发用户 2. 每个用户执行一个标准业务操作 3. 记录所有用户请求的响应时间	1. 所有用户请求的平均响应时间 ≤2 秒 2. 系统无错误或崩溃
PERF-004	（性能）边界 - 验证 1001 并发用户下系统响应时间超过 2 秒	1. 测试环境已部署并运行正常 2. 负载测试工具已配置好 3. 系统处于待测试状态	1. 使用负载测试工具模拟 1001 个并发用户 2. 每个用户执行一个标准业务操作 3. 记录所有用户请求的响应时间	1. 至少有一个用户请求的响应时间 >2 秒 2. 系统无错误或崩溃
PERF-005	（持续时间）验证正常负载长时间运行稳定性	1. 测试环境已部署并运行正常 2. 性能测试工具已配置完毕	1. 使用性能测试工具模拟 1000 个并发用户 2. 用户执行典型功能操作 3. 持续运行 2 小时 4. 监控系统资源使用率和响应时间	系统响应时间始终保持在≤2 秒，内存和 CPU 使用率正常，无内存泄漏
PERF-006	（持续时间）验证高峰负载长时间运行稳定性	1. 测试环境已部署并运行正常 2. 性能测试工具已配置完毕	1. 使用性能测试工具模拟 5000 个并发用户 2. 用户执行典型功能操作 3. 持续运行 1 小时 4. 监控系统资源使用率和响应时间	系统响应时间始终保持在≤5 秒，系统资源使用率在可接受范围内

表 C-11 需求 2.1 对应测试用例：CoT+Few-shot

用例编号	用例标题	前置条件	测试步骤	预期结果
PERF-001	[正常]1000 并发用户正常负载下响应时间达标	1. 系统部署完成且运行正常 2. 性能测试环境已准备就绪 3. 监控工具已部署	1. 使用性能测试工具模拟 1000 个并发用户 2. 执行典型业务操作（如登录、查询、提交等） 3. 监控系统响应时间	所有请求的响应时间均不超过 2 秒，系统运行稳定
PERF-002	[边界]5000 并发用户高峰负载下响应时间达标	1. 系统部署完成且运行正常 2. 性能测试环境已准备就绪 3. 监控工具已部署	1. 使用性能测试工具模拟 5000 个并发用户 2. 执行典型业务操作（如登录、查询、提交等） 3. 监控系统响应时间	所有请求的响应时间均不超过 5 秒，系统运行稳定
PERF-003	[异常]超过 5000 并发用户时系统性能表现	1. 系统部署完成且运行正常 2. 性能测试环境已准备就绪 3. 监控工具已部署	1. 使用性能测试工具模拟 6000 个并发用户 2. 执行典型业务操作 3. 监控系统响应时间和错误率	系统响应时间超过 5 秒，可能出现超时或错误，但系统不应完全崩溃
PERF-004	[边界]系统在最大设计容量下的稳定性	1. 系统部署完成且运行正常 2. 性能测试环境已准备就绪 3. 监控工具已部署	1. 使用性能测试工具模拟系统设计最大容量用户数 2. 持续运行 30 分钟 3. 监控系统资源使用率和错误率	系统在最大设计容量下稳定运行，资源使用率在合理范围内，无严重错误
PERF-005	[异常]系统在超负荷状态下的优雅降级	1. 系统部署完成且运行正常 2. 性能测试环境已准备就绪 3. 监控工具已部署	1. 使用性能测试工具模拟超出系统设计容量的用户数 2. 执行典型业务操作 3. 监控系统行为和错误处理	系统能够优雅降级，返回友好的错误提示而不是完全崩溃，核心功能仍可用
PERF-006	[边界]长时间持续负载下的系统稳定性	1. 系统部署完成且运行正常 2. 性能测试环境已准备就绪 3. 监控工具已部署	1. 使用性能测试工具模拟 3000 个并发用户 2. 持续运行 8 小时 3. 监控系统内存泄漏和性能衰减	系统在长时间运行后性能保持稳定，无内存泄漏，响应时间仍在要求范围内

表 C-12 需求 2.1 对应测试用例：CoT+Few-shot+模板化提示+Adaptive

用例编号	用例标题	前置条件	测试步骤	预期结果
TC101	（正常）验证系统在 1000 并发正常负载下的性能表现	1. 系统已部署在生产环境 2. 性能测试工具已配置完成 3. 测试脚本包含用户登录、浏览商品、提交订单等核心操作	1. 使用性能测试工具模拟 1000 个并发用户 2. 持续运行测试脚本 30 分钟 3. 监控系统响应时间、CPU 使用率、内存使用率等关键指标	1. 系统所有请求的平均响应时间≤2 秒 2. 系统运行稳定，无错误率或错误率低于 0.1% 3. 资源使用率在正常范围内，无内存泄漏或 CPU 过载
TC102	（边界）验证系统在 5000 并发高峰负载下的性能表现	1. 系统已部署在生产环境 2. 性能测试工具已配置完成 3. 测试脚本包含用户登录、浏览商品、提交订单等核心操作	1. 使用性能测试工具模拟 5000 个并发用户 2. 持续运行测试脚本 30 分钟 3. 监控系统响应时间、CPU 使用率、内存使用率等关键指标	1. 系统所有请求的平均响应时间≤5 秒 2. 系统运行稳定，无错误率或错误率低于 0.5% 3. 资源使用率在可接受范围内，系统未崩溃
TC103	（边界）验证系统在超过 5000 并发极限负载下的表现	1. 系统已部署在生产环境 2. 性能测试工具已配置完成	1. 使用性能测试工具模拟 6000 个并发用户 2. 持续运行测试脚本 15 分钟	1. 系统响应时间可能超过 5 秒但仍在可接受范围内 2. (1）系统可能出现部分请求失败但核心功能仍可用 （2）系统未完全崩溃，具备一定的容错能力

表 C-13 需求 2.1 对应的 QA 打分 1

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3	4	5	4	4
Few-shot	3	4	3	2	3
CoT+Few-shot	4	4	4	3	3.75
融合策略	5	5	3	3	4

表 C-14 需求 2.1 对应的 QA 打 2

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3	5	5	3	4
Few-shot	4	5	4	3	4
CoT+Few-shot	5	4	5	4	4.5
融合策略	5	4	4	3	4

表 C-15 需求 2.1 对应的 QA 打分 3

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3	5	5	3	4
Few-shot	4	5	4	3	4
CoT+Few-shot	5	4	5	4	4.5
融合策略	5	4	4	3	4

表 C-16 需求 2.1 对应的 QA 打分 4

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	5	4	4	4.25
Few-shot	4	3	4	4	3.75
CoT+Few-shot	4	3	5	5	4.25
融合策略	5	4	5	5	4.75

表 C-17 需求 3.1 对应测试用例：Zero-shot

序号	用例名称	前置条件	测试步骤	预期结果
1	正常下单流程-成功	1. 用户已登录且账户正常 2. 商品 A 库存充足(>0） 3. 用户有默认收货地址和支付方式	1. 将商品 A 加入购物车 2. 进入购物车，点击“结算” 3. 选择收货地址和支付方式 4. 点击“提交订单” 5. 完成支付流程	1.系统生成状态为待发货的订单 2.商品 A 库存相应减少 3. 系统向用户注册邮箱发送订单确认邮件
2	下单流程-库存不足提示	1. 用户已登录 2. 商品 B 库存为 0	1.将商品 B 加入购物车 2.进入购物车点击结算 3.选择收货地址和支付方式 4. 点击“提交订单”	1.系统弹出提示：库存不足 2. 停留在当前页面，允许用户返回修改购物车或重新尝试
3	下单流程-支付失败提示	1. 用户已登录 2. 商品 A 库存充足 3. 支付渠道模拟失败（或网络异常）	1. 正常提交订单 2. 在支付页面，点击“确认支付”	1. 支付渠道返回失败信息 2. 系统弹出提示：“支付失败，请重试” 3. 订单状态为“待支付”，允许用户重新发起支付

续表 C-17

序号	用例名称	前置条件	测试步骤	预期结果
4	边界值-购买数量为 1（最小值）	1. 用户已登录 2. 商品A库存 >= 1	1. 将商品 A 购买数量设置为 1 2. 完成正常下单流程	1. 下单成功，库存减少 1
5	边界值-购买数量等于最大库存	1. 用户已登录 2. 商品 A 库存为 10	1. 将商品 A 购买数量设置为 10 2. 完成正常下单流程	1. 下单成功，库存减少为 0
6	边界值-购买数量超过最大库存	1. 用户已登录 2. 商品 A 库存为 10	1. 将商品 A 购买数量设置为 11 2. 尝试提交订单	1. 系统提示“库存不足，当前仅剩 10 件”
7	场景-支付失败后重试成功	1. 存在一个因支付失败而状态为“待支付”的订单	1. 在订单列表找到该订单 2. 点击“立即支付” 3. 完成支付流程	1. 支付成功 2. 订单状态更新为“待发货” 3. 库存在第一次提交订单时已锁定，此时扣减
8	场景-库存不足后移出商品并成功下单	1. 用户已登录 2. 购物车中有库存为 0 的商品 B 和库存充足的商品 A	1. 进入购物车，发现商品 B 显示“库存不足” 2. 移除商品 B 3. 对商品 A 进行结算并支付	1. 系统生成仅包含商品 A 的订单 2. 下单成功

表 C-18 需求 3.1 对应测试用例：Few-shot

编号	用例标题	前置条件	测试步骤	预期结果
ORDER-001	（正常）验证商品库存充足时下单成功	1. 用户已登录 2. 商品 A 库存为 100 件 3. 用户收货地址已设置 4. 支付方式已绑定	1. 添加商品 A 到购物车（数量 1） 2. 进入购物车页面 3. 选择收货地址 4. 选择支付方式 5. 点击"提交订单" 6. 完成支付流程	1. 订单生成成功 2. 商品库存减少为 99 件 3. 收到订单确认邮件 4. 订单状态显示"待发货"

续表 C-18

编号	用例标题	前置条件	测试步骤	预期结果
ORDER-002	（异常）验证库存为零时下单失败	1. 用户已登录 2. 商品 B 库存为 0 件 3. 用户收货地址已设置	1. 尝试添加商品 B 到购物车 2. 进入购物车页面 3. 选择收货地址 4. 点击"提交订单"	1. 系统提示"库存不足" 2. 无法进入支付页面 3. 订单未生成
ORDER-003	（异常）验证库存不足时下单失败	1. 用户已登录 2. 商品 C 库存为 2 件 3. 用户收货地址已设置	1. 添加商品 C 到购物车（数量 3） 2. 进入购物车页面 3. 选择收货地址 4. 点击"提交订单"	1. 系统提示"库存不足" 2. 提示当前可用库存数量 3. 提供修改数量选项
ORDER-004	（异常）验证支付失败时订单状态	1. 用户已登录 2. 商品 D 库存充足 3. 模拟支付失败环境	1. 添加商品 D 到购物车 2. 进入购物车页面 3. 选择收货地址 4. 点击"提交订单" 5. 支付过程返回失败	1. 系统提示"支付失败" 2. 订单状态为"待支付" 3. 提供重新支付选项
ORDER-005	（边界）验证购买数量达到库存上限	1. 用户已登录 2. 商品 E 库存为 5 件 3. 用户收货地址已设置	1. 添加商品 E 到购物车（数量 5） 2. 进入购物车页面 3. 选择收货地址 4. 选择支付方式 5. 点击"提交订单" 6. 完成支付	1. 订单生成成功 2. 商品库存减少为 0 件 3. 商品自动标记为"缺货"
ORDER-006	（边界）验证购买单件商品下单成功	1. 用户已登录 2. 商品 F 库存为 1 件 3. 用户收货地址已设置	1. 添加商品 F 到购物车（数量 1） 2. 进入购物车页面 3. 选择收货地址 4. 选择支付方式 5. 点击"提交订单" 6. 完成支付	1. 订单生成成功 2. 商品库存减少为 0 件 3. 收到订单确认邮件

续表 C-18

编号	用例标题	前置条件	测试步骤	预期结果
ORDER-007	（异常）验证未选择收货地址时下单失败	1. 用户已登录 2. 商品 G 库存充足 3. 用户有多个收货地址	1. 添加商品 G 到购物车 2. 进入购物车页面 3. 不选择收货地址 4. 点击"提交订单"	1. 系统提示"请选择收货地址" 2. 无法进入支付页面
ORDER-008	（异常）验证未选择支付方式时下单失败	1. 用户已登录 2. 商品 H 库存充足 3. 用户有多个支付方式	1. 添加商品 H 到购物车 2. 进入购物车页面 3. 选择收货地址 4. 不选择支付方式 5. 点击"提交订单"	1. 系统提示"请选择支付方式" 2. 无法进入支付页面
ORDER-009	（场景）验证支付失败后重试支付成功	1. 用户已登录 2. 存在待支付订单 3. 支付渠道恢复正常	1. 进入订单详情页面 2. 点击"重新支付" 3. 完成支付流程	1. 支付成功 2. 订单状态更新为"待发货" 3. 收到支付成功通知
ORDER-010	（边界）验证超时未支付订单自动取消	1. 用户已登录 2. 存在待支付订单 3. 等待订单超时时间（如 30 分钟）	1. 创建待支付订单 2. 等待超时时间到达 3. 查看订单状态	1. 订单自动取消 2. 库存自动恢复 3. 通知用户订单已取消

表 C-19 需求 3.1 对应测试用例：CoT+Few-shot

编号	用例标题	前置条件	测试步骤	预期结果
ORDER-001	[正常]库存充足且支付成功时下单成功	1. 用户已登录 2. 商品库存充足 3. 支付渠道正常	1. 将商品加入购物车 2. 选择有效的收货地址 3. 选择可用的支付方式 4. 提交订单并完成支付	系统生成订单，跳转至订单成功页面，并向用户发送确认邮件

续表 C-19

编号	用例标题	前置条件	测试步骤	预期结果
ORDER-002	[异常]库存不足时下单失败	1.用户已登录 2.商品库存为 0	1. 将商品加入购物车 2. 选择收货地址和支付方式 3. 提交订单	系统提示"库存不足"，允许用户修改订单或重试，不生成订单
ORDER-003	[异常]支付失败时下单中止	1.用户已登录 2.商品充足 3.支付渠道模拟失败	1. 将商品加入购物车 2. 选择收货地址和支付方式 3. 提交订单并尝试支付	系统提示"支付失败"，允许用户重试支付，订单状态为待支付
ORDER-004	[异常]未选择收货地址时下单失败	1.用户已登录 2.商品充足	1. 将商品加入购物车 2. 不选择收货地址 3. 选择支付方式 4. 提交订单	系统提示"请选择收货地址"，阻止订单提交
ORDER-005	[异常]未选择支付方式时下单失败	1.用户已登录 2.商品库存充足	1. 将商品加入购物车 2. 选择收货地址 3. 不选择支付方式 4. 提交订单	系统提示"请选择支付方式"，阻止订单提交
ORDER-006	[边界]购买数量达到库存上限时下单成功	1.用户已登录 2.商品剩 10 件	1. 将10件商品加入购物车 2. 选择收货地址和支付方式 3. 提交订单并完成支付	系统生成订单，库存减少为 0，下单成功
ORDER-007	[边界]购买数量超过库存上限时下单失败	1. 用户已登录 2. 商品剩 10 件	1. 将11件商品加入购物车 2. 选择收货地址和支付方式 3. 提交订单	系统提示"库存不足，最多可购买 10 件"，阻止订单提交
ORDER-008	[异常]购物车为空时提交订单失败	1. 用户已登录 2. 购物车中无商品	1. 直接点击"提交订单"	系统提示"购物车为空，请先添加商品"，阻止订单提交
ORDER-009	[异常]支付超时处理	1. 用户已登录 2. 商品库存充足 3. 支付渠道响应超时	1. 将商品加入购物车 2. 选择收货地址和支付方式 3. 提交订单并发起支付 4. 等待支付超时	系统提示"支付超时，请重试"，订单状态保持待支付，库存暂时锁定
ORDER-010	[边界]订单支付时限内完成支付	1. 用户已提交订单但未支付 2. 订单处于待支付状态 3. 支付时限为30分钟	1.29 分钟内完成支付操作	支付成功，订单状态更新为已支付，库存相应减少

表 C-20 需求 3.1 对应测试用例：CoT+Few-shot+模板化提示+Adaptive

编号	用例标题	前置条件	测试步骤	预期结果
TC201	验证用户可正常下单并接收确认邮件（正向）	1. 系统运行正常，用户已登录 2. 购物车中包含商品：SKU123（库存≥1） 3. 用户已设置默认收货地址：北京市朝阳区 XX 路 1 号 4. 支付方式已绑定：支付宝账户 user@test.com 5. 邮件服务可用	1. 进入购物车页面，点击“去结算” 2. 确认收货地址为“北京市朝阳区 XX 路 1 号” 3. 选择支付方式为“支付宝” 4. 点击“提交订单”按钮 5. 在支付页面点击“确认支付”	1. 页面跳转至订单确认页，地址与支付方式显示正确 2. 地址信息无误，不可编辑状态 3. 支付方式选择成功，显示“支付宝”图标 4. 订单提交成功，跳转至支付页面 5. 支付成功，跳转至订单完成页，显示“订单已生成”；系统发送确认邮件至 user@test.com，邮件标题为“订单确认 - 您的订单已成功创建”
TC202	验证库存不足时提示并提供修改入口（反向）	1. 系统运行正常，用户已登录 2. 购物车中包含商品：SKU456（库存=0） 3. 用户已设置默认收货地址 4. 支付方式已绑定	1. 进入购物车页面，点击“去结算” 2. 确认收货地址与支付方式 3. 点击“提交订单”按钮	1. 页面跳转至订单确认页，地址与支付方式显示正常 2. 页面无异常，数据加载完整 3. 提交失败，页面停留在订单确认页，弹出提示“库存不足”；“修改订单”按钮显示并可点击，点击后跳转至购物车页面；“重试”按钮显示并可点击，点击后重新校验库存
TC203	验证支付失败时提示并提供重试入口（异常）	1. 系统运行正常，用户已登录 2. 购物车中包含商品：SKU789（库存≥1） 3. 用户已设置默认收货地址 4. 支付方式已绑定，但模拟支付渠道返回失败	1. 进入购物车页面，点击“去结算” 2. 确认收货地址与支付方式 3. 点击“提交订单”按钮 4. 在支付页面点击“确认支付”	1. 页面跳转至订单确认页，信息显示完整 2. 地址与支付方式无误 3. 订单提交成功，跳转至支付页面 4. 支付失败，页面弹出提示“支付失败”；“重试支付”按钮显示并可点击，点击后重新进入支付流程；“返回修改”链接显示并可点击，点击后返回订单确认页

续表 C-20

编号	用例标题	前置条件	测试步骤	预期结果
TC204	验证库存临界值边界场景（边界）	1. 系统运行正常，用户已登录 2. 购物车中包含商品：SKU101（库存=1） 3. 用户 A 与用户 B 同时操作 4. 用户已设置默认收货地址与支付方式	1. 用户 A 提交订单并完成支付 2. 用户 B 在用户 A 支付成功后立即提交同一商品订单	1. 用户 A 支付成功，订单生成，库存减为 0 2. 用户 B 提交失败，页面提示“库存不足”；“修改订单”按钮可点击，跳转至购物车；“重试”按钮可点击，触发库存重新查询
TC205	验证支付超时后状态恢复与重试能力（状态）	1. 系统运行正常，用户已登录 2. 购物车中包含商品：SKU202（库存≥1） 3. 用户已设置默认收货地址 4. 支付页面设置 30 秒超时机制	1. 进入支付页面，停留 35 秒不操作 2. 超时后点击“确认支付” 3. 点击“重试支付”按钮	1. 页面保持支付界面，无自动跳转 2. 支付失败，提示“支付超时，请重试”；“重试支付”按钮显示并可点击 3. 点击后重新进入有效支付流程，可正常提交支付请求
TC206	验证多用户并发抢购最后 1 件库存（并发）	1. 系统运行正常，多个用户已登录 2. 商品 SKU303 库存=1 3. 所有用户购物车均含该商品，且地址/支付方式已配置	1. 用户 A、B、C 同时点击“提交订单” 2. 系统处理并发请求	1. 仅 1 位用户（如 A）支付成功，生成订单，库存=0 2. 其余用户（B、C）页面提示“库存不足”；“修改订单”和“重试”按钮均显示并可点击，点击后分别跳转购物车或触发库存刷新
TC207	验证下单后邮件服务异常时订单状态与用户提示（集成）	1. 系统运行正常，用户已登录 2. 购物车中包含商品：SKU404（库存≥1） 3. 支付成功，但邮件服务模拟宕机	1. 完成下单并支付成功 2. 系统尝试发送确认邮件 3. 用户进入“我的订单”页面查看	1. 支付成功，订单状态为“已支付” 2. 邮件发送失败，系统后台记录错误日志，但不阻断订单流程 3. 订单列表显示该订单，状态正常；页面无邮件发送失败提示，但提供“重新发送确认邮件”按钮，按钮可点击并触发重发逻辑

表 C-21 需求 3.1 对应的 QA 评分 1

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	3	2	3.25
Few-shot	5	5	4	3	4.25
CoT+Few-shot	5	4	4	3	4
融合策略	5	5	5	5	5

表 C-22 需求 3.1 对应的 QA 评分 2

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	5	5	4	3	4.25
Few-shot	4	4	4	3	3.75
CoT+Few-shot	4	4	4	4	4
融合策略	5	5	5	5	5

表 C-23 需求 3.1 对应的 QA 评分 3

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	5	4	4	4.25
Few-shot	3	5	5	4	4.25
CoT+Few-shot	4	5	5	4	4.5
融合策略	5	5	4	5	4.75

表 C-24 需求 3.1 对应的 QA 评分 4

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	5	4	3.5	3.225
Few-shot	3	3	3	3	3
CoT+Few-shot	4	5	4	4	4.25
融合策略	5	5	5	5	5

表 C-25 需求 3.2 对应测试用例：Zero-shot

序号	用例名称	前置条件	测试步骤	预期结果
1	活动开始前抢购尝试	1. 系统时间为 2025-04-01 09:59:59 2. 用户已登录 3. 商品库存为 100	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 按钮不可点击或提示“活动未开始”

续表 C-25

序号	用例名称	前置条件	测试步骤	预期结果
2	活动开始时抢购成功	1. 系统时间为 2025-04-01 10:00:00 2. 用户已登录且未抢购过 3. 商品库存为 100	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 跳转至订单确认页 2. 库存扣减至 99
3	活动结束时抢购尝试	1. 系统时间为 2025-04-01 10:30:00 2. 用户已登录 3. 商品库存大于 0	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 按钮置灰，无法提交或提示“活动已结束”
4	活动结束后抢购尝试	1. 系统时间为 2025-04-01 10:30:01 2. 用户已登录 3. 商品库存大于 0	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 按钮置灰，无法提交或提示“活动已结束”
5	正常抢购成功	1. 系统时间在活动内（如 10:15） 2. 用户已登录且未抢购过 3. 商品库存大于 0	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 跳转至订单确认页 2. 库存实时扣减
6	库存售罄时抢购	1. 系统时间在活动内 2. 用户已登录且未抢购过 3. 商品库存为 0	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 提示“库存已售罄”
7	用户重复抢购尝试	1. 系统时间在活动内 2. 用户已成功抢购过该商品 3. 商品库存大于 0	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 提示“您已抢购过该商品”
8	库存边界值抢购（库存为 1）	1. 系统时间在活动内 2. 用户已登录且未抢购过 3. 商品库存为 1	1. 访问商品详情页 2. 点击“立即抢购”按钮	1. 跳转至订单确认页 2. 库存扣减至 0

表 C-26 需求 3.2 对应测试用例：Few-shot

编号	用例标题	前置条件	测试步骤	预期结果
SALE-001	（正常）验证活动期间库存充足时抢购成功	1. 用户已登录 2. 当前时间为 2025 年 4 月 1 日 10:15 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮 3. 确认订单信息	1. 跳转至订单确认页面 2. 库存数量减少 1 3. 显示抢购成功提示
SALE-002	（异常）验证库存为 0 时抢购失败	1. 用户已登录 2. 当前时间为活动时间内 3. 商品库存为 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	1. 提示"库存已售罄" 2. 按钮变为不可点击状态
SALE-003	（异常）验证同一用户重复抢购失败	1. 用户已登录且已成功抢购过该商品 2. 当前时间为活动时间内 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	提示"您已抢购过该商品"
SALE-004	（异常）验证活动开始前抢购失败	1. 用户已登录 2. 当前时间为 2025 年 4 月 1 日 09:59 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	提示"活动未开始"
SALE-005	（异常）验证活动结束后抢购失败	1. 用户已登录 2. 当前时间为 2025 年 4 月 1 日 10:31 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	1. 按钮置灰不可点击 2. 提示"活动已结束"
SALE-006	（边界）验证最后一件商品抢购成功	1. 用户已登录 2. 当前时间为活动时间内 3. 商品库存为 1 件	1. 进入商品详情页 2. 点击"立即抢购"按钮 3. 确认订单信息	1. 跳转至订单确认页面 2. 库存减少为 0 3. 显示抢购成功提示
SALE-007	（边界）验证活动开始时刻准点抢购	1. 用户已登录 2. 当前时间刚好为 2025 年 4 月 1 日 10:00:00 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	1. 允许抢购操作 2. 跳转至订单确认页面
SALE-008	（边界）验证活动结束时刻抢购	1. 用户已登录 2. 当前时间刚好为 2025 年 4 月 1 日 10:30:00 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	1. 允许抢购操作 2. 跳转至订单确认页面

续表 C-26

编号	用例标题	前置条件	测试步骤	预期结果
SALE-009	（异常） 验证未登录用户抢购失败	1.用户未登录 2.当前时间为活动时间内 3. 商品库存大于 0	1. 进入商品详情页 2. 点击"立即抢购"按钮	跳转至登录页面，要求先登录
SALE-010	（边界） 验证库存从 1 减到 0 的临界状态	1. 用户 A 和用户 B 均已登录 2. 当前时间为活动时间内 3. 商品库存为 1 件	1. 用户 A 点击"立即抢购"并完成支付 2. 用户 B 立即尝试点击"立即抢购"	1. 用户 A 抢购成功 2. 用户 B 提示"库存已售罄"

表 C-27 需求 3.2 对应测试用例：CoT+Few-shot

用例编号	用例标题	前置条件	测试步骤	预期结果
SALE-001	[正常]活动时间内库存充足时抢购成功	1. 系统时间在 2025 年 4 月 1 日 10:00-10:30 之间 2. 用户未抢购过该商品 3. 商品库存大于 0	1. 访问商品详情页 2. 点击"立即抢购"按钮 3. 确认订单信息	抢购成功，跳转至订单确认页，库存实时扣减 1
SALE-002	[异常]库存为 0 时抢购失败	1. 系统时间在活动时间内 2. 商品库存为 0	1. 访问商品详情页 2. 点击"立即抢购"按钮	抢购失败，提示"库存已售罄"，按钮置灰
SALE-003	[异常]用户重复抢购时失败	1. 系统时间在活动时间内 2. 用户已成功抢购过该商品 3. 商品库存大于 0	1. 访问商品详情页 2. 点击"立即抢购"按钮	抢购失败，提示"您已抢购过该商品"
SALE-004	[异常]活动开始前抢购失败	1. 系统时间为 2025 年 4 月 1 日 09:59 2. 商品库存充足	1. 访问商品详情页 2. 点击"立即抢购"按钮	抢购失败，提示"活动未开始"，按钮不可用
SALE-005	[异常]活动结束后抢购失败	1. 系统时间为 2025 年 4 月 1 日 10:31 2. 商品库存充足	1. 访问商品详情页 2. 点击"立即抢购"按钮	抢购失败，提示"活动已结束"，按钮置灰

续表 C-27

编号	用例标题	前置条件	测试步骤	预期结果
SALE-006	[边界]活动开始时间准点抢购成功	1. 系统时间刚好为2025 年 4 月 1 日 10:00:00 2. 用户未抢购过 3. 商品库存充足	1. 访问商品详情页 2. 点击"立即抢购"按钮 3. 确认订单信息	抢购成功，跳转至订单确认页，库存扣减 1
SALE-007	[边界]活动结束时间准点抢购成功	1. 系统时间刚好为2025 年 4 月 1 日 10:30:00 2. 用户未抢购过 3. 商品库存充足	1. 访问商品详情页 2. 点击"立即抢购"按钮 3. 确认订单信息	抢购成功，跳转至订单确认页，库存扣减 1
SALE-008	[边界]抢购第 100 件商品成功	1. 系统时间在活动时间内 2. 用户未抢购过 3. 商品库存为 1 件	1. 访问商品详情页 2. 点击"立即抢购"按钮 3. 确认订单信息	抢购成功，库存变为 0，后续请求提示"库存已售罄"
SALE-009	[边界]抢购第 101 件商品失败	1. 系统时间在活动时间内 2. 用户未抢购过 3. 商品库存为 0 件	1. 访问商品详情页 2. 点击"立即抢购"按钮	抢购失败，提示"库存已售罄"，按钮置灰
SALE-010	[异常]用户登录状态异常时抢购失败	1. 系统时间在活动时间内 2. 用户未登录或会话过期 3. 商品库存充足	1. 访问商品详情页 2. 点击"立即抢购"按钮	系统提示"请先登录"，跳转至登录页面
SALE-011	[边界]高并发抢购时库存准确性	1. 系统时间在活动时间内 2. 100 个用户同时抢购 3. 商品库存为 100 件	1. 模拟 100 个并发用户同时点击"立即抢购" 2. 监控库存变化	正好 100 个用户抢购成功，库存准确变为 0，无超卖现象
SALE-012	[异常]网络异常时抢购处理	1. 系统时间在活动时间内 2. 用户未抢购过 3. 商品库存充足 4. 模拟网络中断	1. 访问商品详情页 2. 点击"立即抢购"按钮 3. 在网络请求过程中断开网络	系统提示"网络异常，请重试"，抢购失败但库存未错误扣减

表 C-28 需求 3.2 对应测试用例：CoT+Few-shot+模板化提示+Adaptive

编号	用例标题	前置条件	测试步骤	预期结果
TC301	验证用户在活动期间可成功抢购 1 件商品（正向）	1. 系统运行正常，用户 userA@test.com 已登录 2. 活动商品“限量款运动鞋”库存=100 3. 当前时间：2025-04-01 10:05:00（活动进行中） 4. 用户未参与过本次抢购	1. 进入商品详情页 2. 点击“立即抢购”按钮	1. 页面显示“立即抢购”按钮为可点击状态 2. 抢购成功，页面跳转至订单确认页；库存减为 99；系统记录 userA@test.com 已抢购，禁止再次下单
TC302	验证库存售罄后新用户请求被拒绝（反向）	1. 系统运行正常，用户 userB@test.com 已登录 2. 商品库存=0（已售罄） 3. 当前时间：2025-04-01 10:15:00（活动进行中） 4. userB 未抢购过该商品	1. 进入商品详情页 2. 点击“立即抢购”按钮	1. “立即抢购”按钮显示为可点击状态 2. 抢购失败，页面弹出提示“库存已售罄”；按钮变为不可点击状态；无跳转，停留在详情页
TC303	验证同一用户重复抢购被拦截（异常）	1. 系统运行正常，用户 userA@test.com 已登录且已成功抢购 1 件 2. 商品库存=99 3. 当前时间：2025-04-01 10:10:00（活动进行中）	1. 进入商品详情页 2. 点击“立即抢购”按钮	1. “立即抢购”按钮显示为可点击状态 2. 抢购失败，页面提示“您已抢购过该商品”；按钮变为不可点击；提供“查看我的订单”链接，链接可点击并跳转至订单页
TC304	验证活动开始前/结束后按钮状态与拦截（边界）	1. 系统运行正常，用户 userC@test.com 已登录 2. 商品库存=100 3. 准备两个时间点： - T1：2025-04-01 09:59:59（活动开始前 1 秒） - T2：2025-04-01 10:30:01（活动结束后 1 秒）	1. 在 T1 时间进入商品详情页 2. 点击“立即抢购”按钮 3. 在 T2 时间刷新页面 4. 再次点击“立即抢购”按钮	1. 页面显示“立即抢购”按钮为灰色不可点击状态，提示“活动未开始” 2. 点击无响应，无请求发出，页面无变化 3. 页面按钮仍为灰色，提示更新为“活动已结束” 4. 点击无响应，无请求发出，页面无变化
TC305	验证用户在活动结束瞬时提交请求被拦截（状态）	1. 系统运行正常，用户 userD@test.com 已登录 2. 商品库存=5 3. 用户在 10:29:55 进入详情页并停留 4. 系统时间精确至毫秒，活动结束时间为 10:30:00.000	1. 10:29:59.900 点击“立即抢购” 2. 请求因网络延迟于 10:30:00.100 到达服务器	1. 前端按钮在点击瞬间仍为可点击，无本地拦截 2. 服务端拒绝请求，返回“活动已结束”；前端显示该提示；按钮立即置灰；提供“返回首页”链接，链接可点击

续表 C-28

编号	用例标题	前置条件	测试步骤	预期结果
TC306	验证最后 1 件库存被多用户并发抢购（并发）	1. 系统运行正常，用户 userE、userF、userG 同时登录 2. 商品库存=1 3. 所有用户在 10:10:00 同时发起抢购请求	1. 三用户同时点击“立即抢购” 2. 系统处理并发请求	1. 仅 1 位用户（如 userE）抢购成功，跳转订单页，库存=0 2. 其余用户（userF、userG）收到“库存已售罄”提示；按钮置灰；提供“刷新重试”按钮，点击后重新查询库存与活动状态
TC307	验证抢购成功后订单系统与库存系统数据一致性（集成）	1. 系统运行正常，用户 userH@test.com 已登录 2. 商品库存=50 3. 订单系统与库存系统为独立模块，需事务同步	1. 用户成功抢购 1 件商品 2. 立即查询库存系统剩余数量 3. 立即查询订单系统生成记录	1. 抢购成功，跳转订单确认页 2. 库存系统显示剩余 49 件 3. 订单系统生成唯一订单号，商品 SKU、用户 ID、时间戳记录完整，状态为“待支付”

表 C-29 需求 3.2 对应的 QA 打分 1

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	3	1	3
Few-shot	3	4	4	3	3.5
CoT+Few-shot	4	4	5	4	4.25
融合策略	5	5	5	4	4.75

表 C-30 需求 3.2 对应的 QA 打分 2

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	2	1	2.75
Few-shot	4	4	4	3	3.75
CoT+Few-shot	5	5	5	4	4.75
CoT+Few-shot + 模板化提示 + Adaptive	5	5	5	4	4.75

表 C-31 需求 3.2 对应的 QA 打分 3

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	5	3	3	3.75
Few-shot	4	5	4	4	4.25
CoT+Few-shot	4	5	5	5	4.75
融合策略	5	5	4	5	4.75

表 C-32 需求 3.2 对应的 QA 打分 4

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3	4	3	3	3.25
Few-shot	5	4	3	4	4
CoT+Few-shot	5	4	4	5	4.5
融合策略	5	5	4	5	4.75

表 C-33 需求 3.3 对应测试用例：Zero-shot

序号	用例名称	前置条件	测试步骤	预期结果
1	正常登录后选题	1. 用户账号有效 2. 试卷状态为“已发布”	1. 访问系统登录页 2. 输入正确用户名和密码登录 3. 在选题页面选择指定试卷	1. 登录成功 2. 进入选题页面，可看到已发布试卷 3. 选题成功，进入答题页，计时开始
2	登录失败无法选题	1. 用户账号无效或密码错误	1. 访问系统登录页 2. 输入错误用户名或密码 3. 尝试访问选题页面	1. 登录失败，提示错误 2. 无法访问选题页面，重定向到登录页
3	选题时试卷未发布	1. 用户已登录 2. 试卷状态为“未发布”	1. 在选题页面查看试卷列表 2. 尝试选择未发布试卷	1. 未发布试卷不可选或灰显 2. 无法进入答题页
4	答题单题提交	1. 用户已登录并选题 2. 答题页显示题目	1. 回答第一题 2. 点击“提交本题”按钮 3. 尝试修改已提交题目	1. 题目提交成功，标记为已答 2. 已提交题目不可修改
5	答题统一提交	1. 用户已登录并选题 2. 所有题目已回答	1. 回答所有题目 2. 点击“统一提交”按钮	1. 所有题目提交成功 2. 跳转至交卷确认页

续表 C-33

序号	用例名称	前置条件	测试步骤	预期结果
6	断网重连恢复	1. 用户正在答题 2. 断网时间≤5 分钟	1. 在答题过程中断网 2. 等待后重连网络 3. 继续答题	1. 断网时自动保存进度 2. 重连后数据恢复，可继续答题
7	断网超时自动交卷	1. 用户正在答题 2. 断网时间＞5 分钟	1. 在答题过程中断网 2. 等待超过 5 分钟 3. 重连网络	1. 系统视为考试超时 2. 自动交卷，跳转至成绩页
8	主动交卷成功	1. 用户已答题 2. 考试时间未结束	1. 点击“交卷”按钮 2. 确认交卷	1. 弹出确认对话框 2. 确认后交卷成功，跳转至成绩页 3. 成绩存入档案
9	自动交卷超时	1. 用户正在答题 2. 考试时间即将结束	1. 等待考试时间结束前 10 秒 2. 观察提示 3. 等待超时	1. 超时前 10 秒出现提示 2. 超时后强制交卷，跳转至成绩页
10	成绩合格显示	1. 用户交卷 2. 得分≥60 分	1. 交卷后查看成绩页	1. 显示得分≥60 2. 显示“合格” 3. 成绩存入档案
11	成绩不合格显示	1. 用户交卷 2. 得分<60 分	1. 交卷后查看成绩页	1. 显示得分<60 2. 显示“不合格” 3. 成绩存入档案
12	考试时长边界	1.用户已登录并选题 2.试卷时长为 60 分钟	1.开始答题 2. 等待 59 分钟 59 秒后提交一题 3. 等待 60 分钟整	1. 在时间内提交有效 2. 超时后自动交卷
13	PC 端兼容性测试	1.PC 设备，Chrome 120.0+ 2.分辨率≥1366×768	1. 登录系统 2. 完成整个考试流程	1.页面正常显示，无错乱 2.按钮可操作，无遮挡 3. 流程顺利完成
14	移动端 iOS 兼容性	1.iPhone15 系列，iOS 17+ 2.分辨率≥720×1280	1.登录系统 2.完成整个考试流程	1.页面适配移动端，无错乱 2.触摸操作正常，无响应问题
15	移动端 Android 兼容性	1.华为 Mate 60 系列，Android14+ 2. 分辨率≥720×1280	1.登录系统 2. 完成整个考试流程	1.页面适配移动端，无错乱 2. 触摸操作正常，无响应问题

表 C-34 需求 3.3 对应测试用例：Few-shot

编号	用例标题	前置条件	测试步骤	预期结果
EXAM-001	（正常）验证完整考试流程成功	1. 用户已注册 2. 试卷 A 状态为"已发布"，时长为 60 分钟 3. 系统运行正常	1. 用户登录系统 2. 选择试卷 A 进入答题 3. 完成所有题目答题 4. 点击"交卷"并确认 5. 查看成绩页面	1. 登录成功 2. 进入答题页面，计时器开始 3. 答题数据保存成功 4. 跳转至成绩页面，显示得分和合格状态 5. 成绩存入用户档案
EXAM-002	（异常）验证未登录用户无法选题	1. 用户未登录 2. 试卷 A 状态为"已发布"	1. 直接访问选题页面 2. 尝试选择试卷 A	系统跳转至登录页面，提示"请先登录"
EXAM-003	（异常）验证选择未发布试卷失败	1. 用户已登录 2. 试卷 B 状态为"未发布"	1. 进入选题页面 2. 尝试选择试卷 B	提示"该试卷暂未发布"，无法进入答题
EXAM-004	（边界）验证答题时长临界值提交	1. 用户已登录 2. 试卷 A 时长为 60 分钟 3. 用户在 59 分 50 秒时答题	1. 开始答题 2. 在 59 分 50 秒时完成最后答题 3. 点击提交	1. 系统接受提交 2. 正常计算成绩 3. 不触发超时强制提交
EXAM-005	（异常）验证答题超时强制交卷	1. 用户已登录 2. 试卷 A 时长为 60 分钟	1. 开始答题 2. 等待 60 分钟时长结束 3. 观察系统行为	1. 结束前 10 秒出现倒计时提示 2. 超时后自动强制交卷 3. 跳转至成绩页面
EXAM-006	（异常）验证断网 5 分钟内恢复	1. 用户已登录并正在答题 2. 网络连接正常	1. 答题过程中断网 2. 继续答题若干题 3. 4 分 50 秒后恢复网络 4. 继续完成考试	1. 断网期间答题数据本地保存 2. 恢复网络后数据同步成功 3. 可正常完成考试流程
EXAM-007	（异常）验证断网超过 5 分钟考试终止	1. 用户已登录并正在答题 2. 网络连接正常	1. 答题过程中断网 2. 继续答题 3. 等待 5 分 10 秒后恢复网络	1. 系统提示"考试超时" 2. 自动交卷 3. 跳转至成绩页面 4. 已答题部分计分

续表 C-34

编号	用例标题	前置条件	测试步骤	预期结果
EXAM-008	（边界）验证得分 59 分不合格	1. 用户已登录 2. 试卷共 10 题，每题 10 分 3. 用户答对 5 题，答错 5 题	1. 完成答题 2. 提交试卷 3. 查看成绩	显示得分 50 分，状态为"不合格"
EXAM-009	（边界）验证得分 60 分合格	1. 用户已登录 2. 试卷共 10 题，每题 10 分 3. 用户答对 6 题，答错 4 题	1. 完成答题 2. 提交试卷 3. 查看成绩	显示得分 60 分，状态为"合格"
EXAM-010	（兼容）验证 Chrome 120+浏览器兼容性	1. Chrome 浏览器版本 120.0 2. 分辨率 1366×768 3. 用户已登录	1. 完成完整考试流程 2. 检查页面布局和功能	1. 页面布局正常，无文字截断 2. 所有功能可用，按钮可操作 3. 计时器工作正常
EXAM-011	（兼容）验证 iOS 17+移动端兼容性	1. iPhone 15，iOS 17 2. 屏幕分辨率≥720×1280 3. 用户已登录	1. 在移动端完成考试流程 2. 检查触摸操作和显示	1. 页面自适应布局正常 2. 触摸操作流畅 3. 所有功能正常工作
EXAM-012	（兼容）验证 Android 14+移动端兼容性	1. 华为 Mate 60，Android 14 2. 屏幕分辨率≥720×1280 3. 用户已登录	1. 在移动端完成考试流程 2. 检查各项功能操作	1. 页面显示正常 2. 所有功能可用 3. 数据同步正常
EXAM-013	（异常）验证单题提交后不可修改	1. 用户已登录并正在答题 2. 试卷支持单题提交	1. 回答第一题并提交 2. 尝试返回修改第一题答案	提交后的题目变为只读状态，无法修改
EXAM-014	（正常）验证统一提交功能正常	1. 用户已登录 2. 试卷支持统一提交模式	1. 完成所有题目答题 2. 点击"统一提交"按钮 3. 确认提交	所有题目一次性提交成功，跳转至成绩页面

表 C-35 需求 3.3 对应测试用例：CoT+Few-shot

用例编号	用例标题	前置条件	测试步骤	预期结果
LOG-001	[正常]用户名和密码均正确时登录成功	系统运行正常，登录页面已打开	1. 输入已注册的有效用户名 2. 输入正确的密码 3. 点击“登录”按钮	登录成功，跳转至选题页面
LOG-002	[异常]用户名不存在时登录失败	登录页面已打开	1. 输入未注册的用户名 2. 输入任意密码 3. 点击“登录”按钮	登录失败，提示“用户名或密码错误”
LOG-003	[异常]密码错误时登录失败	登录页面已打开	1. 输入已注册的有效用户名 2. 输入错误的密码 3. 点击“登录”按钮	登录失败，提示“用户名或密码错误”
LOG-004	[异常]账户已锁定时登录失败	用户账户已被锁定	1. 输入已被锁定的用户名 2. 输入密码 3. 点击“登录”按钮	登录失败，提示“账户已锁定，请10 分钟后再试”
LOG-005	[边界]密码刚好达到最小长度时登录成功	用户密码长度等于最小规则值	1. 输入有效用户名 2. 输入密码（长度 = 最小规则值） 3. 点击“登录”按钮	登录成功，跳转至选题页面
SEL-001	[正常]选择已发布试卷成功	用户已登录，试卷状态为“已发布”	1. 在选题页面选择状态为“已发布”的试卷 2. 点击“开始答题”按钮	成功进入答题页面，计时器开始计时
SEL-002	[异常]选择未发布试卷时失败	用户已登录，试卷状态为“未发布”	1. 在选题页面选择状态为“未发布”的试卷 2. 点击“开始答题”按钮	提示“该试卷暂未发布，无法答题”
SEL-003	[异常]未登录用户尝试选题	用户未登录	1. 直接访问选题页面 URL	跳转至登录页面，提示“请先登录”
ANS-001	[正常]单题提交成功	用户已进入答题页面	1. 选择一道题的答案 2. 点击“提交本题”按钮	该题提交成功，答案不可修改，系统保存答案
ANS-002	[正常]全部答完后统一提交成功	用户已进入答题页面	1. 依次回答所有题目 2. 点击“交卷”按钮	所有答案提交成功，跳转至交卷确认页面
ANS-003	[异常]断网≤5 分钟重连恢复	答题过程中断网	1. 在答题过程中断网 2. 5 分钟内恢复网络连接	系统自动保存已答题数据，可继续答题

续表 C-35

用例编号	用例标题	前置条件	测试步骤	预期结果
ANS-004	[异常]断网＞5 分钟考试超时	答题过程中断网	1. 在答题过程中断网 2. 超过 5 分钟后恢复网络连接	系统强制交卷，跳转至成绩页面，提示“考试超时”
ANS-005	[边界]答题时长结束前 10 秒提示	答题时长即将结束	1. 等待答题时长结束前 10 秒	系统弹出提示“还剩 10 秒自动交卷”
SUB-001	[正常]主动交卷成功	用户已完成答题	1. 点击“交卷”按钮 2. 在确认弹窗中点击“确认提交”	交卷成功，跳转至成绩页面
SUB-002	[异常]主动交卷取消后继续答题	用户点击交卷但取消	1. 点击“交卷”按钮 2. 在确认弹窗中点击“取消”	返回答题页面，可继续修改答案
SUB-003	[正常]自动交卷超时强制提交	答题时长结束	1. 等待答题时长结束	系统自动交卷，跳转至成绩页面
SUB-004	[边界]交卷时间刚好达到规定时长	答题时长达到规定时长	1. 等待答题时长达到规定时长	系统自动交卷，跳转至成绩页面
RES-001	[正常]成绩合格显示合格	得分≥60 分	1. 交卷后进入成绩页面	显示成绩合格，得分正确计算
RES-002	[正常]成绩不合格显示不合格	得分<60 分	1. 交卷后进入成绩页面	显示成绩不合格，得分正确计算
RES-003	[正常]成绩实时存入用户档案	用户已完成考试	1. 交卷后进入成绩页面 2. 查看用户档案	成绩已存入用户档案，显示最新考试记录
COM-001	[正常]PC 端 Chrome 浏览器显示正常	PC 端 Chrome 120.0 及以上版本	1. 在 Chrome 浏览器中打开答题页面 2. 完成登录、选题、答题、交卷操作	页面显示正常，无按钮遮挡、文字截断，操作响应正常
COM-002	[正常]移动端 iOS APP 显示正常	iOS 17 及以上版本的 iPhone 15 系列	1. 在 iOS APP 中打开答题页面 2. 完成登录、选题、答题、交卷操作	页面显示正常，无布局错乱，操作响应正常
COM-003	[正常]移动端 Android APP 显示正常	Android 14 及以上版本的华为 Mate 60 系列	1. 在 Android APP 中打开答题页面 2. 完成登录、选题、答题、交卷操作	页面显示正常，无布局错乱，操作响应正常
COM-004	[边界]PC 端浏览器窗口分辨率 1366×768	PC 端浏览器窗口分辨率 1366×768	1. 调整浏览器窗口至 1366×768 2. 查看答题页面	页面布局正常，无元素重叠或截断
COM-005	[边界]移动端屏幕分辨率 720×1280	移动端设备屏幕分辨率 720×1280	1. 在分辨率 720×1280 的设备上打开答题页面 2. 查看页面布局	页面布局正常，无元素错乱或截断

表 C-36 需求 3.3 对应测试用例：CoT+Few-shot+模板化提示+Adaptive

编号	用例标题	前置条件	测试步骤	预期结果
TC101	（正常）验证用户可完成完整考试流程（登录→选题→答题→交卷→查看成绩）	1. 系统运行正常，用户 test@exam.com 已注册且未登录 2. 试卷 A 状态为“已发布”，时长 60 分钟，包含 5 道单选题（每题 10 分） 3. 用户使用 Chrome 120.0 浏览器，窗口分辨率 1920×1080	1. 输入 test@exam.com 和密码 123456，点击“登录” 2. 进入选题页，选择试卷 A，点击“开始考试” 3. 进入答题页，依次作答 5 道题并选择“统一提交” 4. 点击“交卷”，在弹窗中点击“确认提交” 5. 查看成绩页显示结果	1. 登录成功，跳转至选题页 2. 成功选择试卷 A，倒计时开始（60 分钟） 3. 所有题目可正常作答，答案可保存 4. 交卷成功，跳转至成绩页 5. 成绩显示为 50 分，状态为“不合格”，成绩已存档
TC102	（异常）验证未登录用户无法进入选题模块	1. 用户未登录 2. 试卷 B 处于“已发布”状态 3. 用户尝试直接访问选题页 URL	1. 在浏览器地址栏输入选题页链接（如 https://exam.com/select?paper=B） 并回车	1. (1）页面重定向至登录页 （2）提示“请先登录以继续考试” （3）登录后可跳转回原选题页
TC103	（异常）验证用户无法选择未发布状态的试卷	1. 用户已登录（test@exam.com） 2. 试卷 C 状态为“草稿”，试卷 D 状态为“已发布” 3. 使用 Chrome 120.0 浏览器	1. 进入选题页 2. 查看可选试卷列表 3. 尝试通过修改请求参数手动请求试卷 C	1. 试卷列表仅显示试卷 D 2. 手动请求试卷 C 时系统拒绝，提示“试卷不存在或未发布” 3. 不允许进入答题页
TC104	（边界）验证答题时长刚好达到规定时长上限（60 分钟）	1. 用户已登录 2. 试卷 E 规定时长为 60 分钟，状态“已发布” 3. 用户进入答题页，系统开始计时	1. 用户开始答题，持续作答至第 60 分钟整 2. 观察第 59 分 50 秒是否弹出“即将超时”提示 3. 第 60 分钟到达时系统自动交卷	1. 第 59 分 50 秒弹出倒计时提示“考试将在 10 秒后结束” 2. 第 60 分钟整系统自动提交试卷 3. (1）跳转至成绩页，已保存当前答题结果 （2）成绩按正确题数计算并存档

续表 C-36

编号	用例标题	前置条件	测试步骤	预期结果
TC105	（状态）验证断网 5 分钟内恢复后可继续答题	1. 用户已登录并进入试卷 F 答题页 2. 已作答 2 道题，剩余 3 道未答 3. 网络连接正常，倒计时进行中	1. 手动断开网络连接 2. 等待 4 分 30 秒后恢复网络 3. 刷新页面或继续操作	1. 断网期间页面提示“网络异常，正在重连” 2. 已答题数据自动保存 3. (1) 恢复连接后可继续答题，倒计时继续 （2） 页面未强制交卷，操作正常
TC106	（状态）验证断网超过 5 分钟导致考试终止	1. 用户已登录并进入试卷 G 答题页 2. 已作答 1 道题，剩余 4 道 3. 网络正常，倒计时开始	1. 手动断开网络连接 2. 等待 5 分 30 秒后恢复网络并刷新页面	1. 断网期间提示“网络断开” 2. (1）超过 5 分钟后系统自动交卷 （2）页面跳转至成绩页，仅已提交题目计入得分 （3）提示“考试因超时自动提交”
TC107	（并发）两个用户同时提交同一试卷的最后答案	1. 试卷 H 为公共试卷，用户 U1 和 U2 均已登录 2. 两人同时进入答题最后 1 题，剩余时间 10 秒 3. 使用不同设备，Chrome 120.0 浏览器	1. U1 和 U2 在第 59 分 55 秒同时点击“统一提交”按钮 2. 系统在第 60 分钟处理交卷请求	1. 两人均能提交成功，系统支持并发交卷 2. (1）各自成绩独立计算并保存 （2）无数据覆盖或错乱
TC108	（集成）答题数据从答题模块正确传递至成绩统计模块	1. 用户完成试卷 I 的 5 道题(答对 4 题) 2. 试卷每题 10 分，合格线 60 分 3. 用户主动确认交卷	1. 用户点击“交卷” 2. 在确认弹窗中点击“确定” 3. 进入成绩页查看结果	1. 成绩页显示得分 40 分 2. 状态为“不合格” 3. (1）用户档案中该次考试记录可查，得分与答题明细一致 (2)数据库中成绩表新增一条记录
TC109	（集成）选题成功后答题倒计时准确启动并与试卷绑定	1. 试卷 J 设置时长为 30 分钟，状态“已发布” 2. 用户已登录	1. 用户选择试卷 J 并点击“开始考试” 2. 进入答题页，观察倒计时是否从 30:00 开始 3. 等待 1 分钟后，检查倒计时是否变为 29:00	1. 倒计时正确显示为 30:00 并开始递减 2. 时间与服务器同步，无跳变或停滞 3. 每分钟递减正常，精度在±1 秒内

续表 C-36

编号	用例标题	前置条件	测试步骤	预期结果
TC110	（兼容）验证 PC 端主流浏览器（Chrome 120.0） 功能与布局正常	1. 用户使用 Chrome 120.0 浏览器 2. 窗口分辨率 1366×768 3. 试卷 K 已发布，60 分钟，5 题	1. 登录系统 2. 选题、答题、交卷、查看成绩	1. 系统登录成功 2. (1）所有按钮可见且可点击文字无截断，页面布局完整 （2）倒计时、提交、跳转等功能正常 （3）无样式错乱或功能失效
TC111	（兼容）验证移动端 iOS 17(iPhone 15) 设备操作正常	1. 用户使用 iPhone 15 运行 iOS 17.0 2. 分辨率 1179×2556，使用官方考试 APP v2.3.0 3. 试卷 L 已发布	1. 打开 APP，登录账号 2. 选择试卷 L 并开始答题 3. 作答所有题目后交卷	1. 登录页、选题页、答题页布局适配良好 2. 触控按钮响应正常，无遮挡 3. (1）交卷成功，成绩页正常显示 （2）无白屏、卡顿或闪退
TC112	（兼容）验证移动端 Android 14（华为 Mate 60）设备兼容性	1. 用户使用华为 Mate 60 运行 Android 14 2. 分辨率 1200×2640，考试 APP v2.3.0 3. 试卷 M 已发布	1. 启动 APP，登录账号 2. 选择试卷 M，进入答题页 3. 完成答题并主动交卷	1. 页面元素完整显示，字体清晰 2. 滚动、点击、提交操作流畅 3. (1）交卷后成绩页加载正常 （2）无兼容性报错或功能缺失

表 C-37 需求 3.3 对应的 QA 打分 1

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	3	1	3
Few-shot	3	4	4	2	3.25
CoT+Few-shot	4	4	4	3	3.75
融合策略	5	5	5	5	5

表 C-38 需求 3.3 对应的 QA 打分 2

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	2	1	2.75
Few-shot	4	4	4	3	3.75
CoT+Few-shot	4	4	4	4	4
融合策略	5	5	5	4	4.75

表 C-39 需求 3.3 对应的 QA 打分 3

提示工程技术	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	3	4	4	3	3.5
Few-shot	3	4	5	4	4
CoT+Few-shot	5	4	5	4	4.5
融合策略	5	5	4	5	4.75

表 C-40 需求 3.3 对应的 QA 打分 4

提示策略	正确性	可执行性	完备性	创新性	综合得分
Zero-shot	4	4	3	3	3.5
Few-shot	4	4	4	3	3.75
CoT+Few-shot	2	4	4	3	3.25
融合策略	5	5	5	5	5

致　谢

“独学而无友，则孤陋而寡闻。”在成长的旅途上，正是师友们的无私帮助，让我得以避开弯路，行稳致远。当这篇论文画上句点，回望从选题、探索到写作的日日夜夜，我心中充满了无尽的感激。

首先，承蒙我的导师罗熊老师的悉心指导。从研究方向的把握到具体难题的攻克，他始终倾囊相授，以其深厚的学养为我拨云见日。在此，谨向罗老师致以最崇高的敬意与感谢。

论文写作并非易事，字斟句酌的过程充满了挑战。幸有同窗挚友，在日常闲谈中互相疏解压力。那些看似平淡的陪伴，是这段岁月里最珍贵的礼物。

此外，还要感谢我的家人，他们始终是我最坚实的后盾。

最后，感谢学校的培养。校园内良好的学习氛围与优美的环境，是这段旅程中最宝贵的背景。谨向所有关心、帮助过我的师友，呈上我最衷心的感谢！

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2015 年 9 月至 2019 年 6 月	在北华大学计算机科学与技术专业攻读学士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

（1）荣获北京科技大学 2026 届优秀毕业生

四、在学期间发表的论文

独创性说明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本文所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作 者 签 名 ：_____ 日期：2025 年 12 月 25 日

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名： 导师签名： 日期：__2025.12.25__

学位论文数据集

关键词*	密级*	中图分类号*	UDC	论文资助
测试用例自动生成；大语言模型；提示工程；QLoRA 微调	公开	TP311		
学位授予单位名称*		学位授予单位代码*	学位类别*	学位级别*
北京科技大学		10008	工学	硕士
论文题名*		并列题名		论文语种*
基于大语言模型的测试用例自动生成技术研究及系统实现				中文
作者姓名*	曹倩		学号*	M202340074
培养单位名称*		培养单位代码*	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学院路 30 号	100083
学科专业*		研究方向*	学制*	学位授予年*
软件工程		大模型应用；软件测试	2.5 年	2026 年
论文提交日期*	2025 年 12 月 25 日			
导师姓名*	罗熊		职称*	教授
评阅人	答辩委员会主席*		答辩委员会成员	
	刘宇		汪红兵、王洪泊、陈媛、王宗杰	
电子版论文提交格式　文本（√）　图像（ ）　视频（ ）　音频（ ）　多媒体（ ）　其他（ ） 推荐格式：application/msword; application/pdf				
电子版论文出版（发布）者		电子版论文出版（发布）地		权限声明
论文总页数*	157			
共 33 项，其中带*为必填数据，为 22 项。				